Universal Feature Learning Dynamics Revealed by Self-Averaging Observables

Moritz Rautenberg



Munich 2026

Universal Feature Learning Dynamics Revealed by Self-Averaging Observables

Moritz Rautenberg

Master's Thesis

Faculty of Physics
Ludwig–Maximilians–Universität München

submitted by
Moritz Rautenberg
born in Hamburg

Supervised by
Pascal de Jong
Daniel Pals
Prof. Dr. Steffen Rulands

Munich, 10th August 2026

Durch selbstmittelnde Observablen aufgedeckte Dynamiken des universellen Feature-Lernens

Moritz Rautenberg



Munich 2026

Durch selbstmittelnde Observablen aufgedeckte Dynamiken des universellen Feature-Lernens

Moritz Rautenberg

Masterarbeit

Fakultät für Physik
Ludwig–Maximilians–Universität München

eingereicht von
Moritz Rautenberg
geboren in Hamburg

Betreut von
Pascal de Jong
Daniel Pals
Prof. Dr. Steffen Rulands

München, 10 August 2026

Abstract

Feature learning describes the ability of a neural network to independently construct transformed representations from data and constitutes the central property responsible for the universal applicability and the success of deep learning. Whether this ability is preserved as a network's width is increased is determined by the employed abc-parametrization. This thesis therefore compares the standard parametrization (SP) and the maximal-update parametrization (μP) on a one-hidden-layer MLP, which represents a minimal architecture that exhibits feature learning. Based on the empirical neuron distribution, the width universality of any neuron-wise observable under μP is explained, as well as the systematic width dependence observed under SP. Since feature learning is exposed by an evolution of the empirical neuron distribution, the same framework makes it directly measurable as correlations between hidden representations. Under μP these correlations span a low-dimensional subspace that carries almost the entire predictive performance of the network, which allows the model to be reconstructed from a small fraction of its parameters at essentially unchanged generalization performance. Increasing the width does not create this structure but reduces the sampling noise that obscures it at finite width, so that a wider network gradually reveals more of the fixed feature hierarchy of the infinite-width limit. Under SP feature learning instead deteriorates with width and the same reconstruction fails, which identifies feature learning as the mechanism through which an increased width translates into improved generalization performance in the investigated setup.

Contents

1 Introduction **1**

2 Deep Learning Theory **3**
2.1 Deep Learning vs. Classical Machine Learning 3
2.2 Multilayer Perceptrons 5
2.3 Loss Function and Training Objective 8
2.4 Optimization via Gradient Descent 10

3 abc-Parametrizations **13**
3.1 Feature Learning in a One-Hidden-Layer MLP 13
3.1.1 Standard Parametrization (SP) 15
3.1.2 Maximal-Update Parametrization (μP) 16
3.1.3 Comparison of SP and μP 16
3.2 Feature Learning in the Infinite-Width Limit 19
3.3 Experimental Setup 21

4 Tensor Programs Framework **23**
4.1 SGD Training Dynamics of an MLP under μP 23
4.2 Deterministic Training Dynamics and Mean-Field Limit 29
4.3 Implications for Feature Learning 31

5 Results and Discussion **35**
5.1 Universality Across Training Observables under μP 35
5.2 Width-Dependent Deviations under SP 42
5.3 A Spectral Perspective on Feature Learning 46
5.3.1 Dimensionality Reduction Through PCA 58

6 Summary and Conclusion **63**
6.1 Summary and Conclusion 63
6.2 Limitations and Outlook 66

A Appendices **67**
A.1 Feature Learning in a One-Hidden-Layer MLP 67
A.1.1 Standard Parametrization (SP) 68
A.1.2 Maximal-Update Parametrization (μP) 73
A.1.3 Why the Random Signed Sum Remains of Order One 75
A.2 Generalization of Layer-Wise Update Norms 77

A.3 Coordinate-Distribution Analysis of SP 80
A.3.1 Setup and Coordinate Distributions at Initialization 80
A.3.2 First Forward Pass ($t = 0$) 80
A.3.3 First Backward Pass ($t = 0$) 81
A.3.4 One SGD Step and the Scale Mismatch 81
A.3.5 Scaling Bookkeeping: Why $\boldsymbol{f}_t$ Is Not a Single Expectation . . . 82

Acknowledgements **85**

Bibliography **87**

1 Introduction

Artificial intelligence (AI) has attracted a lot of attention in research but also in everyday life in recent years. This impact is largely attributable to advances in deep learning, a subfield of AI that comprises all machine learning models based on neural networks, with applications ranging from quantum simulations over computer vision to language modelling [1–6]. Despite this progress, a mechanistic understanding of deep neural networks remains a distant goal. Neural networks still constitute black boxes with respect to the principles that underlie their capabilities, and the field of mechanistic interpretability is concerned with identifying the core concepts responsible for their expressive power [7].

Machine learning models are optimized on training data in order to predict the correct label for data samples that the model has not encountered during training. To this end, the model does not evaluate the raw input but transformed representations of the input data sample. These transformed representations of an input sample are called features and are generally obtained through a non-linear transformation of the input components that exposes structure which is not discernible in the raw data format [8]. Classical machine learning methods rely on features that are designed manually, which requires at least a partial understanding of the structure of the data. Neural networks instead construct their features autonomously during optimization, which is referred to as feature learning [9–12]. This ability constitutes the central property of neural networks. The knowledge about the structure of the data that is required to generalize is no longer manually designed but extracted by the model itself during training, which allows neural networks to be applied to essentially any problem setup.

Complex tasks that contain high-dimensional data require large network architectures, since a larger number of trainable parameters generally provides the model with a richer set of features that can be optimized during training [13]. This is supported by the empirical observation that the performance of neural networks improves predictably over many orders of magnitude when model size, dataset size, and compute are scaled jointly, which is described by neural scaling laws [14–17]. These scaling laws are, however, purely empirical, and the mechanism from which they originate has not been identified so far. Moreover, the predicted performance gain is realized only under specific conditions on the optimization procedure [18], which is controlled by the set of hyperparameters [19]. Recent work [9, 10] has shown that not only the size of the network but also the extent to which its features are adapted during training determines the training dynamics and the performance gain that can be expected from a larger model. Each node of a layer computes one feature, so that the width of a network defines the number of available features [9]. The question of how feature learning

determines the performance of a network therefore becomes a question about the width dependence of the training dynamics [9]. In this context, it remains unclear by which mechanism the improved generalization of larger networks arises from the features that the network actually builds. Answering this requires understanding how the additional capacity of a larger model is allocated at the microscopic level of individual features. It is, for example, not known whether larger models build more features or instead use their higher-dimensional hidden space to build better ones, nor whether these two possibilities can be distinguished in general. The framework of abc-parametrizations describes a model's feature-learning capacity depending on the width of the network and thereby provides the theoretical setting for the research question addressed in this thesis:

How does the abc-parametrization determine the evolution of hidden representations during training, and how is this reflected in generalization performance as model width increases?

Answering this question first requires a precise definition of a feature for the MLP architecture studied in this thesis. Since no general consensus on the definition of a feature in neural networks exists [9, 12, 20], this work adopts the definition of [9], which identifies features with the hidden-layer activations of an MLP. Feature learning is then defined as these activations changing by $O(1)$ during training as the width tends to infinity [9]. The thesis is structured as follows. Chapter 2 introduces the MLP architecture together with the utilized optimization algorithm, and states the definition of a feature that will be used throughout the thesis. Chapter 3 introduces the abc-parametrizations and compares the standard parametrization (SP) and the maximal-update parametrization (μP) with respect to their implications for feature learning [9]. On this basis the Tensor Programs Framework (TPF) is introduced in Chapter 4, where the weights of the network are described as random variables, which allows the training dynamics under SP and μP to be compared at finite width as well as in the infinite-width limit. The permutation symmetry of the architecture is subsequently exploited to define the empirical neuron distribution [21, 22], which replaces the fixed labelling of the hidden neurons by a description of the network as a population of neurons rather than as an ordered set. This perspective explains how macroscopic observables such as the test loss or the layer-wise update norm become self-averaging in the infinite-width limit, and how fluctuations of the individual weights sampled at initialization arise at finite width. Chapter 5 finally complements this description by a spectral perspective on feature learning, which characterizes the learned representations through correlations and relates the self-averaging observables to the organization of features into collective modes.

2 Deep Learning Theory

Machine learning (ML) is a subfield of artificial intelligence that refers to a broad class of methods, which are capable of learning patterns from data to make predictions or decisions without being explicitly programmed for each task. In this setting, a dataset

$$\mathcal{D} := \left\{ (\boldsymbol{x}_i, \boldsymbol{y}_i) \,\middle|\, \boldsymbol{x}_i \in \mathbb{R}^d,\ \boldsymbol{y}_i \in \mathbb{R}^p \right\}_{i=1\ldots M} \tag{2.1}$$

consists of M input-label pairs $(\boldsymbol{x}_i, \boldsymbol{y}_i)$, where each multivariate input sample $\boldsymbol{x}_i$ is associated with a label $\boldsymbol{y}_i$. A supervised-learning model defines a parameterized mapping

$$\boldsymbol{f}_\theta \colon \mathbb{R}^d \longrightarrow \mathbb{R}^p, \tag{2.2}$$

where θ denotes the trainable model parameters. The objective is to adjust these parameters such that the prediction $\boldsymbol{f}_\theta(\boldsymbol{x}_i)$ approximates the corresponding label $\boldsymbol{y}_i$ and generalizes to input samples that were not used during training. The quality of the model's predictions is quantified by an objective function, commonly referred to as the loss function, which measures the prediction error across the dataset.

ML methods are widely used in the analysis of physical data across research fields such as particle physics, cosmology, solid-state physics, and statistical physics [23]. Advances in the theoretical foundations and practical concepts of machine learning, and deep learning in particular, are therefore relevant across a broad range of research areas in physics, while also extending beyond physics.

This thesis follows the notation used in the textbook *Deep Learning for Physics Research* [23]. The textbook provides a general introduction to deep learning using examples and analogies from physics, making it particularly suitable for readers with a background in physics. Using the same notation should make it easier to connect the concepts introduced in this thesis to the more detailed explanations provided in the textbook.

2.1 Deep Learning vs. Classical Machine Learning

The following section classifies different types of Machine Learning that are all subfields of artificial intelligence based on their feature learning capabilities. Conceptually, features will be introduced in the following where Machine-learning methods are organ-

ized according to this concept, as it represents the integral property of deep learning that will be investigated in this thesis. As illustrated in Figure 2.1, machine learning is commonly viewed as a subfield of artificial intelligence. Representation learning forms a subfield of machine learning, and deep learning in turn forms a subfield of representation learning [23]. These distinctions are conceptual rather than absolute, but they provide a useful framework for understanding how different models process data.
Many classical machine-learning methods rely heavily on features that are selected or designed prior to optimizing the model for a given task. Given an input vector $\boldsymbol{x} \in \mathbb{R}^d$, a feature can be represented as a function $\phi_k(\boldsymbol{x})$ that extracts or combines information from the input components. A collection of such features defines a hidden representation

$$\boldsymbol{\phi}(\boldsymbol{x}) = (\phi_1(\boldsymbol{x}), \ldots, \phi_K(\boldsymbol{x}))^T , \tag{2.3}$$

that must be manually designed and tailored to the specific task. The choice of these features is typically informed by prior knowledge of the dataset and the underlying task that should be solved by the ML model. Linear regression, logistic regression, symbolic regression, and kernel methods are examples of classical approaches for which the choice of input representation can strongly influence the quality of the predictions [23].
Representation-learning methods reduce the dependence on manually designed features by learning useful transformations of the input data during training. The learned representation is adjusted according to the optimization objective and can therefore capture structures that may not be evident in the original input variables. Principal component analysis is a simple example of representation learning, as it identifies directions of large variance in the data and uses them to construct a lower-dimensional representation. Although principal component analysis does not use labels and does not directly solve a prediction task, it illustrates the general idea that informative representations can be inferred solely from data instead of being designed manually [23].

Deep-learning methods extend this principle by learning multiple layers of representations. Each layer transforms the representation produced by the preceding layer, allowing the model to construct increasingly abstract features from the original input variables. Through this hierarchical composition, microscopic information contained in the individual input components can be combined into representations that are useful for predicting the correct output label. This layered structure of learned representations is a defining characteristic of deep learning.

Deep-learning architectures have been applied successfully to a wide range of data types. Examples include multilayer perceptrons (MLPs), convolutional neural networks, recurrent neural networks, and transformer architectures. Transformers form the foundation of modern large language models and are a prominent example of the capabilities of deep learning. However, a comprehensive theoretical description of the principles governing deep-learning models is still an active area of research. This thesis

therefore focuses on MLPs, which provide a particularly clear architecture for studying the learning of hierarchical representations, which is the underlying principle of the latest remarkable advancements in the field of deep learning [12, 24, 25].

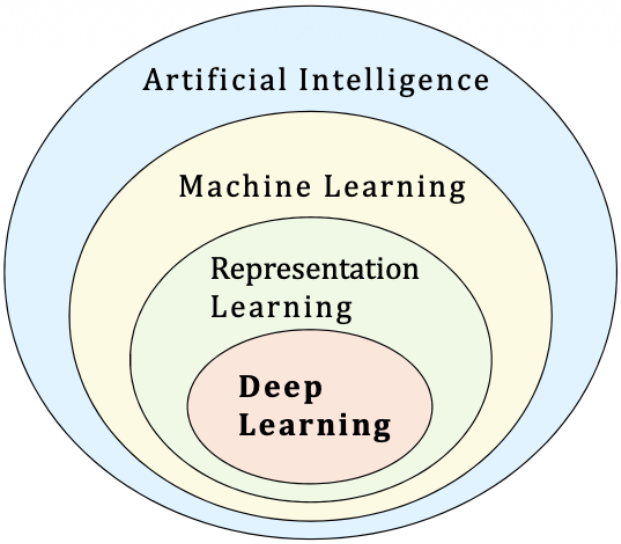


Figure 2.1: *Machine learning is a subfield of artificial intelligence (AI). Representation learning is a subcategory of machine learning, and deep learning is a subcategory of representation learning. Taken from [23].*

2.2 Multilayer Perceptrons

A multilayer perceptron (MLP) is a fully connected feed-forward neural network. It consists of an input layer, one or more hidden layers, and an output layer. Let $\boldsymbol{z}^l$ denote the vector of neuron activations in layer l, and let n_l denote the number of neurons in this layer. The input layer is defined as

$$\boldsymbol{z}^0 := \boldsymbol{x} \in \mathbb{R}^d, \qquad n_0 = d, \tag{2.4}$$

while the output layer satisfies

$$\boldsymbol{z}^{L+1} \in \mathbb{R}^p, \qquad n_{L+1} = p. \tag{2.5}$$

The network contains L hidden layers, represented by the activation vectors

$$\left\{\boldsymbol{z}^l\right\}_{l=1}^{L}. \tag{2.6}$$

In an MLP, every neuron is connected to every neuron in the adjacent layers.

The mapping from layer $l-1$ to layer l is given by

$$\boldsymbol{z}^{\,l} = \sigma^{l-1}\left(W^{l-1}\boldsymbol{z}^{\,l-1} + \boldsymbol{b}^{l-1}\right), \qquad l = 1, \ldots, L+1, \tag{2.7}$$

where

$$W^{l-1} \in \mathbb{R}^{n_l \times n_{l-1}} \tag{2.8}$$

is the weight matrix and

$$\boldsymbol{b}^{l-1} \in \mathbb{R}^{n_l} \tag{2.9}$$

is the bias vector associated with the mapping between the two layers. The expression inside the parentheses in Equation 2.7 defines an affine transformation. The activation function σ^{l-1} is subsequently applied component-wise and introduces the nonlinearity required for the network to learn complex mappings. The activation function used in the output layer may differ from those used in the hidden layers, depending on the learning task, where usually the same activation function is used across all hidden layers. Common choices for hidden-layer activation functions include the rectified linear unit (ReLU),

$$\mathrm{ReLU}(x) = \max\left(0, x\right), \tag{2.10}$$

and smooth alternatives such as the hyperbolic tangent. Nonlinear activation functions are essential because a composition of purely affine transformations is itself affine and would therefore not gain additional expressive power from further hidden layers. For regression tasks, the output layer is chosen to be linear. For classification tasks, the output logits are converted into normalized class probabilities using the softmax function introduced in Section 2.3.

Composing the mappings between consecutive layers yields the complete neural network,

$$\boldsymbol{f}_\theta(\boldsymbol{x}) := \boldsymbol{z}^{\,L+1} = \sigma^{L}\left(W^{L}\boldsymbol{z}^{\,L} + \boldsymbol{b}^{\,L}\right) \in \mathbb{R}^{p}, \tag{2.11}$$

where θ denotes the collection of all trainable weights and biases. Under this convention, the MLP contains L hidden layers between the input layer $\boldsymbol{z}^{\,0}$ and the output layer $\boldsymbol{z}^{\,L+1}$.

Figure 2.2 illustrates a forward pass through a small neural network with one hidden layer containing two neurons. At each neuron, an affine transformation is followed by the application of an activation function. Figure 2.3 shows a more representative MLP architecture with three hidden layers. The same sequence of operations is applied throughout the wider network, although the individual weights and neurons are not labeled explicitly for the sake of clarity.

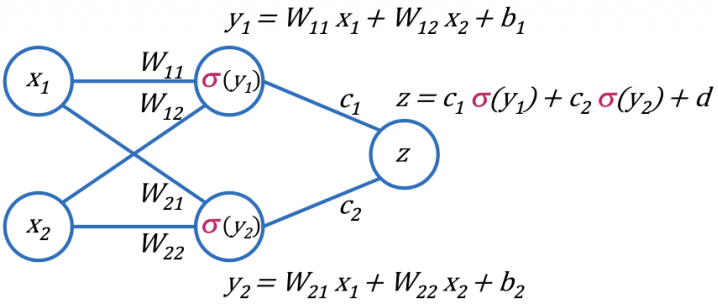


Figure 2.2: *The forward pass through a neural network consists of two operations at each neuron: (1) an affine transformation and (2) the application of an activation function σ. Taken from [23].*

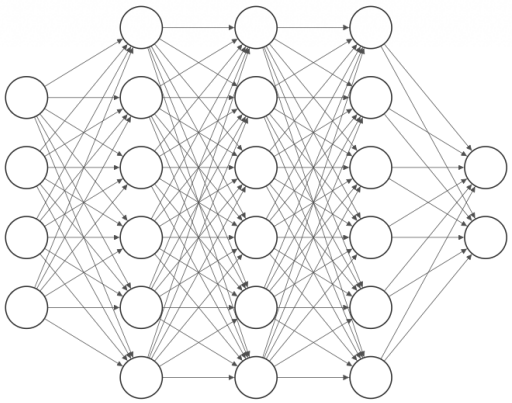


Figure 2.3: *MLP with three hidden layers, each comprising six neurons, that maps a four-dimensional input sample to a two-dimensional output. Taken from [23].*

2.3 Loss Function and Training Objective

After the network architecture has been specified, the trainable parameters θ, consisting of all weight matrices and bias vectors, are initialized before training. The purpose of the loss function is to quantify the deviation between the network prediction $\boldsymbol{f}_\theta(\boldsymbol{x}_i)$ and the corresponding target label $\boldsymbol{y}_i$. During training, the architecture of the network remains fixed, while the values of the trainable weights and biases are iteratively adjusted to minimize the training loss.
For a regression task, the target label is continuous. The sample-wise squared-error loss is defined as

$$L_{\text{reg}}\left(\boldsymbol{x}_i, \boldsymbol{y}_i; \theta\right) = \left\|\boldsymbol{f}_\theta(\boldsymbol{x}_i) - \boldsymbol{y}_i\right\|_2^2 . \tag{2.12}$$

For a scalar regression task, Equation 2.12 reduces to the squared difference between the predicted and true values.
For a classification task, each input sample is assigned to one of N_c discrete classes. The target labels can be represented by one-hot encoded vectors. For a sample belonging to class c, the corresponding component of the target vector is equal to one, while all other components are equal to zero. Thus,

$$p_{i,c} \in \{0,1\}, \qquad \sum_{c=1}^{N_c} p_{i,c} = 1. \tag{2.13}$$

The final affine transformation of the neural network produces the logits

$$\boldsymbol{h}(\boldsymbol{x}_i) = W^L \boldsymbol{z}^L(\boldsymbol{x}_i) + \boldsymbol{b}^L. \tag{2.14}$$

The softmax function converts these logits into normalized class probabilities,

$$q_{i,c} = \frac{\exp\left(a_c(\boldsymbol{x}_i)\right)}{\sum_{c'=1}^{N_c} \exp\left(a_{c'}(\boldsymbol{x}_i)\right)}, \qquad c = 1, \ldots, N_c. \tag{2.15}$$

The sample-wise cross-entropy loss is then given by

$$L_{\text{class}}\left(\boldsymbol{x}_i, \boldsymbol{y}_i; \theta\right) = -\sum_{c=1}^{N_c} p_{i,c} \log\left(q_{i,c}\right). \tag{2.16}$$

For both regression and classification tasks, the empirical loss over a dataset $\mathcal{S} \subseteq \mathcal{D}$ can be written as

$$\mathcal{L}_{\mathcal{S}}(\theta) = \frac{1}{|\mathcal{S}|} \sum_{(\boldsymbol{x}_i, \boldsymbol{y}_i) \in \mathcal{S}} L\left(\boldsymbol{x}_i, \boldsymbol{y}_i; \theta\right), \tag{2.17}$$

where $\mathcal{L}(\boldsymbol{x}_i, \boldsymbol{y}_i; \theta)$ denotes the appropriate sample-wise loss for the task under consideration.

The objective of training is not only to minimize the loss for the samples used during optimization but also to achieve accurate predictions for previously unseen data. This capability is referred to as generalization. In a typical machine-learning setup, the complete dataset is divided into three pairwise disjoint subsets,

$$\mathcal{D} = \mathcal{D}_{\text{tr}} \dot{\cup} \mathcal{D}_{\text{val}} \dot{\cup} \mathcal{D}_{\text{te}}, \tag{2.18}$$

where $\mathcal{D}_{\text{tr}}$, $\mathcal{D}_{\text{val}}$, and $\mathcal{D}_{\text{te}}$ denote the training, validation, and test sets, respectively. The training set is used to update the model parameters. The validation set is commonly used to select hyperparameters, compare model configurations, or determine an appropriate stopping point. Hyperparameters are parameters that allow to guide the optimization process itself and do not represent trainable parameters as the weight matrices that are continuously updated in each optimization step. The test set is reserved for evaluating the generalization capability of the final model.
In the experiments conducted in this thesis, no hyperparameters are tuned based on a validation dataset. It is therefore sufficient to divide the complete dataset into two pairwise disjoint subsets,

$$\mathcal{D} = \mathcal{D}_{\text{tr}} \dot{\cup} \mathcal{D}_{\text{te}}. \tag{2.19}$$

The training set $\mathcal{D}_{\text{tr}}$ is used to optimize the model parameters, whereas the test set $\mathcal{D}_{\text{te}}$ is used only to monitor the model's generalization capability. In particular, the test data do not influence the parameter updates, the choice of hyperparameters, or any other decisions made during training. A model is said to overfit when it achieves a low training loss but performs substantially worse on previously unseen data. In this case, the model has adapted too strongly to properties of the training samples that do not generalize to the underlying data distribution. Comparing the training and test losses provides a basic diagnostic for this behavior.
The corresponding losses are

$$\mathcal{L}_{\text{tr}}(\theta) = \frac{1}{|\mathcal{D}_{\text{tr}}|} \sum_{(\boldsymbol{x}_i, \boldsymbol{y}_i) \in \mathcal{D}_{\text{tr}}} L\left(\boldsymbol{x}_i, \boldsymbol{y}_i; \theta\right), \tag{2.20}$$

and

$$\mathcal{L}_{\text{te}}(\theta) = \frac{1}{|\mathcal{D}_{\text{te}}|} \sum_{(\boldsymbol{x}_i, \boldsymbol{y}_i) \in \mathcal{D}_{\text{te}}} L\left(\boldsymbol{x}_i, \boldsymbol{y}_i; \theta\right). \tag{2.21}$$

Only the training loss in Equation 2.20 is considered to compute parameter updates. The optimization procedure used to minimize this loss is introduced in the following subsection.

2.4 Optimization via Gradient Descent

During training, the weights and biases of the neural network are iteratively adjusted to reduce the training loss. The complete set of trainable parameters at optimization step t is collected in a parameter vector θ_t. Gradient descent updates this vector in the direction of steepest decrease of the loss with respect to the parameters. For the training loss $\mathcal{L}_{\text{tr}}$, the parameter update is defined as

$$\theta_{t+1} = \theta_t + \Delta\theta_t, \tag{2.22}$$

with

$$\Delta\theta_t = -\eta \nabla_{\theta_t} \mathcal{L}_{\text{tr}}(\theta_t), \tag{2.23}$$

where $\eta > 0$ denotes the learning rate. The gradient provides the direction of steepest increase of the loss at the current parameter configuration. Its negative therefore determines the direction of steepest local decrease. The learning rate controls the magnitude of the update step in parameter space.

Gradient descent is a first-order optimization method because the update is based on the local first-order approximation of the loss landscape. For sufficiently small learning rates, the parameter updates follow the local geometry of the loss landscape closely. However, very small learning rates can result in slow optimization, whereas excessively large learning rates can lead to unstable training or prevent convergence. Figure 2.5 illustrates how the choice of the learning rate affects both the trajectory through parameter space and the evolution of the training loss.

The gradients of the loss with respect to the network parameters are computed efficiently using backpropagation. Backpropagation applies the chain rule recursively, starting from the output layer and proceeding through the preceding layers of the network. In this way, the derivatives with respect to all weight matrices and bias vectors can be calculated. The procedure is illustrated schematically in Figure 2.4.

Computing the exact gradient of the empirical training loss requires evaluating all samples in the training dataset before each parameter update. For large datasets, this procedure can be computationally expensive. In practice, the gradient is therefore commonly approximated using a mini-batch

$$B \subseteq \mathcal{D}_{\text{tr}} \tag{2.24}$$

at optimization step t. The mini-batch loss is defined as

$$\mathcal{L}_B(\theta_t) = \frac{1}{|B|} \sum_{(\boldsymbol{x}_i, \boldsymbol{y}_i) \in B} \mathcal{L}\left(\boldsymbol{x}_i, \boldsymbol{y}_i; \theta_t\right). \tag{2.25}$$

The corresponding parameter update is

$$\Delta\theta_t = -\eta \nabla_{\theta_t} \mathcal{L}_B(\theta_t). \tag{2.26}$$

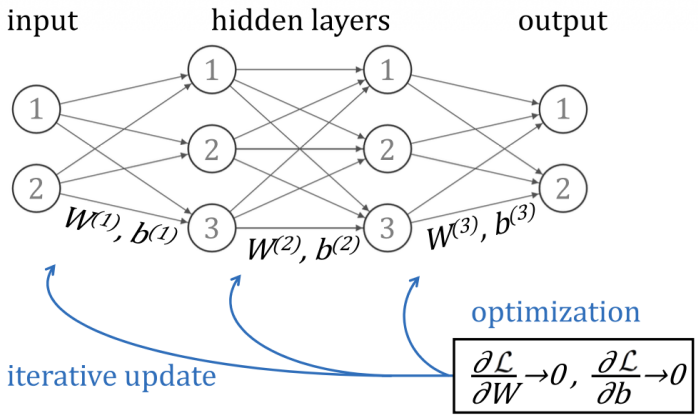


Figure 2.4: *Optimization of the network parameters through iterative updates that reduce the objective function $\mathcal{L}$. Backpropagation is used to compute the partial derivatives with respect to the weights and biases. Taken from [23].*

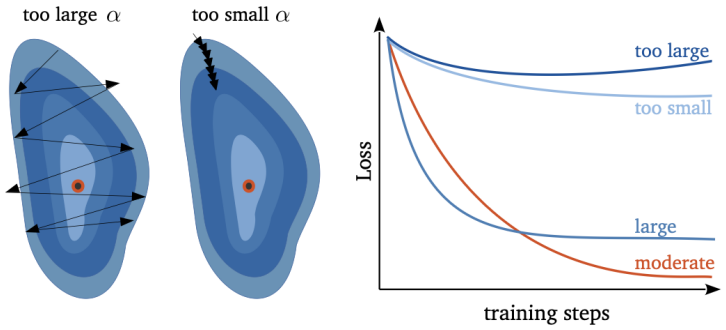


Figure 2.5: *Illustration of the effect of different learning rates on the update steps in parameter space (black arrows) on the left-hand side and on the evolution of the training loss on the right-hand side. Taken from [23].*

Because each mini-batch contains only a subset of the training samples, the resulting gradient is generally a noisy estimate of the full-batch gradient. This stochasticity reduces the computational cost per update and can also influence the optimization dynamics. This procedure is referred to as stochastic gradient descent (SGD). Full-batch gradient descent is recovered when

$$B = \mathcal{D}_{\mathrm{tr}}, \tag{2.27}$$

whereas an update based on a single sample corresponds to SGD in the strictest sense. To ensure that the training samples are considered without introducing a fixed ordering bias, the training dataset is typically shuffled and divided into mini-batches. These mini-batches are then processed consecutively. One complete pass through the training dataset is called an epoch.
More advanced optimization algorithms, such as Adam, incorporate additional information from consecutive update steps. These methods are not introduced here because only SGD is used in the experiments conducted in this thesis.

3 abc-Parametrizations

The ability of deep neural networks to adapt their internal representations of the input data during training is referred to as feature learning. As discussed in the previous chapter, this ability is the central advantage of deep learning over many classical machine-learning methods, where the quality of the prediction often depends strongly on a manually designed fixed set of input features. In a neural network, by contrast, features are represented by the hidden activations of each layer and are adjusted through the optimization process.
This chapter introduces how feature learning during the optimization of the network's parameters depends on the initialization of the trainable parameters and the learning rate that is applied in each training step. This introduction into feature learning focuses on a one-hidden-layer MLP and compares the standard parametrization (SP) with the maximal-update parametrization μP, where the two different parameterizations will be introduced in the following sections. The central distinction between the two parameterizations is that, in the infinite-width limit, SP leads to a regime in which the hidden features become asymptotically frozen, whereas μP in comparison is able to preserve nontrivial feature learning.

3.1 Feature Learning in a One-Hidden-Layer MLP

To illustrate the difference between the standard parametrization (SP) and the maximal-update parametrization μP, an MLP with one hidden layer of width n is considered in the following. Using the notation introduced in the previous chapter, this corresponds to $L = 1$ and $n_1 = n$. For simplicity, a scalar regression output $p = n_2 = 1$ with a linear output activation $\sigma^1 = \mathrm{id}$ is considered, and biases are omitted as they do not affect the central width-scaling arguments. The scalar index $t \in \mathbb{N}_0$ denotes the SGD update step, where $t = 0$ corresponds to initialization, such that W_t^0 and W_t^1 denote the weight matrices after t optimization steps. For a given input $\boldsymbol{x} \in \mathbb{R}^d$, the hidden preactivation vector, the hidden activation vector, and the scalar network output at step t are defined as

$$\boldsymbol{h}_t^0(\boldsymbol{x}) := W_t^0 \boldsymbol{x} \in \mathbb{R}^n, \qquad \boldsymbol{z}_t^1(\boldsymbol{x}) := \sigma^0\left(\boldsymbol{h}_t^0(\boldsymbol{x})\right) \in \mathbb{R}^n, \tag{3.1}$$

$$f_t(\boldsymbol{x}) := W_t^1 \boldsymbol{z}_t^1(\boldsymbol{x}), \qquad W_t^0 \in \mathbb{R}^{n \times d}, \qquad W_t^1 \in \mathbb{R}^{1 \times n}, \tag{3.2}$$

where the activation function σ^0 is applied component-wise. At SGD step t, the network is updated using a training pair $(\boldsymbol{x}_t, y_t)$, for which the derivative of the loss with respect to the network output is defined as

$$\zeta_t := \frac{\partial \mathcal{L}\left(f_t(\boldsymbol{x}_t), y_t\right)}{\partial f_t(\boldsymbol{x}_t)}. \tag{3.3}$$

For typical loss functions and targets of order one, this derivative satisfies $\zeta_t = O(1)$, meaning that its magnitude does not explicitly scale with the hidden-layer width n. Similarly, the hidden activation function is assumed to satisfy $\sigma^0(h^0_{t,j}) = O(1)$ and $(\sigma^0(h^0_{t,j}))' = O(1)$ for typical preactivations $h^0_{t,j} = O(1)$, which includes smooth activation functions such as the hyperbolic tangent and applies to active ReLU neurons almost everywhere [9, 10].

An SGD step with a layer-wise learning rate η_{W^l} updates the weight matrices according to $W^l_{t+1} = W^l_t - \eta_{W^l}\,\zeta_t\,\nabla_{W^l_t} f_t(\boldsymbol{x}_t)$. Both the weights and the features change between two consecutive steps. Writing $\Delta W^l := W^l_{t+1} - W^l_t$ and $\Delta \boldsymbol{z}^1(\boldsymbol{x}') := \boldsymbol{z}^1_{t+1}(\boldsymbol{x}') - \boldsymbol{z}^1_t(\boldsymbol{x}')$, the full change of the network output for an arbitrary input $\boldsymbol{x}'$ decomposes exactly as

$$f_{t+1}(\boldsymbol{x}') - f_t(\boldsymbol{x}') = \underbrace{\Delta W^1\, \boldsymbol{z}^1_t(\boldsymbol{x}')}_{\text{output-weight contribution}} + \underbrace{W^1_t\, \Delta \boldsymbol{z}^1(\boldsymbol{x}')}_{\text{feature contribution}} + \underbrace{\Delta W^1\, \Delta \boldsymbol{z}^1(\boldsymbol{x}')}_{\text{cross term}}. \tag{3.4}$$

The two mechanisms that distinguish SP from μP are already visible in the leading contributions of this decomposition. The output-weight contribution is driven by the output-layer update, which, evaluated at the training input $\boldsymbol{x}_t$ and on the current features, reads

$$\Delta W^1\, \boldsymbol{z}^1_t(\boldsymbol{x}_t) = -\eta_{W^1} \zeta_t\, \boldsymbol{z}^1_t(\boldsymbol{x}_t)^\top \boldsymbol{z}^1_t(\boldsymbol{x}_t), \tag{3.5}$$

and controls whether the output update remains stable as $n \to \infty$.
The feature contribution is driven by the change of the hidden representations, which follows from the change of the hidden preactivation vector,

$$\boldsymbol{h}^0_{t+1}(\boldsymbol{x}') - \boldsymbol{h}^0_t(\boldsymbol{x}') = -\eta_{W^0} \zeta_t \left[(W^1_t)^\top \odot (\sigma^0)'\left(\boldsymbol{h}^0_t(\boldsymbol{x}_t)\right)\right] \boldsymbol{x}_t^\top \boldsymbol{x}', \tag{3.6}$$

where $\odot$ denotes component-wise multiplication, and controls whether the features can still evolve in the infinite-width limit. It is important to note that Equation A.41 is only the isolated output-weight contribution and not the full output change in Equation A.40. It is nevertheless the quantity that governs output stability, because under SP it is the dominant contribution, while the feature and cross terms are subleading, whereas under μP all three contributions are of the same order and are therefore treated jointly. The detailed derivation of these equations and of all scaling results stated below can be found in [9, 10], where a detailed derivation is also provided in the Appendix A.1.

3.1.1 Standard Parametrization (SP)

The standard parametrization combines the commonly used LeCun initialization, $W^l \sim \mathcal{N}(0, \frac{1}{n_l})$, with a single global learning rate that is shared across all layers. For the one-hidden-layer MLP this corresponds to

$$(W_0^0)_{jk} \sim \mathcal{N}\left(0, \frac{1}{d}\right), \qquad (W_0^1)_{jk} \sim \mathcal{N}\left(0, \frac{1}{n}\right), \qquad \eta_n = n^{-c}, \tag{3.7}$$

where the width-dependent learning-rate scaling $\eta_n \sim n^{-c}$ is introduced to analyze stability in the infinite-width limit, while formal definitions of SP set $c = 0$, since no universal task-agnostic scaling exponent can be applied in practice. The LeCun initialization ensures that the coordinates of the hidden preactivation and activation vectors are of order one, $h_{t,j}^0(\boldsymbol{x}) = O(1)$ and $z_{t,j}^1(\boldsymbol{x}) = O(1)$, and that each output weight satisfies $W_{t,j}^1 = O(n^{-1/2})$, such that the network output remains of order $f_t(\boldsymbol{x}) = O(1)$.

The key observation is that the inner product in Equation A.41 is a sum of n coordinates of order one, $\boldsymbol{z}_t^1(\boldsymbol{x}_t)^\top \boldsymbol{z}_t^1(\boldsymbol{x}_t) = O(n)$, such that the output update scales as $O(n^{1-c})$. For the network output to remain stable as $n \to \infty$, the learning rate must therefore be scaled with the exponent $c = 1$, corresponding to the largest stable and nontrivial effective learning rate

$$\eta_n = O\left(\frac{1}{n}\right). \tag{3.8}$$

This stability requirement, however, suppresses the evolution of the hidden representations: since each coordinate of $(W_t^1)^\top$ in Equation 3.6 is of order $O(n^{-1/2})$ and the shared learning rate contributes the factor $\eta_n = O(n^{-1})$, each hidden preactivation coordinate changes only by $O(n^{-c-1/2})$, which for $c = 1$ yields

$$h_{t+1,j}^0(\boldsymbol{x}') - h_{t,j}^0(\boldsymbol{x}') = O\left(n^{-3/2}\right) \longrightarrow 0, \tag{3.9}$$

$$z_{t+1,j}^1(\boldsymbol{x}') - z_{t,j}^1(\boldsymbol{x}') = O\left(n^{-3/2}\right) \longrightarrow 0, \tag{3.10}$$

where the scaling of the postactivations follows from a first-order expansion of the activation function and holds equally for Lipschitz-continuous activation functions such as ReLU [9, 10]. The hidden representations therefore become asymptotically frozen in the infinite-width limit [9]. As the features remain at their initial values, the evolution of the network output under gradient descent is governed by a fixed feature kernel, and the network approaches the NTK regime, often referred to as the lazy-learning regime, in which training can be formulated as kernel gradient descent in function space [26–30].

3.1.2 Maximal-Update Parametrization (μP)

Under μP, the initialization and the layer-wise learning rates are chosen such that the hidden representations can change by an order-one amount without causing the network output to diverge. The hidden-layer weights are initialized as under SP, while the initialization variance of the output-layer weights is reduced and the layers are assigned different width-dependent learning rates,

$$(W_0^0)_{jk} \sim \mathcal{N}\left(0, \frac{1}{d}\right), \qquad (W_0^1)_{jk} \sim \mathcal{N}\left(0, \frac{1}{n^2}\right), \qquad \eta_{W^0} = n\eta, \qquad \eta_{W^1} = \frac{\eta}{n}, \quad (3.11)$$

where $\eta = O(1)$ denotes a width-independent base learning rate. The reduced initialization variance implies $W_{t,j}^1 = O(n^{-1})$ and thus $nW_{t,j}^1 = O(1)$, while the hidden preactivations and activations remain of order one as under SP. At initialization, the network output is a sum of n approximately independent terms of order $O(n^{-1})$ and therefore satisfies $f_0(\boldsymbol{x}) = O(n^{-1/2}) \longrightarrow 0$ as $n \to \infty$, while correlations between the weights and the learned features can generate outputs of order $O(1)$ during training.

The intuition behind the layer-wise learning rates becomes apparent when they are inserted into Equations 3.5 and 3.6. In the hidden layer, the amplified learning rate $\eta_{W^0} = n\eta$ exactly compensates the smaller magnitude of the output weights, since $nW_{t,j}^1 = O(1)$, such that each hidden preactivation coordinate and each hidden feature change by an order-one amount:

$$h_{t+1,j}^0(\boldsymbol{x}') - h_{t,j}^0(\boldsymbol{x}') = O(1), \quad (3.12)$$

$$z_{t+1,j}^1(\boldsymbol{x}') - z_{t,j}^1(\boldsymbol{x}') = O(1). \quad (3.13)$$

In the output layer, the suppressed learning rate $\eta_{W^1} = \eta/n$ compensates the order-$O(n)$ inner product of the hidden activations in Equation 3.5, such that the output update remains of order $O(1)$ and each coordinate of the updated output weights remains of order $O(n^{-1})$. Consequently, the network output remains bounded, $f_{t+1}(\boldsymbol{x}') = O(1)$, even though individual hidden features evolve by an order-one amount [9, 10]. Thus, μP preserves stable and nontrivial feature learning in the infinite-width limit.

3.1.3 Comparison of SP and μP

The central difference between SP and μP is how the initialization variances and learning rates are scaled with network width. Under SP, keeping the output update stable requires the learning-rate scaling $\eta_n = O(n^{-1})$, which forces the updates of the hidden representations to decay as $O(n^{-3/2})$, such that the features become frozen in the infinite-width limit and the training dynamics are governed by a fixed kernel corresponding to the lazy-learning or NTK regime [26–30]. In contrast, μP rescales the initialization variance of the output weights and assigns layer-wise learning rates, such that the updates of the hidden representations remain of order one while the network

output stays bounded, and feature learning is preserved in the infinite-width limit [9, 10].

It is important to distinguish the learning-rate scaling used in the SP analysis from the usual SP convention. The scaling $\eta_n = O(n^{-1})$ was introduced only to discuss stability and feature freezing in the infinite-width limit. In the standard abc-parametrization table below, SP is therefore still listed with $c_l = 0$, corresponding to a width-independent learning rate, which is usually used for the SP. Consequently, the pre-and postactivation changes are of $O(n^{-\frac{1}{2}})$ in the usual convention of the SP where $c_l = 0$, which can be seen from Equation 3.10. Equivalent formulations of μP exist in the literature. Some use explicit multipliers in front of the weight matrices instead of layer-wise learning-rate scalings. However, as these are only conventional preferences the same stable feature-learning behavior is obtained after the corresponding rescaling of weights and learning rates. In this thesis, the layer-wise learning-rate convention, as introduced above, is used.

The SP and μP parameterizations can be summarized by the exponents a_l, b_l, and c_l, which determine the multiplier, initialization variance, and learning-rate scaling, respectively. In the convention used here, the multiplier exponent is set to $a_l = 0$, so the relevant differences between SP and μP appear in the initialization exponent b_l and the learning-rate exponent c_l.

$$(\widetilde{W}_t^{\,l})_{jk} = n_l^{-a_l}(W_t^{\,l})_{jk}, \qquad (W_0^{\,l})_{jk} \sim \mathcal{N}\left(0, n_l^{-2b_l}\right). \tag{3.14}$$

$$\eta_l = \eta_0 n_l^{-c_l}. \tag{3.15}$$

To validate the statements from the previous subsections that predict the evolution of hidden representations during training under SP and μP, as in equations 3.9, 3.10, 3.12 and 3.13 and derived in detail in Appendix A.1, Figure 3.1 shows the scaling of pre-and postactivations with increasing width. The change of pre-and postactivations between consecutive update steps are labeled as

$$\langle\overline{\Delta h_t}\rangle_{\mathcal{D}_{\text{te}}} := \mathbb{E}_{\boldsymbol{x}_i \in \mathcal{D}_{\text{te}}}\left[\frac{1}{n}\sum_{j=1}^{n}|h^0_{t+1,j}(\boldsymbol{x}_i) - h^0_{t,j}(\boldsymbol{x}_i)|\right] \tag{3.16}$$

$$\langle\overline{\Delta z_t}\rangle_{\mathcal{D}_{\text{te}}} := \mathbb{E}_{\boldsymbol{x}_i \in \mathcal{D}_{\text{te}}}\left[\frac{1}{n}\sum_{j=1}^{n}|z^0_{t+1,j}(\boldsymbol{x}_i) - z^0_{t,j}(\boldsymbol{x}_i)|\right], \tag{3.17}$$

where the average across the test dataset allows to obtain a robust estimation of the scaling behaviour. As predicted from the scaling arguments in the previous subsections, changes of the pre-and postactivations remain roughly $O(1)$ for μP and decay as $O(n^{-1/2})$ for SP.

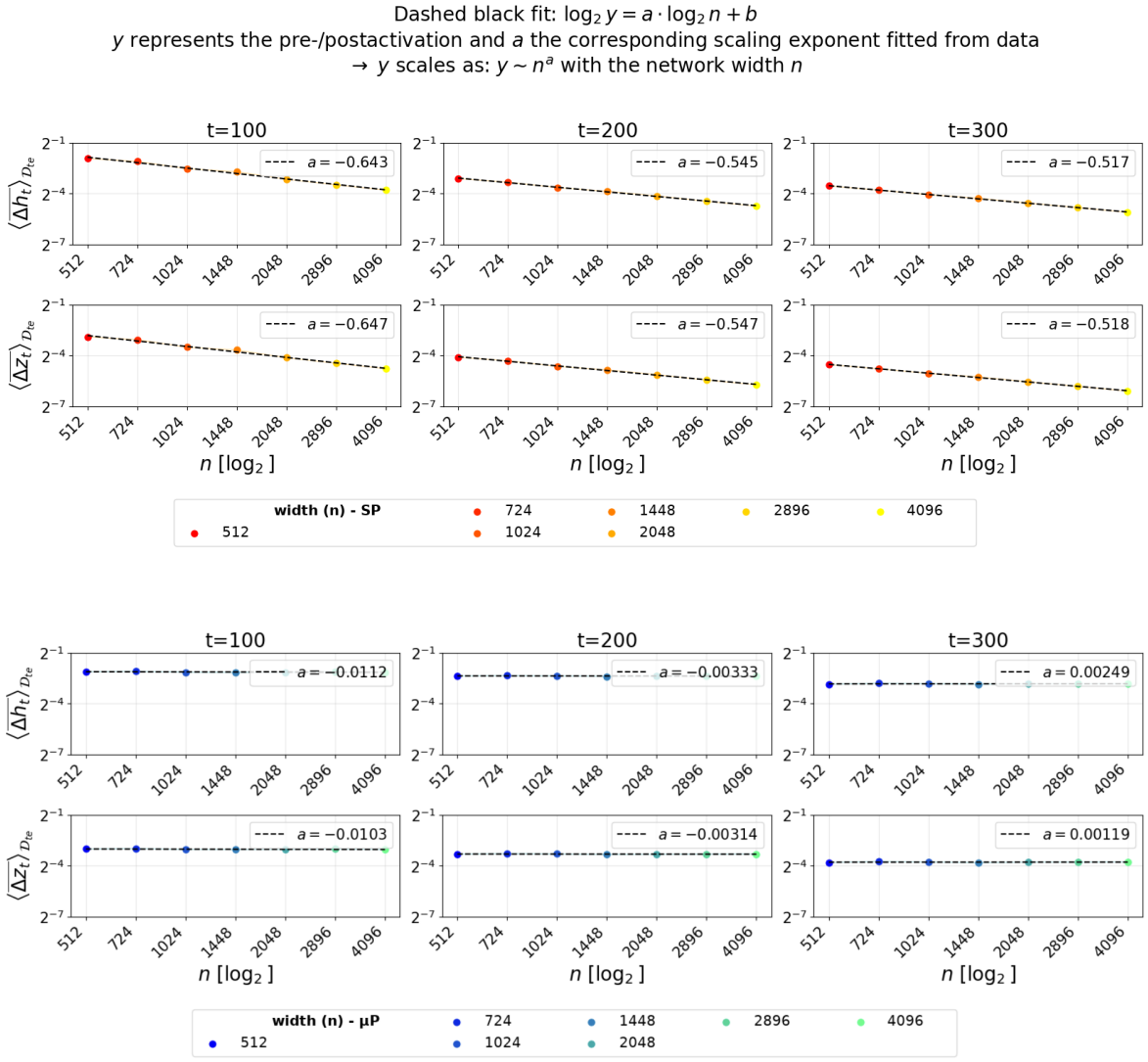


Figure 3.1: *Changes of the hidden-layer preactivations and postactivations with increasing hidden-layer width n at three different training steps for a one-hidden-layer MLP trained on CIFAR-10. The legend indicates the width of the hidden layer (n) of each network and the utilized parametrization. Under SP, the activation changes decay approximately as $O(n^{-1/2})$, indicating that feature learning vanishes in the infinite-width limit. Under µP, the activation changes remain of order $O(1)$, demonstrating non-vanishing feature learning at all network widths.*

	Input weights & all biases		Output weights		Hidden weights
a_l	0		0		0
b_l	1/2		1	1/2	1/2
c_l	-1	0	1	0	0
Multiplier ($n_l^{-a_l}$)	1		1		1
Init. Var. ($n_l^{-2b_l}$)	1/fan_in		$1/\text{fan_in}^2$	1/fan_in	1/fan_in
SGD LR ($n_l^{-c_l}$)	fan_out	1	1/fan_in	1	1

Table 3.1: *Coefficients a_l, b_l, and c_l and the resulting scaling factors $n_l^{-a_l}$, $n_l^{-b_l}$, and $n_l^{-c_l}$ are colored cyan for μP and gray for SP, whereas coefficients written in black are equal for μP and SP. The quantity n_l can refer to the input or output dimension of the corresponding layer, denoted by* fan_in *and* fan_out*, respectively. Additionally, abc-parameterizations are not necessarily unique in general, meaning that multiple different sets of coefficients a_l, b_l, and c_l can effectively yield the same parametrization [9].*

3.2 Feature Learning in the Infinite-Width Limit

Empirical scaling laws were able to show that the performance of neural networks can improve predictably when model size, dataset size, and training compute are jointly increased [31]. Therefore, the performance advantage of large neural networks should not be interpreted as a consequence of model size alone, but as the result of a suitable scaling of model capacity, available data, and compute [15, 31, 32]. This observation motivates theoretical limits in which the width of a neural network is increased while the corresponding training dynamics remain stable and well defined.

The infinite-width limit is not introduced because realistic networks are infinitely wide, but because it provides a controlled reference limit for studying the behavior of large finite networks. Depending on the parametrization, neural networks can converge to analytically tractable limiting objects, such as Gaussian processes at initialization or deterministic kernel dynamics during training [26, 28–30]. However, the form of the infinite-width limit depends crucially on the chosen parametrization. Thus, the infinite-width limit does not describe a unique universal object, but rather the limiting training dynamics induced by a specific scaling prescription.

As discussed in Section 3.1.3, the standard parametrization approaches the NTK regime in the infinite-width limit. In this regime, the hidden representations remain asymptotically close to their initial values, and gradient descent can be described by kernel gradient descent based on a fixed Neural Tangent Kernel [26, 27, 30]. This lazy-learning regime is well understood analytically, because it turns the training dynamics into a tractable function-space problem. However, as feature learning is suppressed it fails to capture the central mechanism that distinguishes deep neural networks from

fixed-kernel methods, namely the ability of finite-width neural networks to adapt their internal representations during training [9].

For this reason, a theoretical framework that describes feature learning not only in large finite systems but also in the infinite-width limit can significantly advance our understanding of the limiting feature learning behaviour approached by finite-size systems when their width is increased. The maximal-update parametrization was introduced to define such a feature-learning infinite-width limit. Under μP, the network output remains stable while hidden representations can change by order-one amounts during training, so the infinite-width limit retains feature learning instead of collapsing to a fixed-kernel description [9–11]. Therefore, in comparison to SP, μP presents a more suitable reference for studying how feature learning behaves as the width of a neural network is increased.

A second motivation for μP is its practical connection to hyperparameter transfer, which represents a huge practical advantage compared to SP, as HP tuning is computationally very costly. Under μP, the learning-rate tuned at smaller widths remains close to optimal at larger widths, which indicates that finite-width neural networks represent an approximation of the infinite-width limit [10, 33–35]. Empirical studies further suggest that neural networks trained under μP can exhibit consistent training trajectories, predictions, internal representations, preactivation distributions, and edge-of-stability behavior across sufficiently large widths [36, 37]. These results support the interpretation that large but finite networks can, over relevant stages of training, be understood as approximations to an infinite-width feature-learning limit.

The existence of such a limit also motivates the study of finite-width effects. Since realistic models always have finite width, their training trajectories are expected to deviate from the corresponding infinite-width dynamics at some point during training. Such finite-width effects have been observed at both the microscopic level, for example in the actual feature kernel, and the macroscopic level, for example in the loss function [27]. Importantly, these finite-width effects quantify how far a finite network remains from its infinite-width reference dynamics, which is the underlying mechanism of the aforementioned scaling laws.

Increasing the width of a neural network increases the amount of trainable parameters that are optimized during training, which is directly connected to the number of internal features the neural network can store. This leads to the intuition that increasing its width allows a neural network to capture more complex relationships in the data using the enlarged number of features and thereby improve its prediction performance. However, referring again to the introduced feature learning behaviour of MLPs under SP and μP in the infinite-width limit, as introduced in section 3.1.3, this perspective on feature learning showed that the feature learning properties in the infinite-width

limit crucially depend on the utilized abc-parametrization. Combining the fact that wider networks comprise more internal features with the abc-parametrization dependent feature learning behaviour in the infinite-width limit leads naturally to the question addressed in the following parts of this thesis: **How does the abc-parametrization determine the evolution of hidden representations during training, and how is this reflected in generalization performance as model width increases?**

3.3 Experimental Setup

As outlined in section 3.2 this thesis focuses on understanding the effect of increasing width on a neural network's feature learning capabilities and how this is reflected in macroscopic training observables. To isolate the effect of increasing width in the clearest possible setup the same one-hidden-layer MLP architecture that has been used before to illustrate feature learning under SP and μP is used as experimental setup. The output dimension is represented by p in the following, as multiple different tasks will be considered that will also have multivariate outputs, while the introduced feature learning limits and can be directly transferred to this multivariate output case.

The network's components are visualized in 3.2 and again labeled as:

$$\boldsymbol{f}_t(\boldsymbol{x}_i) = W_t^1 \boldsymbol{z}_t, \qquad \boldsymbol{z}_t = \sigma(\boldsymbol{h}_t), \qquad \boldsymbol{h}_t = W_t^0 \boldsymbol{x}_i, \tag{3.18}$$

$$\tag{3.19}$$

$$\boldsymbol{x}_i \in \mathbb{R}^d, \qquad \boldsymbol{h}_t \in \mathbb{R}^n, \qquad \boldsymbol{f}_t(\boldsymbol{x}_i) \in \mathbb{R}^p. \tag{3.20}$$

To remove any effects that might be caused by advanced optimizers, as for example Adam, mini-batch SGD is used in all training setups. Furthermore, the batch-size is kept constant and the same random batch sequence is used for all network's that are trained on the same task but with different widths to remove any effect during training that might be caused by the order of data samples. Additionally, no training step dependent learning-rate schedule is employed but a constant learning-rate is used during the whole training process. For clarity biases are also omitted in this setup. Finally, also no regularization methods are used throughout training and the training process is terminated when the network starts to overfit the training data, which can be identified from a rising test loss in late training stages. At this point, it is emphasized again that each network's prediction performance is only measured based on the test loss, where the test data is completely excluded from the training process to provide an unbiased estimation of generalization performance. Depending on the type of task, the MSE or cross-entropy loss function is used to estimate the network's generalization error from the sample based predictions. The MLP is trained on different benchmark datasets, where the results will be discussed based on the CIFAR10 image classification

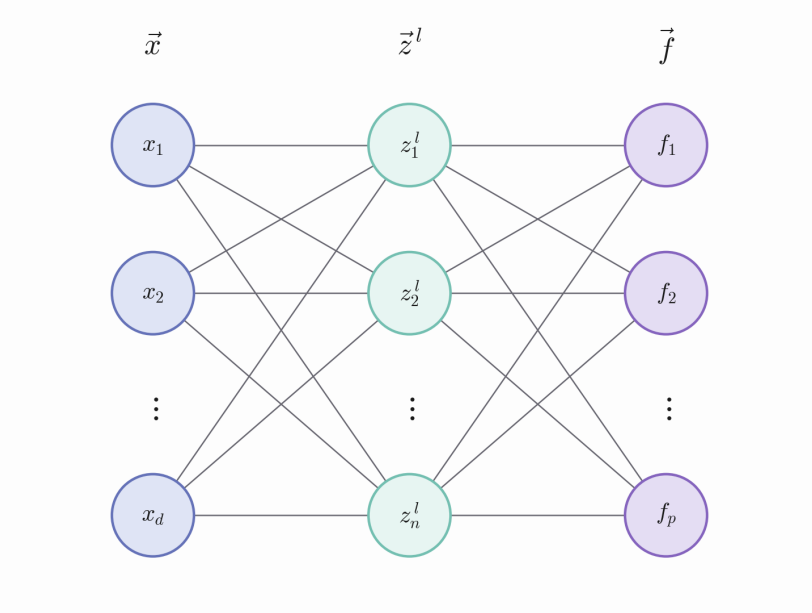


Figure 3.2: *Visualization of the neural network that is used in the experimental setup.*

dataset, but generalize to other datasets as well, as for example the SVHN dataset. Referring to the different feature-learning dynamics under SP and μP as outlined in section 3.2, width induced changes in the training dynamics are compared between SP and μP based on this training setup.

Under the μP framework the weight matrices and learning rates are parametrized as:

$$W_t^0 = (W_t^0)_{jk} \in \mathbb{R}^{n \times d}, \qquad W_t^1 = (W_t^1)_{jk} \in \mathbb{R}^{p \times n}, \tag{3.21}$$

$$\tag{3.22}$$

$$(W_0^0)_{jk} \sim \mathcal{N}\left(0, \frac{1}{d}\right), \qquad (W_0^1)_{jk} \sim \mathcal{N}\left(0, \frac{1}{n^2}\right). \tag{3.23}$$

$$\eta_{W^0} = \eta n, \qquad \eta_{W^1} = \eta\, n^{-1}. \tag{3.24}$$

Under SP weight matrices and learning rates are parametrized as:

$$W_t^0 = (W_t^0)_{jk} \in \mathbb{R}^{n \times d}, \qquad W_t^1 = (W_t^1)_{jk} \in \mathbb{R}^{1 \times n}, \tag{3.25}$$

$$\tag{3.26}$$

$$(W_0^0)_{jk} \sim \mathcal{N}\left(0, \frac{1}{d}\right), \qquad (W_0^1)_{jk} \sim \mathcal{N}\left(0, \frac{1}{n}\right). \tag{3.27}$$

$$\eta_{W^0} = \eta, \qquad \eta_{W^1} = \eta. \tag{3.28}$$

4 Tensor Programs Framework

The following section provides an introduction into the Tensor Programs Framework (TPF) that allows one to derive the training dynamics of a neural network in the infinite-width limit under any abc-parametrization. This theoretical framework will be used in the following sections to understand how self-averaging can give rise to universality across scalar observables of the training process. Within the TPF, weights are described by random variables, which is motivated by the fact that all weights are drawn independently from the same distribution and are therefore iid at initialization, where the training process subsequently induces correlations between them. Furthermore, due to the symmetry of an MLP all neurons are exchangeable, which makes a strict labeling of neurons inappropriate considering the fact that weights are randomly sampled at initialization. The TPF explicitly avoids this strict labeling by formulating training observables, as the network's output $\boldsymbol{f}_t$, as empirical averages across all neurons of the network. This will allow to understand how the entire training dynamics of the network become deterministic under μP in the infinite-width limit. In this context, neuron-wise observables arise naturally, and we will use them to explain how self-averaging gives rise to universal behaviour across networks of different widths.

4.1 SGD Training Dynamics of an MLP under μP

For simplicity, the TPF is illustrated in the following based on the introduced one-hidden-layer MLP architecture that is trained under the μP, with a mini-batch size of $B = 1$. The extension to arbitrary mini-batch size is discussed in Section A.2. The data samples are denoted by $\boldsymbol{x}_t$, where the subscript t labels the SGD update step. This notation generally does not force a specific ordering of the dataset for applying the TPF, as in practice the data may be shuffled before training. As in this setup only one hidden layer is used, the layer index l is omitted where $\boldsymbol{f}_t$ is always used instead of $\boldsymbol{z}_t^2$ such that $\boldsymbol{z}_t$ always refers to $\boldsymbol{z}_t^1$, $\boldsymbol{h}_t$ to $\boldsymbol{h}_t^0$ and σ to σ^0 to simplify the notation.

$$\boldsymbol{f}_t(\boldsymbol{x}_i) = W_t^1 \boldsymbol{z}_t, \qquad \boldsymbol{z}_t = \sigma(\boldsymbol{h}_t), \qquad \boldsymbol{h}_t = W_t^0 \boldsymbol{x}_i, \tag{4.1}$$

$$\tag{4.2}$$

$$\boldsymbol{x}_i \in \mathbb{R}^d, \qquad \boldsymbol{h}_t \in \mathbb{R}^n, \qquad \boldsymbol{f}_t(\boldsymbol{x}_i) \in \mathbb{R}^p. \tag{4.3}$$

Since the μP is used, the weight matrices $W_t^0 = (W_t^0)_{jk} \in \mathbb{R}^{n\times d}$ and $W_t^1 = (W_t^1)_{jk} \in \mathbb{R}^{p\times n}$ as well as the learning rates are parametrized as:

$$(W_0^0)_{jk} \sim \mathcal{N}\Big(0, \frac{1}{d}\Big), \qquad (W_0^1)_{jk} \sim \mathcal{N}\Big(0, \frac{1}{n^2}\Big). \tag{4.4}$$

$$\eta_{W^0} = \eta n, \qquad \eta_{W^1} = \eta\, n^{-1}. \tag{4.5}$$

Coordinate Distributions at Initialization ($t = 0$)

It will be useful to represent the weights of the network by a scalar random variable X that follows a certain distribution. At initialization ($t = 0$) all weights are iid such that in short notation the weights of the network can be represented as

$$X^{W_0^0} \sim \mathcal{N}(0, \frac{1}{d}), \qquad X^{nW_0^1} \sim \mathcal{N}(0, 1). \tag{4.6}$$

Furthermore, $X^{nW_0^1} \sim \mathcal{N}(0,1)$ was introduced instead of $X^{W_0^1} \sim \mathcal{N}(0, \frac{1}{n^2})$, as it is advantageous that the distribution of random variables X does not depend on the number of neurons in the hidden layer for comparing expectation values across networks of different widths.

First Forward Pass ($t = 0$)

The first input sample $\boldsymbol{x}_0$ is mapped into the hidden representation $\boldsymbol{h}_0$ through W_0^0,

$$\boldsymbol{h}_0 = W_0^0 \boldsymbol{x}_0, \qquad (\boldsymbol{h}_0)_j = \sum_{k=1}^{d} (W_0^0)_{jk} (\boldsymbol{x}_0)_k. \tag{4.7}$$

Since all weights are iid at initialization, the distribution of $\boldsymbol{h}_0$ can be calculated in closed form

$$X^{\boldsymbol{h}_0} \sim \mathcal{N}(0, \frac{\|\boldsymbol{x}_0\|^2}{d}), \tag{4.8}$$

where $\|\boldsymbol{x}_0\|$ is just a scalar number, as $\boldsymbol{x}_0$ represents the first sample of the dataset, and not a random variable. The indicated vector norm always refers to the L2 norm in the following, if not indicated otherwise. Applying the activation function to $\boldsymbol{h}_0$ yields the post activation $\boldsymbol{z}_0$ at $t = 0$

$$\boldsymbol{z}_0 = \sigma(\boldsymbol{h}_0). \tag{4.9}$$

The distribution of $\boldsymbol{z}_0$ is denoted by

$$X^{\boldsymbol{z}_0} = \sigma\left(X^{\boldsymbol{h}_0}\right). \tag{4.10}$$

However, it is generally not possible to obtain a closed form of this distribution for a network of finite width at an arbitrary training step that does not correspond to the situation at initialization where all weights are iid. Furthermore, the important point is that the distribution of $X^{\boldsymbol{z}_0}$ does not depend on the width of the network after applying the activation function component-wise to $\boldsymbol{h}_0$, as the distribution of $X^{\boldsymbol{h}_0}$ is independent of n. Each component of the network's output after the first forward pass is therefore given by

$$\begin{aligned}(\boldsymbol{f}_0)_j &= \sum_{k=1}^{n} (W_0^1)_{jk} (\boldsymbol{z}_0)_k \\ &= \frac{1}{n} \sum_{k=1}^{n} (nW_0^1)_{jk} (\boldsymbol{z}_0)_k \\ &\xrightarrow{n\to\infty} \mathbb{E}\left[X^{(nW_0^1)_{jk}} X^{(\boldsymbol{z}_0)_k}\right] \\ &:= (\mathring{\boldsymbol{f}}_0)_j. \end{aligned} \tag{4.11}$$

Using that the distributions of $X^{(nW_0^1)_{jk}}$ and $X^{(\boldsymbol{z}_0)_k}$ do not depend on the network's width, the first forward pass becomes deterministic in the infinite-width limit ($n \to \infty$). This results from summing over infinitely many realizations of $\frac{1}{n}(nW_0^1)_{jk}(\boldsymbol{z}_0)_k$, which converges to the expectation value of $X^{(nW_0^1)_{jk}} X^{(\boldsymbol{z}_0)_k}$. This means that in the infinite-width limit the network's output at initialization $(\boldsymbol{f}_0)_j$ will always be equivalent to $(\mathring{\boldsymbol{f}}_0)_j$, which is independent of the specific realization of each of the weights at initialization, because infinitely many realizations are considered.

First Backward Pass ($t = 0$)

To simplify the computation of the parameter updates based on the gradients the partial derivatives of the output w.r.t the pre-and postactivations are computed in the following. Note that $(d_{\boldsymbol{h}_0})_j$ and $(d_{\boldsymbol{z}_0})_j$ are still vectors and represent the derivative of each component of $\boldsymbol{f}_0$ w.r.t $(\boldsymbol{h}_0)_j$ and $(\boldsymbol{z}_0)_j$ scaled with an additional factor of n. In the case of $d_{\boldsymbol{h}_0} = d_{\boldsymbol{z}_0} \odot \sigma'(\boldsymbol{h}_0)$, this represents a short notation using the Hadamard product (element-wise product).

$$
\begin{aligned}
(d_{\boldsymbol{z}_0})_j &:= n\frac{\partial}{\partial(\boldsymbol{z}_0)_j}\boldsymbol{f}_0 \\
&= n\frac{\partial}{\partial(\boldsymbol{z}_0)_j}(W_0^1\boldsymbol{z}_0) \\
&= n(W_0^1)_{:,j}
\end{aligned} \tag{4.12}
$$

$$
\begin{aligned}
(d_{\boldsymbol{h}_0})_j &:= n\frac{\partial}{\partial(\boldsymbol{h}_0)_j}\boldsymbol{f}_0 \\
&= (d_{\boldsymbol{z}_0})_j \odot \sigma'(\boldsymbol{h}_0)
\end{aligned} \tag{4.13}
$$

SGD Updates

At each optimization step under SGD the network's weights W_t^l are updated according to the following update rule

$$
\begin{aligned}
W_{t+1}^l &= W_t^l + \Delta W_t^l && (4.14)\\
&= W_t^l - \eta_{W^l}\,\nabla_{W_t^l}\,\mathcal{L}_B(\boldsymbol{f}_t) && (4.15)\\
&= W_t^l - \eta_{W^l}\,\mathcal{L}_B'(\boldsymbol{f}_t)\,\nabla_{W_t^l}\boldsymbol{f}_t. && (4.16)
\end{aligned}
$$

The mini-batch loss $\mathcal{L}_B = L(\boldsymbol{f}_t(\boldsymbol{x}_t))$ only contains one data sample $\boldsymbol{x}_t$ as a consequence of setting $B = 1$ for this derivation. Additionally, $\boldsymbol{f}_t$ will be used as a short notation for $\boldsymbol{f}_t(\boldsymbol{x}_t)$ in the following. Therefore, the weight updates are then given by

$$
W_{t+1}^l = W_t^l - \eta_{W^l}\,L'(\boldsymbol{f}_t)\,\nabla_{W_t^l}\boldsymbol{f}_t. \tag{4.17}
$$

The derivative of the loss function w.r.t component $(\boldsymbol{f}_t)_j$ of the network's prediction is defined as $(\boldsymbol{\zeta}_t)_j := \frac{\partial L}{\partial\,(\boldsymbol{f}_t)_j}$ such that $\boldsymbol{\zeta}_t := \nabla_{\boldsymbol{f}_t}L(\boldsymbol{f}_t)$, which will be useful in the following derivation.
Now the previously computed gradients $(d_{\boldsymbol{h}_0})_j$ and $(d_{\boldsymbol{z}_0})_j$ can be used to compute the weight updates explicitly.

Case $l = 0$:

$$(W_1^0)_{jk} = (W_0^0)_{jk} - \eta_{W^0} \frac{\partial L(\boldsymbol{f}_0)}{\partial (W_0^0)_{jk}} \tag{4.18}$$

$$= (W_0^0)_{jk} - \eta n \frac{\partial L(\boldsymbol{f}_0)}{\partial (\boldsymbol{h}_0)_j} \frac{\partial (\boldsymbol{h}_0)_j}{\partial (W_0^0)_{jk}} \tag{4.19}$$

$$= (W_0^0)_{jk} - \eta n \; \boldsymbol{\zeta}_0 \frac{(d_{\boldsymbol{h}_0})_j}{n} (\boldsymbol{x}_0)_k \tag{4.20}$$

$$= (W_0^0)_{jk} - \eta \, \boldsymbol{\zeta}_0 \, (d_{\boldsymbol{h}_0})_j (\boldsymbol{x}_0)_k \tag{4.21}$$

Case $l = 1$:

$$(W_1^1)_{jk} = (W_0^1)_{jk} - \eta_{W^1} \frac{\partial L(\boldsymbol{f}_0)}{\partial (W_0^1)_{jk}} \tag{4.22}$$

$$= W_0^1 - \frac{\eta}{n} (\boldsymbol{\zeta}_0)_j \, (\boldsymbol{z}_0)_k \tag{4.23}$$

Coordinate Distributions After One SGD Update ($t = 1$)

$$X^{(W_1^0)_{jk}} = X^{(W_0^0)_{jk}} - \eta \, \boldsymbol{\zeta}_0 \, X^{(d_{\boldsymbol{h}_0})_j} (\boldsymbol{x}_0)_k \tag{4.24}$$

$$X^{n(W_1^1)_{jk}} = X^{n(W_0^1)_{jk}} - \eta \, (\boldsymbol{\zeta}_0)_j \, X^{(\boldsymbol{z}_0)_k} \tag{4.25}$$

As long as the number of neurons in the hidden layer is finite, the exact value of the loss function $L(\boldsymbol{f}_0)$ and the outer derivative of the loss function at $t = 0$ labeled as $\boldsymbol{\zeta}_0$ will depend on the exact configuration of weights at initialization. However, in the infinite-width limit $\boldsymbol{f}_0 \overset{n\to\infty}{\longrightarrow} \mathring{\boldsymbol{f}}_0$ and therefore also $\boldsymbol{\zeta}_0 = L'(\boldsymbol{f}_0) \overset{n\to\infty}{\longrightarrow} L'(\mathring{\boldsymbol{f}}_0) := \mathring{\boldsymbol{\zeta}}_0$ will converge almost surely [9, 10] to a deterministic value.

Even though the distribution of $X^{(W_t^l)_{jk}}$ is in principle known after every SGD update iteration, obtaining an analytical closed form of the distribution is not possible in most realistic settings [9]. However, since $\boldsymbol{f}_0$ becomes deterministic the updated weights $X^{(W_1^0)_{jk}}$ and $X^{n(W_1^1)_{jk}}$ can be formulated using only deterministic values, and random variables with deterministic distributions in the infinite-width limit.

$$X^{(W_1^0)_{jk}} \xrightarrow{n\to\infty} X^{(W_0^0)_{jk}} - \eta \, \mathring{\boldsymbol{\zeta}}_0 \, X^{(d_{\boldsymbol{h}_0})_j} (\boldsymbol{x}_0)_k \,, \tag{4.26}$$

$$X^{n(W_1^1)_{jk}} \xrightarrow{n\to\infty} X^{n(W_0^1)_{jk}} - \eta \, (\mathring{\boldsymbol{\zeta}}_0)_j \, X^{(\boldsymbol{z}_0)_k}. \tag{4.27}$$

Therefore, also the ΔW_t^l become deterministic in the infinite-width limit and are for the two layers $l = 0$ and $l = 1$ given by

$$X^{(\Delta W_0^0)_{jk}} = X^{(W_1^0)_{jk}} - X^{(W_0^0)_{jk}}, \tag{4.28}$$

$$X^{(\Delta W_0^0)_{jk}} \xrightarrow{n\to\infty} -\eta\, \mathring{\boldsymbol{\zeta}}_0\, X^{(d_{\boldsymbol{h}_0})_j}(\boldsymbol{x}_0)_k, \tag{4.29}$$

$$X^{n(\Delta W_0^1)_{jk}} = X^{n(W_1^1)_{jk}} - X^{n(W_0^1)_{jk}}, \tag{4.30}$$

$$X^{n(\Delta W_0^1)_{jk}} \xrightarrow{n\to\infty} -\eta\, (\mathring{\boldsymbol{\zeta}}_0)_j\, X^{(\boldsymbol{z}_0)_k}. \tag{4.31}$$

These definitions of the layer-wise update steps will be used in later parts of the thesis, as they build neuron-wise observables, which will be defined in 4.2.

Second Forward Pass ($t = 1$)

Analogously to the first forward pass, the second forwards pass is computed based on the updated weights of the network and again yields a network output that becomes deterministic in the infinite-width limit

$$\boldsymbol{h}_1 = W_1^0 \boldsymbol{x}_1, \qquad X^{(\boldsymbol{h}_1)_j} = \sum_{k=1}^{d} X^{(W_1^0)_{jk}} (\boldsymbol{x}_1)_k, \tag{4.32}$$

$$\boldsymbol{z}_1 = \sigma(\boldsymbol{h}_1), \qquad X^{\boldsymbol{z}_1} = \sigma(X^{\boldsymbol{h}_1}), \tag{4.33}$$

$$\tag{4.34}$$

$$(\boldsymbol{f}_1)_j = \frac{1}{n} \sum_{k=1}^{n} \underbrace{(nW_1^1)_{jk} (\boldsymbol{z}_1)_k}_{:=(\tilde{\boldsymbol{f}}_1)_{jk}} \tag{4.35}$$

$$= \frac{1}{n} \sum_{k=1}^{n} (\tilde{\boldsymbol{f}}_1)_{jk} \tag{4.36}$$

$$\xrightarrow{n\to\infty} \mathbb{E}\Big[(\tilde{\boldsymbol{f}}_1)_{jk}\Big] \tag{4.37}$$

$$:= (\mathring{\boldsymbol{f}}_1)_j. \tag{4.38}$$

By applying this iterative procedure it becomes evident that in the infinite-width limit the evolution of the network's output becomes deterministic at all training steps t

$$\boldsymbol{f}_t \xrightarrow{n\to\infty} \mathring{\boldsymbol{f}}_t \qquad \forall\ t. \tag{4.39}$$

4.2 Deterministic Training Dynamics and Mean-Field Limit

Having established that the training dynamics become deterministic under μP in the infinite-width limit we will now introduce the concept of neuron-wise observables that build on the empirical neuron-distribution. This will clarify under which conditions an arbitrary observable of the network will converge to a deterministic limit throughout the entire training process. To illustrate this idea we start by defining a new parameter vector that contains all weights corresponding to a single neuron $z_{t,j}$ [21]

$$\theta_{t,j} := ((W_t^0)_{j,:}, (W_t^1)_{:,j}) \in \mathbb{R}^{d+p}. \tag{4.40}$$

Here $(W_t^0)_{j,:}$ represent row j of W_t^0 and $(W_t^1)_{:,j}$ column j of (W_t^1), which contain all weights that connect node $z_{t,j}$ to the input and the output layer, as illustrated in Figure 4.1.

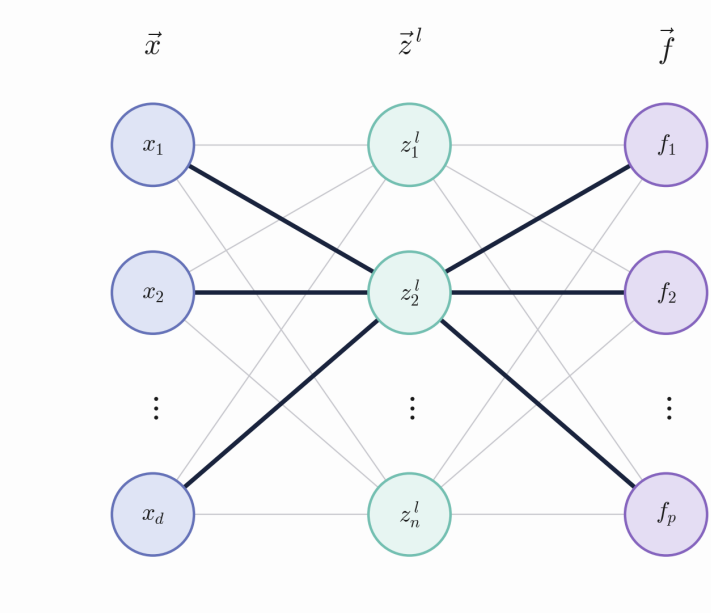


Figure 4.1: *Visualization of the investigated MLP architecture, where the weights connected to node $z_{t,2}$ are highlighted to illustrate the definition of $\theta_{t,j}$ in equation 4.40 based on neuron 2.*

Considering again the derived equations for the weight updates of W_t^0 and W_t^1

$$X^{(\Delta W_0^0)_{jk}} = -\eta\, \boldsymbol{\zeta}_0\, X^{(d_{\boldsymbol{h}_0})_j}(\boldsymbol{x}_0)_k \tag{4.41}$$

$$\xrightarrow{n\to\infty} -\eta\, \mathring{\boldsymbol{\zeta}}_0\, X^{(d_{\boldsymbol{h}_0})_j}(\boldsymbol{x}_0)_k \tag{4.42}$$

$$\tag{4.43}$$

$$X^{n(\Delta W_0^1)_{jk}} = -\eta\,(\boldsymbol{\zeta}_0)_j\, X^{(\boldsymbol{z}_0)_k} \tag{4.44}$$

$$\xrightarrow{n\to\infty} -\eta\,(\mathring{\boldsymbol{\zeta}}_0)_j\, X^{(\boldsymbol{z}_0)_k} \tag{4.45}$$

shows that in the infinite-width limit the updates of weights contained in $\theta_{t,j}$ that connect to neuron $z_{t,j}$ only depend on other weights that are also contained in $\theta_{t,j}$, but not on weights that connect to any other neuron $z_{t,a}$ $(a \neq j)$. This is due to the fact that in the infinite-width limit the loss function and its derivative $\boldsymbol{\zeta}_t \xrightarrow{n\to\infty} \mathring{\boldsymbol{\zeta}}_t$ become deterministic and therefore do not depend on the network's weights anymore. Thus, it holds in the infinite-width limit that all weights contained in $\theta_{t,j}$ and $\theta_{t,a}$ $(a \neq j)$ are statistically independent, which is not true for networks of finite width as in this case $\boldsymbol{\zeta}_t$ depends explicitly on all weights of the network [22]. As $\theta_{t,j}$ refers to all weights that connect to neuron $z_{t,j}$, all neurons are interchangeable and independent in the infinite-width limit [22].

Based on the set of parameters that connect to a single neuron $\{\theta_{t,j}\}_{j=1}^n$, the finite-width empirical neuron distribution is defined, which converges to a continuous density of neurons in the infinite-width limit [21]

$$\rho_t^n(\theta) := \frac{1}{n}\sum_{j=1}^{n} \delta(\theta - \theta_{t,j})$$

$$\xrightarrow{n\to\infty} \mathring{\rho}_t(\theta). \tag{4.46}$$

In the following, the term neuron-wise observable is used for a function $g(\theta_{t,j}) : \mathbb{R}^{d+p} \to \mathbb{R}$ that depends only on the parameters connecting to a single neuron. For any neuron-wise observable, integration with respect to ρ_t^n gives the empirical average of this neuron-wise observable based on the empirical neuron distribution [22]

$$\mathbb{E}_{\rho_t^n}[g] = \int_{\mathbb{R}^{d+p}} g(\theta)\,\rho_t^n(\theta) d\theta = \frac{1}{n}\sum_{j=1}^{n} g(\theta_{t,j}). \tag{4.47}$$

Thus, the empirical expectation value of a neuron-wise observable converges to the true expectation value in the infinite-width limit $\mathbb{E}_{\rho_t^n}[g] \xrightarrow[n\to\infty]{a.s.} \mathbb{E}_{\mathring{\rho}_t}[g]$, where all neurons are statistically independent [9, 10]. Furthermore, the empirical average of any neuron-wise observable of a network of finite width represents an approximation of the infinite-width limit as long as correlations between parameters that correspond to different neurons are negligible. The contribution of each neuron to the final prediction $(\tilde{\boldsymbol{f}}_t)_{jk}$ represents such a neuron-wise observable, which can be verified from Equation 4.38. Hence, the empirical average of $(\tilde{\boldsymbol{f}}_t)_{jk}$, which recovers the final prediction of the network, represents an approximation of the infinite-width limit

$$(\boldsymbol{f}_t)_j(n) \;=\; \int_{\mathbb{R}^{d+p}} (\tilde{\boldsymbol{f}}_t)_j(\theta)\,\rho_t^n(\theta)\,d\theta \;=\; \frac{1}{n}\sum_{k=1}^{n} \underbrace{(\tilde{\boldsymbol{f}}_t)_j(\theta_{t,k})}_{=(\tilde{\boldsymbol{f}}_t)_{jk}} \;=\; \mathbb{E}_{\rho_t^n}\Big[(\tilde{\boldsymbol{f}}_t)_j\Big]. \tag{4.48}$$

In comparison, under SP the empirical neuron distribution still converges to a deterministic limit where all neurons are statistically independent, but the network's prediction converges to a GP $\boldsymbol{f}_t(n) \xrightarrow[n\to\infty]{d} \boldsymbol{F}_t$ [28] that evolves according to kernel gradient descent during training [28]. Furthermore, for a general continuous function $\Phi : \mathbb{R}^{d_{\text{out}}} \to \mathbb{R}$, the continuous mapping theorem [38] states

$$\Phi(\boldsymbol{f}_t(n)) \xrightarrow[n\to\infty]{d} \Phi(\boldsymbol{F}_t). \tag{4.49}$$

Since F_t is random, $\Phi(F_t)$ remains also random in the infinite-width limit. Hence, the loss and its derivative do not converge to deterministic values under SP, even though the empirical neuron distribution itself becomes deterministic.

$$L(\boldsymbol{f}_t(n)) \xrightarrow[n\to\infty]{d} L(\boldsymbol{F}_t), \tag{4.50}$$

$$\nabla_f L(\boldsymbol{f}_t(n)) \xrightarrow[n\to\infty]{d} \nabla_f L(\boldsymbol{F}_t). \tag{4.51}$$

4.3 Implications for Feature Learning

The previous section introduced the empirical neuron distribution, which groups all parameters of the network that connect to a single neuron and converges to a continuous deterministic distribution in the infinite-width limit. Furthermore, the preceding introduction of the TPF showed that under μP preactivations and activations follow deterministic distributions and the network's prediction even converges to a deterministic function in the infinite-width limit [9, 10]. Consequently, for a fixed training-data sequence, or equivalently for a fixed mini-batch sequence, the network's output follows a deterministic trajectory during the entire training process in the infinite-width limit under μP. This does not imply that individual neurons or individual learned features become deterministic. Instead, individual hidden neurons remain random and exchangeable, while their empirical distribution converges to a deterministic limit.

This distinction is important when interpreting feature learning in the infinite-width limit. Under SP the network's prediction represents a GP that evolves according to deterministic kernel dynamics in the infinite-width limit, while hidden representations remain frozen [26, 29, 30]. Therefore, the empirical neuron distribution also remains frozen during the training process $\mathring{\rho}_t = \mathring{\rho}_0$. In contrast, μP is constructed such that hidden representations can undergo nontrivial changes even as the width tends to infinity. Therefore, the infinite-width limit under μP can describe feature-learning dynamics rather than only kernel gradient descent, which corresponds to an evolving empirical neuron distribution $\mathring{\rho}_t \neq \mathring{\rho}_0$ [9, 11, 35].

In this sense, finite-width networks trained under μP approach a deterministic feature-

learning limit as the width is increased. This provides the theoretical basis for the width-stability of training dynamics under μP and for hyperparameter transfer across model sizes, as exploited by μ-Transfer [10]. More recent work further supports the view that feature-learning networks parametrized according to μP can display consistent training behavior across widths at realistic model scales [36].

However, the existence of a deterministic infinite-width feature-learning limit does not by itself determine how accurately a network of finite width can approximate this limit. As introduced in Equation 4.47, a neuron-wise observable $g(\theta_{t,j})$ can be aggregated over the neurons contained in the hidden layer through its empirical neuron average

$$\mathbb{E}_{\rho_t^n}[g] = \frac{1}{n}\sum_{j=1}^{n} g(\theta_{t,j}). \tag{4.52}$$

In the infinite-width limit, this empirical average converges to the expectation with respect to the deterministic limiting neuron distribution,

$$\mathbb{E}_{\rho_t^n}[g] \xrightarrow[n\to\infty]{} \mathbb{E}_{\rho_t}[g] = \int_{\mathbb{R}^{d+p}} g(\theta)\rho_t(\theta)\, d\theta. \tag{4.53}$$

At initialization, the neuron parameters $\{\theta_{0,j}\}_{j=1}^n$ of a finite-width network are sampled independently from the initialization distribution. Consequently, empirical averages over neuron-wise observables approximate their infinite-width expectations according to the law of large numbers at initialization. However, when a network of finite width is trained, all neurons contribute jointly to the network's prediction and therefore to the loss and its derivative. As this loss signal enters into each parameter update, the dynamics of a finite-width network couples the different neurons such that neuron-wise observables are no longer statistically independent. As this effect accumulates during training, it is expected that empirical averages across neuron-wise observables show deviations from the infinite-width limit at some point during training, when the empirical neuron distribution of the finite-width network is no longer compatible with infinite width limit.

This coupling admits a direct interpretation in terms of the joint distribution of the neuron parameters. At any training step t, the configuration $\{\theta_{t,j}\}_{j=1}^n$ of a finite-width network can be considered as a single sample from a joint distribution $\mu_t^{(n)}$ on Θ^n, where the randomness originates from the initialization. At initialization, the joint distribution factorizes exactly into a product of identical single-neuron marginal distributions, $\mu_0^{(n)} = \mu_0^{\otimes n}$ [22]. However, the prediction of a finite-width network is exposed to $O(n^{-\frac{1}{2}})$ sampling fluctuations by the CLT, which creates a displacement shared across all neurons. This shared displacement breaks the factorization of the joint distribution, so that exact independence between neurons only holds at initialization. Evaluating a neuron-wise observable for a finite-width network at an arbitrary training step $t \neq 0$ is

therefore equivalent to sampling from a joint distribution that is distorted relative to its infinite-width counterpart caused by the shared displacement that breaks factorization. Therefore, this distortion is directly reflected by deviations of empirical averages of neuron-wise observables from their infinite-width expectations. In particular, the shared displacement does not average out when aggregating neuron-wise observables across all neurons of the network, in contrast to the independent sampling noise. Since the distortion is generated at every update step and couples back into the subsequent dynamics, its effect accumulates over training, so that empirical averages are expected to deviate increasingly from their infinite-width values, beyond the initial sampling fluctuations, as the joint distribution $\mu_t^{(n)}$ of neurons departs from the factorized structure. In contrast, as the displacement is of $O(n^{-\frac{1}{2}})$, propagation of chaos restores the product structure in the infinite-width limit such that $\mu_t^{(k)} \to \mu_t^{\otimes k}$ for any collection of k neurons, and the distortion induced by a finite-width network vanishes [22].
Under standard parametrization, the same mechanism governs the training dynamics, but with two important differences. First, the limiting single-neuron distribution does not evolve, $\rho_t = \rho_0$, so the finite-width joint distribution $\mu_t^{(n)}$ represents a small distortion of the stationary infinite-width distribution $\mu_0^{\otimes n}$ rather than of a moving, feature-learning limit. Therefore, all feature learning dynamics need to be encoded in the distortion itself.

In this chapter we introduced the TPF where weights are considered as random variables. Using this framework we showed that the network's output follows a deterministic trajectory under μP in the infinite-width limit throughout the entire training process. Furthermore, the empirical neuron-distribution was introduced that exploits the symmetry of the investigated one-hidden-layer MLP architecture to describe the network as a distribution rather than a strictly labeled set of neurons. Since all neurons become statistically independent in the infinite-width limit, we understood why any neuron-wise observable is self-averaging and therefore converges to a deterministic limit in the infinite-width limit under μP.

5 Results and Discussion

In this chapter the introduced concept of neuron-wise observables is used to explain the universal training dynamics of the layer-wise update norm and the test loss across different network widths under μP based on the TPF. Furthermore, this perspective is used to expose the explicit width dependence of the same training observables under SP, which persists throughout the entire training process. The concept of neuron-wise observables finally serves as the foundation for a spectral view of feature learning. This view reveals that finite-width corrections can be understood as a sequential revelation of the infinite-width feature-learning dynamics. Increasing a network's width gradually reduces the sampling noise that otherwise obscures feature learning during training, so that progressively more of the underlying infinite-width structure becomes visible. As all quantities that describe the different components of the network are multivariate at this point, the vector notation is omitted throughout the remaining chapter.

5.1 Universality Across Training Observables under μP

As outlined in Section 4.3, grouping the weights in form of the empirical neuron distribution explains why empirical averages over neuron-wise observables become deterministic in the infinite-width limit throughout the entire training process. Consequently, the dynamics of these observables should be independent of the network's width, up to trivial scaling constants, as long as the empirical neuron distribution of the finite-size network sufficiently approximates the infinite-width limiting distribution, whose evolution is governed by the infinite-width training dynamics. This picture of width-independent training dynamics under μP holds up to the point at which a finite-width network can no longer approximate the infinite-width limit and is illustrated below on the layer-wise update step size and the test loss.

Layer-Wise Update Step Size

A direct measure of the step size in weight space at each optimization step is the update norm. Starting from the parameter update

$$\theta_{t+1} = \theta_t + \Delta\theta_t, \tag{5.1}$$

the update norm is obtained as

$$\|\Delta\theta_t\| = \|\eta \nabla_{\theta_t} \mathcal{L}_B\|. \tag{5.2}$$

Thus, the layer-wise update norm is given by restricting $\Delta\theta_t$ to layer l

$$\|\Delta W_t^l\| = \|\eta_l \nabla_{W_t^l} \mathcal{L}_B\|. \tag{5.3}$$

For clarity the mini-batch size is again set to one in the following derivation, where a generalization to arbitrary mini-batch size is contained in the Appendix A.2. For a mini-batch size of $B = 1$ the layer-wise update norm can be written as

$$\left\|\Delta W_t^l\right\|^2 = \sum_{j=1}^{n} \sum_{k=1}^{d \wedge p} \left(\eta_{W^l} \frac{\partial L(f_t)}{\partial (W_t^l)_{jk}} \right)^2, \tag{5.4}$$

where the sum limit $d \wedge p$ refers to d for $l = 0$ and to p for $l = 1$. Using the results of the TPF derivation of Section 4.1 allows to write the layer-wise step size using random variables associated with the weight matrices W_t^0 and W_t^1. Furthermore, the layer-wise step size can be formulated as an empirical average over these random variables that form an empirical average of neuron-wise observables.

Case $l = 0$:

$$\begin{aligned}
\left\|\Delta W_t^0\right\|^2 &= \sum_{j=1}^{n} \sum_{k=1}^{d} \left(X^{(\Delta W_t^0)_{jk}} \right)^2 && (5.5)\\
&= \sum_{j=1}^{n} \underbrace{\sum_{k=1}^{d} \left(\eta\, \zeta_t X^{(d_{h_0})_j} (x_t)_k \right)^2}_{:= \tilde{X}_{t,j}^0} && (5.6)\\
&= n \underbrace{\frac{1}{n} \sum_{j=1}^{n} \tilde{X}_{t,j}^0}_{:= \langle \tilde{X}_{t,j}^0 \rangle_n} && (5.7)\\
&= n \langle \tilde{X}_{t,j}^0 \rangle_n && (5.8)
\end{aligned}$$

Case $l = 1$:

$$\left\|\Delta W_t^1\right\|^2 = \sum_{j=1}^{n}\sum_{k=1}^{p}\left(X^{(\Delta W_t^1)_{jk}}\right)^2 \tag{5.9}$$

$$= \sum_{j=1}^{n}\sum_{k=1}^{p}\left(-\frac{\eta}{n}\left(\zeta_t\right)_j X^{(z_0)_k}\right)^2 \tag{5.10}$$

$$= \frac{1}{n^2}\sum_{j=1}^{n}\underbrace{\sum_{k=1}^{p}\left(\eta\left(\zeta_t\right)_j X^{(z_0)_k}\right)^2}_{:=\tilde{X}_{t,j}^1} \tag{5.11}$$

$$= \frac{1}{n}\underbrace{\frac{1}{n}\sum_{j=1}^{n}\tilde{X}_{t,j}^1}_{:=\langle\tilde{X}_{t,j}^1\rangle_n} \tag{5.12}$$

$$= \frac{1}{n}\langle\tilde{X}_{t,j}^1\rangle_n \tag{5.13}$$

Here $\tilde{X}_{t,j}^l$ defines a neuron-wise observable, which allows to use the perspective of the empirical neuron distribution to understand the limiting behaviour of the empirical average. Therefore, all $\{\tilde{X}_{t,j}^l\}_{j=1\ldots n}$ are pairwise independent in the infinite-width limit, such that the empirical average $\langle\tilde{X}_{t,j}^1\rangle_n$ converges to the true expectation value, as in Equation 4.47, for an infinite number of neurons in the hidden layer ($n \to \infty$).

Ratio of Layer-Wise Update Step Size

This allows the ratio of the layer-wise update norms to be computed for two networks with widths n and $\tilde{n} = mn$ in the infinite-width limit. For the hidden layer the update norms read

$$\left\|\Delta W_t^0\right\|^2(n) = n\langle\tilde{X}_{t,j}^0\rangle_n, \tag{5.14}$$

$$\left\|\Delta W_t^0\right\|^2(\tilde{n}) = \tilde{n}\langle\tilde{X}_{t,j}^0\rangle_{\tilde{n}}, \tag{5.15}$$

so that their ratio becomes

$$\frac{\left\|\Delta W_t^0\right\|^2(mn)}{\left\|\Delta W_t^0\right\|^2(n)} = m\frac{\langle\tilde{X}_{t,j}^0\rangle_{mn}}{\langle\tilde{X}_{t,j}^0\rangle_n}. \tag{5.16}$$

The empirical expectation value converges to $\mu^0 := \mathbb{E}[\tilde{X}_{t,j}^0] = \lim_{n\to\infty}\langle\tilde{X}_{t,j}^0\rangle_n$, so that for $\mu^0 \neq 0$ the continuous mapping theorem [38] yields

$$\lim_{n\to\infty} \frac{\|\Delta W_t^0\|^2 (mn)}{\|\Delta W_t^0\|^2 (n)} = m. \tag{5.17}$$

The same reasoning applies to the output layer ($l = 1$), where the layer-wise update norm reads

$$\left\|\Delta W_t^1\right\|^2 (n) = \frac{1}{n}\langle \tilde{X}_{t,j}^1 \rangle_n, \tag{5.18}$$

$$\left\|\Delta W_t^1\right\|^2 (\tilde{n}) = \frac{1}{\tilde{n}}\langle \tilde{X}_{t,j}^1 \rangle_{\tilde{n}}, \tag{5.19}$$

so that the ratio for two networks with widths n and $\tilde{n} = mn$ is given by

$$\frac{\|\Delta W_t^1\|^2 (mn)}{\|\Delta W_t^1\|^2 (n)} = \frac{1/nm}{1/n} \frac{\langle \tilde{X}_{t,j}^1 \rangle_{mn}}{\langle \tilde{X}_{t,j}^1 \rangle_n} \tag{5.20}$$

$$= \frac{1}{m} \frac{\langle \tilde{X}_{t,j}^1 \rangle_{mn}}{\langle \tilde{X}_{t,j}^1 \rangle_n}. \tag{5.21}$$

For $\mu^1 := \mathbb{E}[\tilde{X}_{t,j}^1] = \lim_{n\to\infty} \langle \tilde{X}_{t,j}^1 \rangle_n \neq 0$ one obtains

$$\lim_{n\to\infty} \frac{\|\Delta W_t^1\|^2 (mn)}{\|\Delta W_t^1\|^2 (n)} = \frac{1}{m}. \tag{5.22}$$

This result can be generalized to arbitrary mini-batch size B used to compute the gradients in every training step. Furthermore, it is useful to monitor the gradients only every τ update steps, as this averages out fluctuations in the training dynamics and reduces the amount of data that has to be stored during training. A generalization of Equations 5.17 and 5.22 to arbitrary batch size, with weights saved only every τ training steps, is therefore provided in Appendix A.2. Since this generalization is the one relevant for the experiments and the scaling constants remain unchanged under it, the results are discussed based on the generalized case in the following. To indicate that τ consecutive update steps are accumulated and an arbitrary mini-batch size is used, the corresponding cumulative layer-wise step size is labeled as

$$\|u_{t,\tau}^l\|^2 := \left\| \sum_{s=t\cdot\tau}^{(t+1)\cdot\tau-1} \eta_{W^l} \nabla_{W_s^l} \mathcal{L}_{B_s}(f_s) \right\|^2. \tag{5.23}$$

As outlined in Section 4.2, expectation values of neuron-wise observables approximate the infinite-width training dynamics under μP based on the empirical neuron distribution, and it holds according to Equations 5.17 and 5.22 that

$$\lim_{n\to\infty} \frac{\left\|u^0_{t,\tau}\right\|^2 (mn)}{\left\|u^0_{t,\tau}\right\|^2 (n)} = m, \tag{5.24}$$

$$\lim_{n\to\infty} \frac{\left\|u^1_{t,\tau}\right\|^2 (mn)}{\left\|u^1_{t,\tau}\right\|^2 (n)} = \frac{1}{m}. \tag{5.25}$$

This motivates the definition of a layer-wise scaling constant $c^l(m)$, which quantifies the constant scaling factor in the infinite-width limit,

$$\lim_{n\to\infty} \frac{\left\|u^l_{t,\tau}\right\|^2 (mn)}{\left\|u^l_{t,\tau}\right\|^2 (n)} = c^l(m) = \begin{cases} m & l = 0, \\ \dfrac{1}{m} & l = 1. \end{cases} \tag{5.26}$$

Thus, the set of cumulative update norms $\left\|u^l_{t,\tau}\right\|^2 (mn)$ rescaled by $c^l(m)$ should follow the same trajectory in the infinite-width limit,

$$\frac{\left\|u^l_{t,\tau}\right\|^2 (mn)}{c^l(m)} = \left\|u^l_{t,\tau}\right\|^2 (n) \qquad \forall\, l, m. \tag{5.27}$$

Therefore, at a finite base width n the rescaled cumulative layer-wise update norms $\frac{\left\|u^l_{t,\tau}\right\|^2 (mn)}{c^l(m)}$ should collapse onto a single universal curve, as long as the corresponding finite empirical neuron distribution sufficiently approximates the deterministic infinite-width limit.

This theoretical prediction can be validated from Figure 5.1, which shows the raw and the rescaled cumulative layer-wise update norms. At initialization and during the early stages of training, the rescaled layer-wise update norms align closely across different widths, and this agreement improves with increasing network width. In late training stages, however, networks of smaller width show deviations, which arise because after a large number of training steps their empirical neuron distribution no longer provides a good approximation of the infinite-width limit.

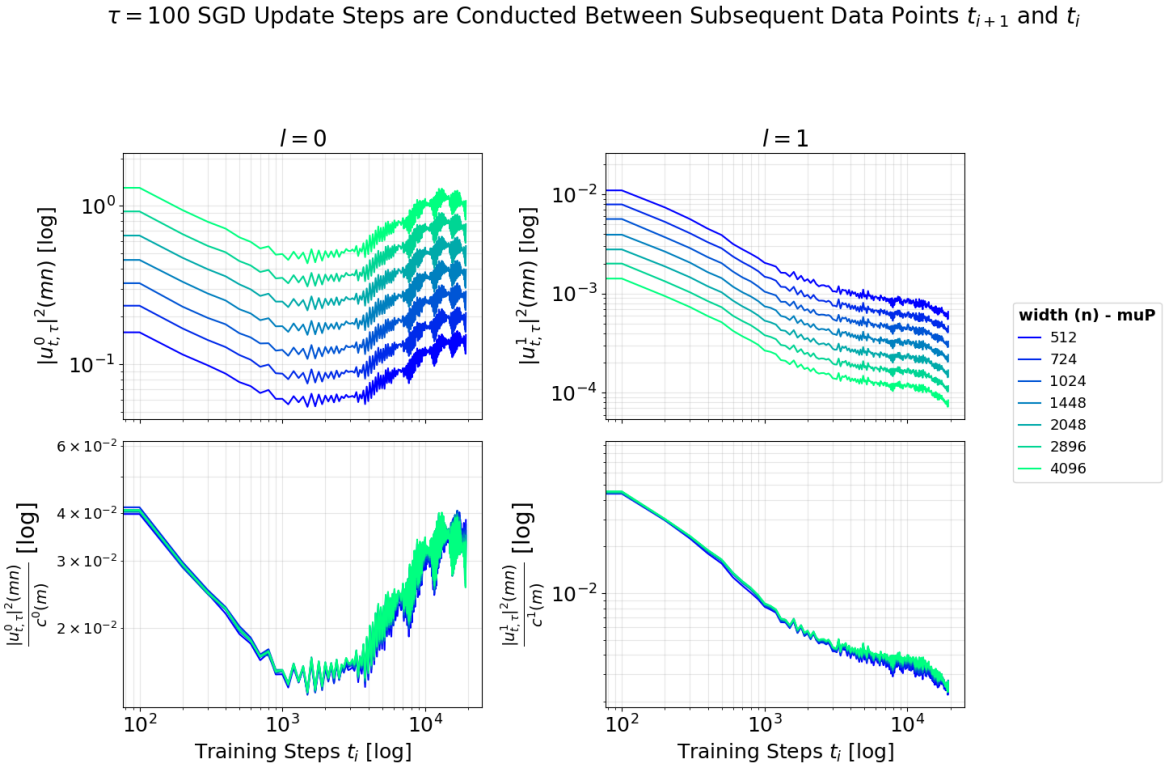


Figure 5.1: *Raw and rescaled layer-wise update norms for a set of one-hidden-layer MLPs trained on CIFAR10 under μP. The legend indicates the width of the hidden layer (n) of each network and the utilized parametrization. The left column refers to weight matrix W_t^0 and the right column to W_t^1. The base width (n) is always indicated by the width of the smallest network, which is $n = 512$ in this case, where $m = 2^{k/2}$ with $k = 0 \dots 6$.*
Rescaling the layer-wise update norms produces a precise collapse onto a universal trajectory, where the network with the smallest width shows the largest deviations in late training stages.

Loss Function

Consider again the output of a neural network $f_t(n)$ at some arbitrary time t in the TPF, as introduced in Equation 4.38. As already identified, each component of the network's prediction is a neuron-wise observable whose empirical average across all neurons yields the network's prediction,

$$(f_t)_j(n) = \frac{1}{n}\sum_{k=1}^{n} \underbrace{(\tilde{f}_t)_j(\theta_{t,k})}_{=(\tilde{f}_t)_{jk}} \tag{5.28}$$

$$= \langle (\tilde{f}_t)_j \rangle_n. \tag{5.29}$$

Therefore, analogously to Section 5.1, it holds for the outputs of two networks of width n and $\tilde{n} = mn$ that

$$(f_t)_j(n) = \langle (\tilde{f}_t)_j \rangle_n, \tag{5.30}$$

$$(f_t)_j(\tilde{n}) = \langle (\tilde{f}_t)_j \rangle_{\tilde{n}}, \tag{5.31}$$

and their ratio becomes deterministic in the infinite-width limit,

$$\lim_{n\to\infty} \frac{(f_t)_j(\tilde{n})}{(f_t)_j(n)} = \lim_{n\to\infty} \frac{\langle (\tilde{f}_t)_j \rangle_{mn}}{\langle (\tilde{f}_t)_j \rangle_n} \tag{5.32}$$

$$= 1. \tag{5.33}$$

Since the prediction of the network becomes deterministic in the infinite-width limit, the same holds for the test loss by the continuous mapping theorem [38]. This applies across regression and classification tasks, since both the cross-entropy and the MSE are continuous functions. The test loss on the test set $\mathcal{D}_{te}$ is

$$\mathcal{L}_{\mathcal{D}_{te}}(f_t(n)) = \frac{1}{|\mathcal{D}_{te}|} \sum_{x_i \in \mathcal{D}_{te}} L\Big(f_t(x_i)\Big), \tag{5.34}$$

such that its ratio between two widths satisfies

$$\lim_{n\to\infty} \frac{\mathcal{L}_{\mathcal{D}_{te}}(f_t(\tilde{n}))}{\mathcal{L}_{\mathcal{D}_{te}}(f_t(n))} = 1. \tag{5.35}$$

As with the update norms in Figure 5.1, the loss curves collapse across different network sizes during the initial training stages, as can be verified in Figure 5.2. As Equations 5.33 and 5.35 show, no explicit scaling constant is required for the loss functions, in contrast to the layer-wise update step size, in order to obtain empirical averages of

neuron-wise observables that approach the same universal infinite-width limit. In late training stages, where a clear width dependence of the loss curves becomes visible, the finite-width networks again deviate from the infinite-width limiting behaviour. Since the network's output couples into every parameter update as well as into the loss function, it is the common source of the structural deviation of finite-width networks from the infinite-width limit, which also explains the width hierarchy in the observed deviations. More precisely, networks with small widths deviate earlier and further from the infinite-width training dynamics, since the displacement of the network's prediction, which shifts the training dynamics away from the infinite-width limiting behaviour, scales as $O(n^{-1/2})$ by the Central Limit Theorem (CLT).

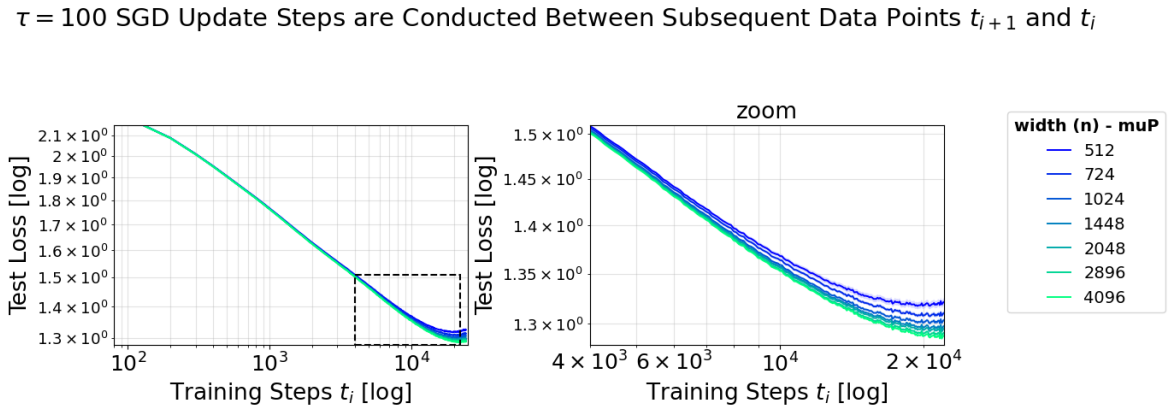


Figure 5.2: *Test loss of a set of one-hidden-layer MLPs trained on CIFAR10 under μP. The legend indicates the width of the hidden layer (n) of each network and the utilized parametrization. The plot shows the evolution of the test loss throughout the training process, which provides an unbiased estimator of the network's generalization performance. The base width (n) is always indicated by the width of the smallest network, which is $n = 512$ in this case, where $m = 2^{k/2}$ with $k = 0 \dots 6$. During the initial phases of training the test loss collapses onto a universal curve. In late training stages the accumulated finite-width displacements, which scale as $O(n^{-1/2})$, produce a clear hierarchy of deviations across the different network widths. The loss curves show averages across five independent training runs that used independent initializations of weights.*

5.2 Width-Dependent Deviations under SP

While empirical averages of neuron-wise observables collapse across widths under μP, a clear and systematic width hierarchy is visible under SP from the earliest stages of training. This behaviour follows directly from the coordinate-distribution analysis of Section 3.2 applied to SP. The full derivation is given in Appendix A.3, and only the central result is discussed here, which is consistent with the known infinite-width limit under SP [9, 26, 39].

Under SP, the initialization $(W_0^1)_{jk} \sim \mathcal{N}(0, \frac{1}{n})$ and the absence of the $1/n$ readout normalization change the probabilistic character of the network's output. The output at initialization is a CLT-type sum of n zero-mean terms of size $O(n^{-1/2})$ and therefore converges only in distribution to a Gaussian Process (GP), rather than to a deterministic value as is the case for μP. Each SGD step then adds an LLN-type correction to the GP, so that after one update step the network's output can be formulated as in Equation 5.37 (Appendix A.3). In this expression the second line expands the empirical expectation value of a finite-width network $\langle X^{z_0} X^{z_1} \rangle_n$, which is based on only n samples, into the true expectation value $\mathbb{E}\left[X^{z_0} X^{z_1}\right]$ obtained from the infinite-width limit plus corrections of leading order $O(n^{-1/2})$,

$$(f_1)_j = \sum_{k=1}^{n} (W_0^1)_{jk} (z_1)_k - \eta\, \zeta_0\, n \langle X^{z_0} X^{z_1} \rangle_n \tag{5.36}$$

$$= \underbrace{\sum_{k=1}^{n} (W_0^1)_{jk} (z_1)_k}_{O(1)\text{ Gaussian (CLT)}} - \underbrace{\eta\, \zeta_0\, n}_{O(\eta\, n)} \underbrace{\mathbb{E}\left[X^{z_0} X^{z_1}\right]}_{\text{(LLN)}} + O(\sqrt{n}\, \eta). \tag{5.37}$$

Analogously, after every training step the output at time t is a persistent $O(1)$ Gaussian initialization term plus a sum of update corrections, each of which scales as $\eta\, n$. Under μP, by contrast, the $1/n$ readout normalization ensures that the output evolves as a single deterministic expectation value $\mathbb{E}[X^{nW_t^1} X^{z_t}]$ that is width-independent to leading order at any fixed η.

The width dependence of the network's output under SP cannot be compensated by the learning rate alone. Scaling the learning rate as $\eta = O(n^{-1})$ restores width-independent leading-order dynamics, but suppresses feature learning even more strongly. The first-layer coordinate updates drop from $O(n^{-1/2})$ to $O(n^{-3/2})$, as already outlined in Section 3.1.3, and the dynamics reduce to those of a frozen feature kernel around the random Gaussian initialization. Note that the feature kernel is already frozen in the infinite-width limit at fixed η, since in this case the first-layer coordinate updates are already of $O(n^{-1/2})$, so that the rescaling of the learning rate does not introduce the feature freezing but only reinforces it.

μP resolves these problems of SP by construction. Under SP the trajectories are width-inconsistent and ultimately unstable, feature learning is suppressed, and the features freeze in the infinite-width limit, whereas all of these issues are absent under μP. The collapse of the μP observables together with the SP hierarchy analyzed here provides a direct experimental visualization of this problem. In addition, this perspective offers an intuitive explanation for why μP allows the learning rate to be transferred across width. In the infinite-width limit the training dynamics under μP are width-independent, so that the optimal learning rate attains a well-defined width-independent value. Since finite-width networks approach this limit with corrections of order $O(n^{-1/2})$, their optimal learning rate converges to the limiting value as the

width is increased and is therefore roughly invariant across model width. This is what enables the optimal learning rate to be tuned at a small base width and transferred to much wider networks. In contrast, SP setups require the learning rate to be readjusted whenever the network's width is increased, since the corrections are of $O(\eta\, n)$ and grow linearly with width, so that each width necessarily has a different optimal learning rate. In the training setup used here, one global learning rate is applied across all widths for the networks trained under SP. Thus, the width hierarchy in the deviations of the finite-size networks can be observed at all phases of training, including the initial training stages, since the corrections to the infinite-width limit scale as $O(\eta\, n)$ and are therefore directly proportional to the network width. These corrections are much stronger than under μP, where the leading-order corrections are of $O(n^{-1/2})$. Hence, under SP the layer-wise update norm and the loss function show structural deviations throughout the entire training process, and the clear width hierarchy in the early training stages is explained in particular by the strong width dependence of the leading-order corrections in Equation 5.37.

This width hierarchy is also reflected in the generalization performance of the networks. Under μP the observables collapse across width, and increasing the network's width systematically improves the test loss, since wider networks approximate the feature-learning infinite-width limit more closely. Under SP the growing $O(\eta\, n)$ corrections push wider networks further away from a width-consistent limit at the fixed global learning rate, so that increasing the width does not produce the same systematic improvement in generalization performance. This behaviour is analyzed in more detail from a spectral perspective in the following section.

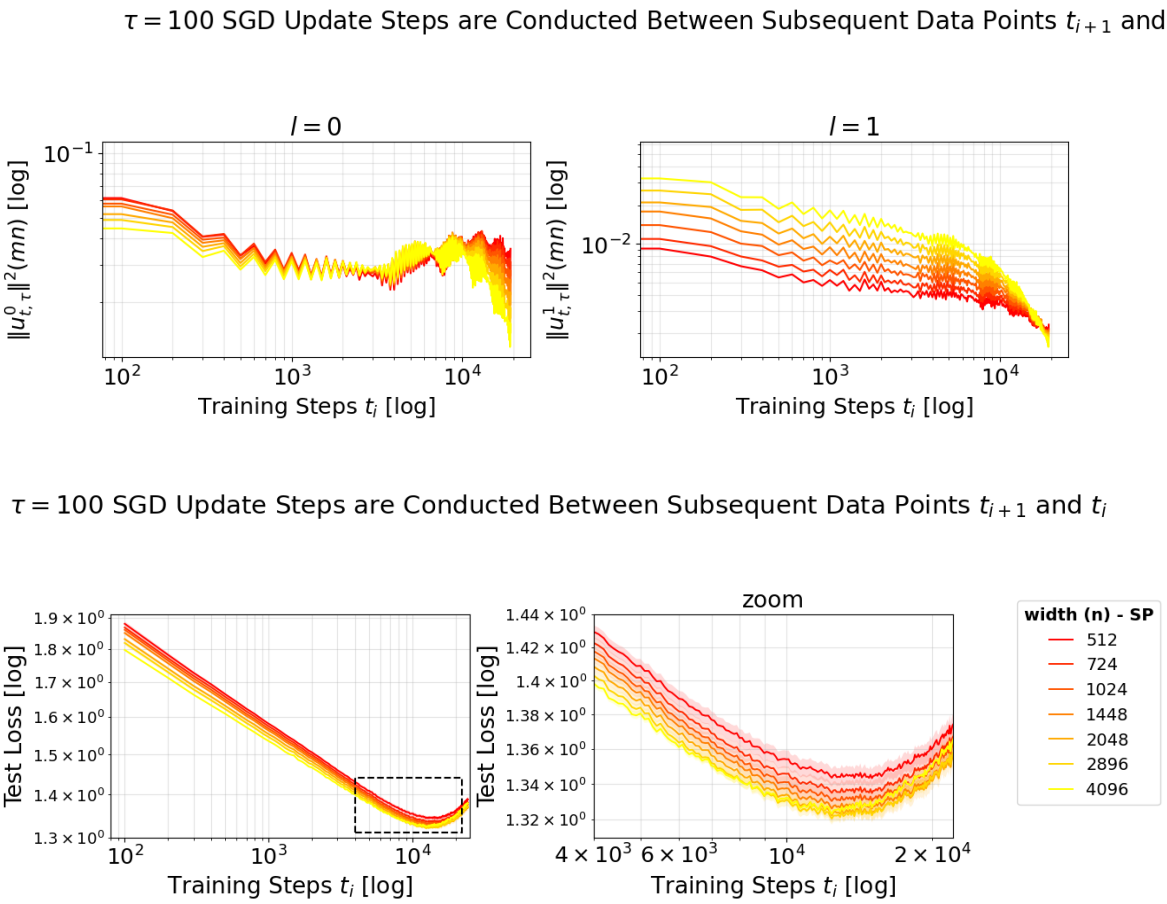


Figure 5.3: *Layer-wise update norms and loss function for a set of one-hidden-layer MLPs trained on CIFAR10 under SP. The legend indicates the width of the hidden layer (n) of each network and the utilized parametrization. The left column refers to weight matrix W_t^0 and the right column to W_t^1 in the top row of plots. The base width is always indicated by the width of the smallest network, which is $n = 512$ in this case, where $m = 2^{k/2}$ with $k = 0 \dots 6$. The bottom row shows the test loss of all network widths averaged across five independent training runs. In comparison to μP, the layer-wise update norms and the test loss show a clear width dependence under SP that cannot be resolved by constant rescaling. The loss curves show averages across five independent training runs that used independent initializations of weights.*

5.3 A Spectral Perspective on Feature Learning

In sections 5.1 and 5.2 neuron-wise observables built the foundation to explain the universal behaviour of scalar training observables such as the layer-wise update norm and the test loss. The same framework is now applied to an observable that probes feature learning directly, namely the overlap structure between the hidden representations of the individual input components. Since every entry of this overlap structure is again an empirical average of a neuron-wise observable, the universality established in the previous sections carries over to the feature-learning analysis developed here.
Feature learning is the central property that distinguishes neural networks from classical machine learning models. It refers to the ability of a neural network to independently identify meaningful structures in the data and to adjust its internal representations accordingly. A model without this ability merely fits a fixed random feature map, which is known as the lazy-learning or kernel regime. The following section develops a geometric view of feature learning in which the central objects are the hidden representations of the individual input components.

Each input component owns a direction in hidden space, and the model can independently adapt during training whether these hidden representations overlap or remain orthogonal. At random initialization the directions are near-orthogonal and unstructured as each entry of all hidden representation is randomly sampled from a centered Gaussian distribution. A feature-learning network instead aligns the directions of input components that should be jointly evaluated, which results in a non-zero overlap of the corresponding hidden representations. In the considered example case where each data sample represents an image, input components that correspond to nearby pixels form edges, textures, and objects are therefore expected to have overlapping hidden representations. The degree and the structure of these overlaps can thus be read as a signature of feature learning, and the perspective of the empirical neuron distribution is once again used to explain the universal behaviour observed across widths. Finally, the spectral decomposition of the overlapping hidden representations is analyzed to investigate the spectral evolution during training, which reveals the set of features learned by the network throughout the training process. This feature hierarchy is exploited to perform a remarkable dimensionality reduction of the trained model, which preserves the performance of the full-rank model while requiring only a fraction of its parameters. In the following the key concepts are introduced referring to an image-type dataset, as this represents an instructive example. The methods are, however, not restricted to image datasets and can generally be applied to any classification or regression task.

Hidden Representations

To simplify the notation, the time and layer indices are omitted in the following sections, since only the weight matrix W^0 is discussed and the training steps are not

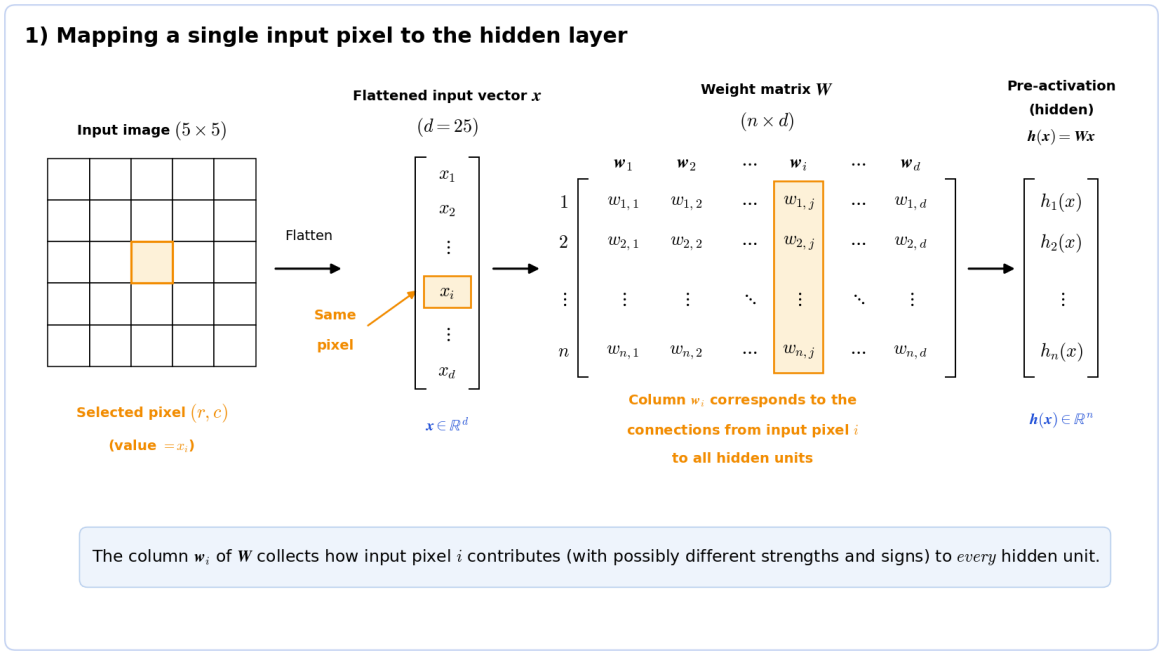


Figure 5.4: *A pixel x_i of the flattened input selects the column w_i of the first-layer weight matrix $W \in \mathbb{R}^{n \times d}$. The hidden pre-activation $h(x) = Wx = \sum_i x_i w_i$ is the superposition of these per-pixel directions. Every input component thus owns one direction in the n-dimensional hidden space.*

considered explicitly. The analysis focuses on the weight matrix W^0, since in this experimental setup W^1 already represents the readout layer, which is optimized during training to linearly combine the features z built by the model into the final prediction. Consequently, the evolution of W^0 during training constitutes the central object of feature learning in this one-hidden-layer architecture, since any change in the learned representations must manifest itself in W^0, whereas W^1 merely determines how these features are weighted in the output. The first-layer weight matrix maps an input sample x to the model's hidden representation h according to

$$W = [\, w_1, \ldots, w_d \,] \in \mathbb{R}^{n \times d}, \qquad x = (x_1, \ldots, x_d)^\top, \tag{5.38}$$

$$h(x) \;=\; Wx \;=\; \sum_{i=1}^{d} x_i \, w_i. \tag{5.39}$$

Here column i of W is denoted by w_i, and each input component x_i is represented by the corresponding n-dimensional vector w_i in the model's hidden space. The superposition of all these hidden-space vectors weighted by the input components x_i forms the final hidden representation h of the input sample x. This mapping from input samples to hidden representations is illustrated in Figure 5.4 using a flattened image as an example data sample.

Overlap of Hidden Representations

The hidden representations $\{w_i \in \mathbb{R}^n\}_{i=1...d}$ that correspond to different input components can either be mutually orthogonal or overlap, which is adjusted by the model during training. The geometric overlap of the trained model's hidden representations therefore offers a direct way to interpret how the model learned to combine different input components in order to identify meaningful structures in the data.

- For orthogonal columns with $w_i^\top w_j = 0$ the input components i and j write into independent hidden directions. Each input component influences the hidden state as an isolated coordinate, and no combination of hidden units sees the two input components jointly. The network therefore treats the input as a collection of unrelated input components.
- For overlapping columns with $w_i^\top w_j \neq 0$ the hidden representations partially share directions. The input components i and j thus influence each other in the space of hidden representations, since they jointly contribute to the resulting combination of hidden units. Overlap can therefore be interpreted as the mechanism by which the first layer binds input components into composite features.

These two cases are in the following illustrated in an idealized basis in which every hidden representation is aligned with one of the standard basis vectors of the hidden space $\{e_i := (0, ..., 1, ...0)^\top \mid e_i \in \mathbb{R}^n\}_{i=1}^n$. If the input component x_1 is represented by $w_1 = c_1 e_1$ and x_2 by $w_2 = c_2 e_2$ with $e_1 \perp e_2$, the two representations are orthogonal. The set of neurons excited by x_1 is then disjoint from the set excited by x_2, and the hidden pre-activation $h(x) = \sum_i x_i w_i$ reduces to a rescaled copy of the input in which each input component acts on its own coordinate. The first-layer activations $z = \sigma(h)$ therefore process every input component in isolation, but do not combine input components to form abstract features. If instead x_1 and x_2 are represented along the same direction, for example $w_1 \parallel w_2 \parallel e_1$, their contributions add along the first coordinate, so that the first hidden unit responds to the combined quantity $x_1 + x_2$ and represents a joint feature of the two input components. In this idealized basis the meaning of superposition is immediately clear. Orthogonal representations correspond to input components that the network treats as unrelated, whereas overlapping representations correspond to input components that the network binds into a common feature.
Referring again to the example of an image, the overlap structure of the hidden representations reveals which pixels are jointly evaluated through their non-zero overlap, as illustrated in Figure 5.5.
However, in a trained network the hidden representations generally do not align with the standard basis vectors of the hidden space, so the idealized picture cannot be read off the individual coordinates directly. What survives is the geometric relation between the representations rather than their absolute orientation. Under an orthogonal rotation Q of the hidden space every representation is mapped to Qw_i, while the pairwise overlaps are preserved, since $(Qw_i)^\top (Qw_j) = w_i^\top w_j$, whereas the individual coordinates

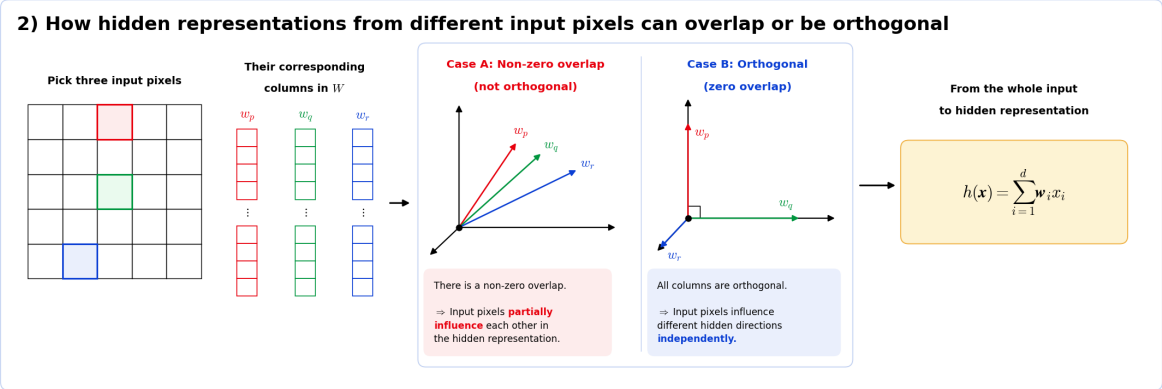


Figure 5.5: *The figure shows three pixels p,q,r and their corresponding hidden representations w_p, w_q, w_r. In case A the columns have non-zero overlap. The pixels partially influence each other inside $h(x)$, which allows hidden units to respond to joint multi-pixel patterns. In case B the columns are orthogonal and the pixels act on independent hidden directions. This overlap structure of the columns determines how input components interact in the hidden representation.*

of each w_i are not. The idealized basis used above is therefore only one representative of an entire equivalence class of hidden bases that share the same overlap structure. A natural basis will be provided later by the collective modes of the superposition matrix in Equation 5.42, which form an orthonormal basis of the hidden space along which the strongest overlap structure is identified, so that the basis vectors are taken by these collective directions.

This overlap structure is precisely the object that is engineered during feature learning. The first-layer map $z = \sigma(Wx)$ is a feature map that sends the raw input to a new representation, and the columns w_i determine which combinations of input components each hidden coordinate responds to. In the natural basis of collective modes from Equation 5.42 this construction becomes explicit, since each collective direction responds to a specific learned combination of input components and thereby acts as a detector for a new feature. The engineering of the overlap structure is therefore the same process as the construction of a data-adapted feature map, in which the model learns which input components to bind into meaningful features rather than treating them as unrelated coordinates. The superposition matrix directly represents this feature map, since its collective modes represent the new features that the network has built during training.

The Superposition Matrix

The entire overlap structure between the hidden representations is summarized by the normalized Gram matrix

$$C_{ij} = \frac{1}{n} w_i^\top w_j, \qquad C = \frac{1}{n} W^\top W, \tag{5.40}$$

which is referred to as the superposition matrix in the following. Each entry C_{ij} represents an empirical average over a neuron-wise observable, which directly links the feature-learning analysis of this section to the framework of the empirical neuron distribution from Sections 5.1 and 5.2. Row or column j of C states how strongly the hidden representation of input component j overlaps with those of all other input components.

Using again the example in which the input data sample is represented by an image, each column of the superposition matrix can be transformed back into the original image geometry. This visualizes which pixels of the input image are jointly evaluated by the trained model through the overlap of their hidden representations, as illustrated in Figure 5.6. The superposition matrix at initialization and the corresponding input-space reconstructions for three different reference pixels are shown in Figure 5.7. Comparing the superposition matrix at initialization, where no structure is visible, to the superposition matrix after training reveals that the model has learned to jointly evaluate pixels in the local vicinity of the reference pixel. The network therefore learned to evaluate local patterns that depend on multiple input pixels in order to identify structures in the image during training.

Since the superposition matrix is composed of empirical averages of neuron-wise observables, the emergence of structure can only be caused by an evolution of the empirical neuron distribution during training, which is precisely what feature learning refers to. Comparing the training dynamics of SP and μP shows that feature learning occurs during training in particular for networks of large width trained under μP, which results in the emergence of meaningful structure in the superposition matrix. In comparison, under SP the evolution of the superposition matrix is strongly suppressed, as can be verified by comparing Figures 5.7 and 5.8, as no meaningful structure is visible in the superposition matrix.

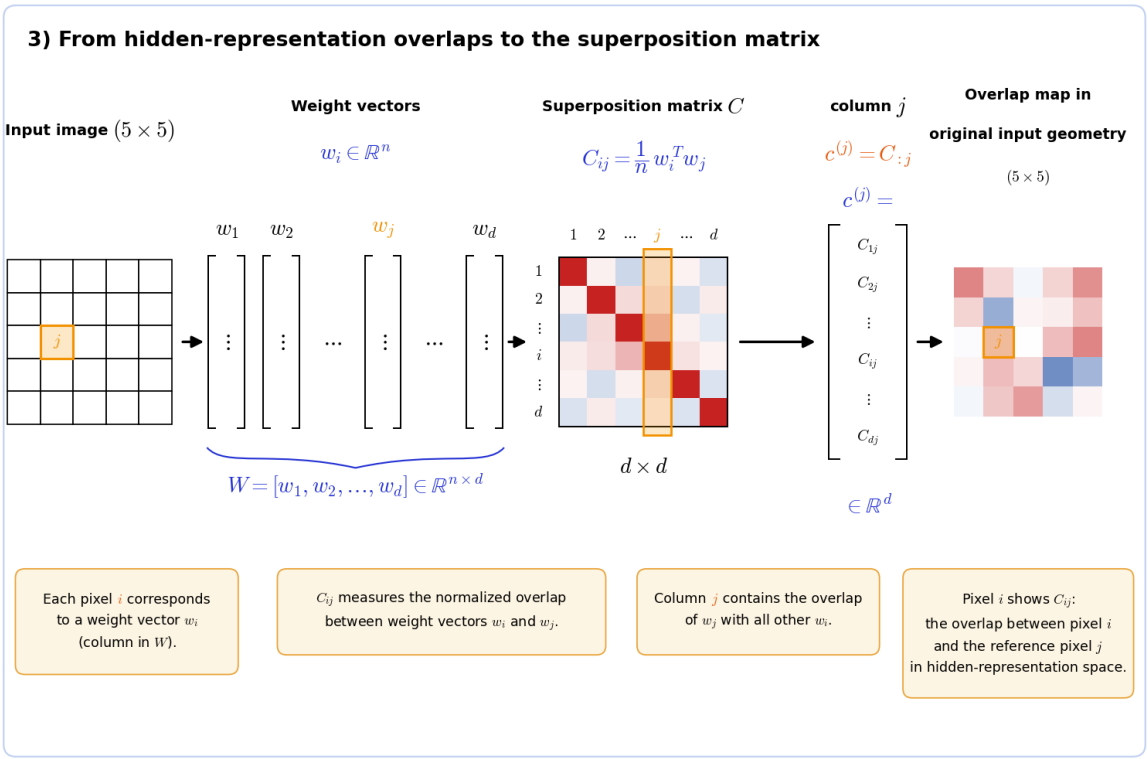


Figure 5.6: *The normalized Gram matrix, also called the superposition matrix, summarizes the overlap structure across all input components. Column j of C contains the overlap of the reference pixel j with every other pixel. It can be reshaped into the original image geometry, which gives an interpretable overlap map. Positive and negative values in red and blue indicate a non-zero overlap between the hidden representations, and values near zero in white indicate independent directions.*

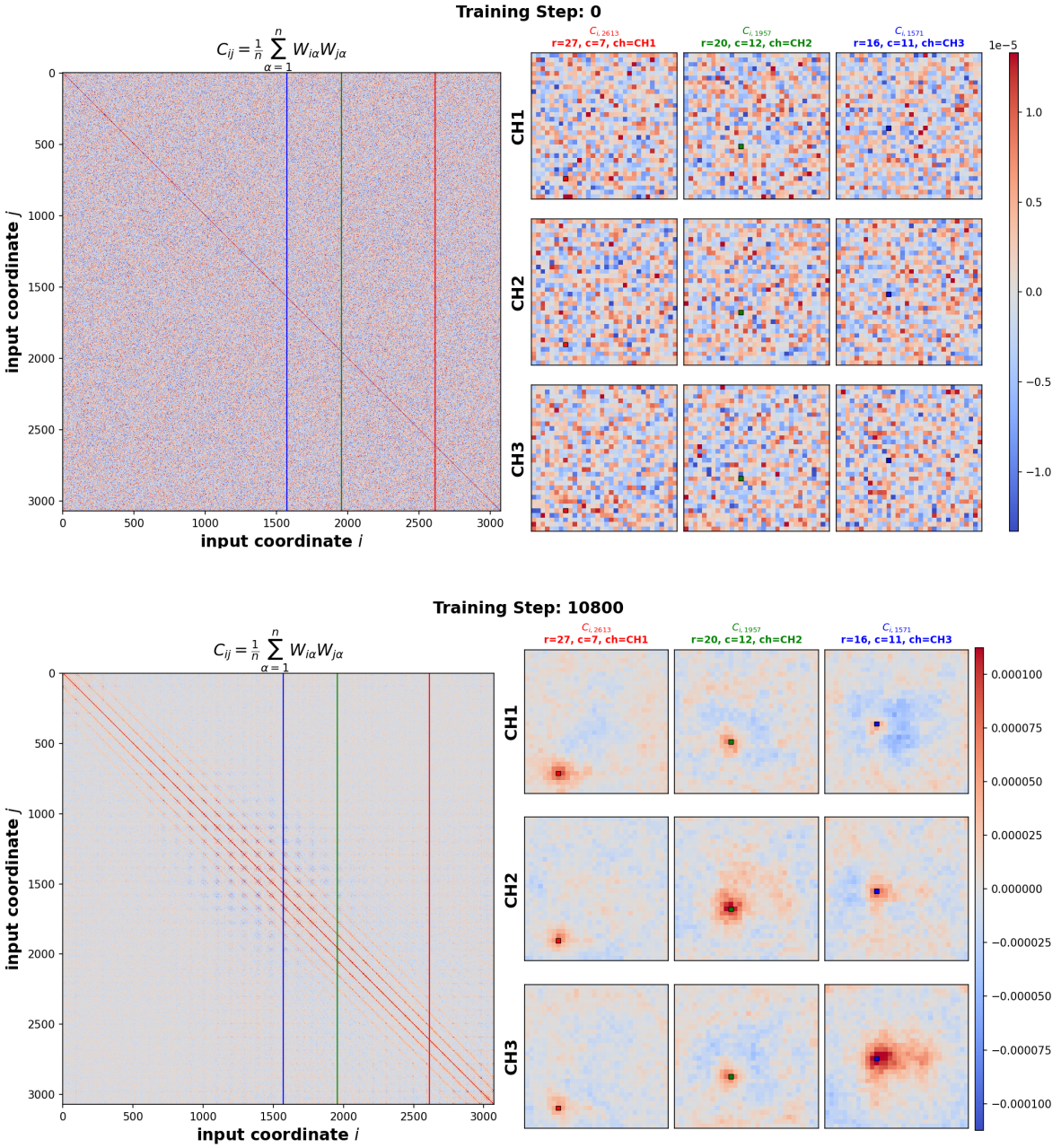


Figure 5.7: *The superposition matrix is shown before and after the training process for a one-hidden-layer MLP ($n = 4096$) trained on CIFAR-10 under μP. Next to the superposition matrix the overlaps of the hidden representations are visualized in the original image geometry across the three input channels of each pixel. This retransformation is shown for three different reference pixels indicated by the blue, green and red line in the superposition matrix. Under μP it is clearly visible that the network learned to jointly evaluate pixels in the local vicinity of the reference pixel during training, which indicates feature learning, since the superposition matrix is composed of empirical averages of neuron-wise observables. The superposition matrix after the training process corresponds to the training step at which the network achieved the best generalization performance, i.e. the lowest test loss.*

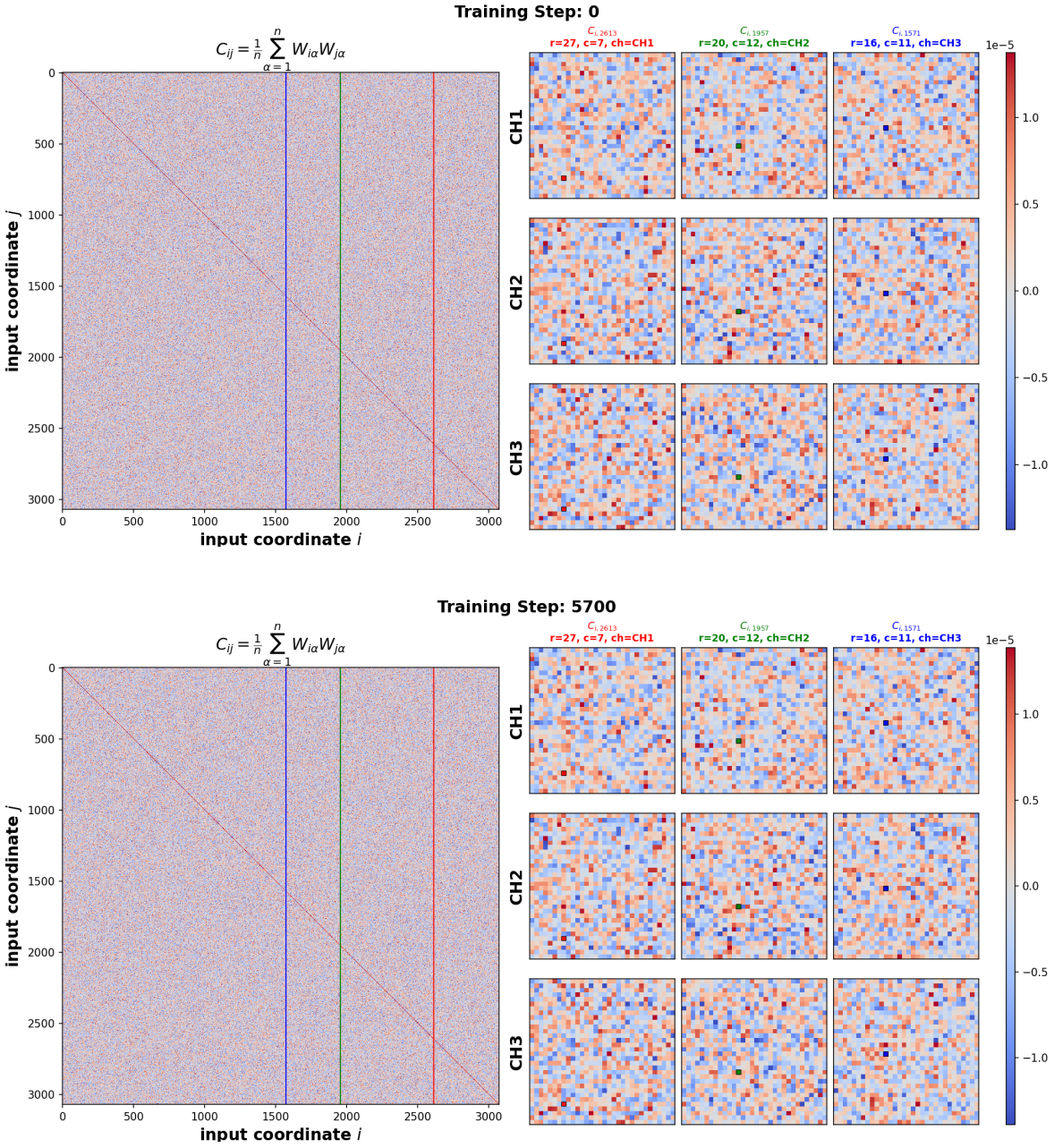


Figure 5.8: *The superposition matrix is shown before and after the training process for a one-hidden-layer MLP ($n = 4096$) trained on CIFAR-10 under SP. Next to the superposition matrix the overlaps of the hidden representations are visualized in the original image geometry across the three input channels of each pixel. This retransformation is shown for three different reference pixels indicated by the blue, green and red line in the superposition matrix. Under SP no structure emerged in the superposition matrix during the training process due to the suppression of feature learning. The superposition matrix after the training process corresponds to the training step at which the network achieved the best generalization performance, i.e. the lowest test loss.*

Evolution of the Eigenspectrum

The evolution of the superposition matrix can be understood quantitatively through its eigenspectrum,

$$C \;=\; V\Lambda V^{\top}, \qquad \Lambda = \mathrm{diag}(\lambda_1, \ldots, \lambda_r), \quad r = \mathrm{rank}(C) \leq \min(n,d). \tag{5.41}$$

Each eigenvector $v_\alpha \in \mathbb{R}^d$ lives in input space and corresponds to the pixel pattern that describes which input components participate in collective mode α. The eigenvalue λ_α measures how much of the total correlation of the hidden representations is carried by that mode. The corresponding direction in the network's hidden space is obtained by projecting the mode back through W,

$$u_\alpha \;=\; \frac{1}{\sqrt{\lambda_\alpha}} W\, v_\alpha \;\in\; \mathbb{R}^n, \qquad \lambda_\alpha > 0, \tag{5.42}$$

where the set $\{u_\alpha\}_{\alpha=1}^{r}$ is orthogonal. Every hidden representation can then be written exactly as a superposition of these collective directions,

$$w_i \;=\; \sum_{\alpha=1}^{r} \sqrt{\lambda_\alpha}\, v_{i\alpha}\, u_\alpha. \tag{5.43}$$

The overlap structure established during training can be separated from random overlaps by comparing the eigenspectrum against a decorrelated null spectrum. The null spectrum is obtained by permuting the neuron indices independently within each column of W^0, which preserves the norm and the marginal statistics of every column but destroys all correlations between different columns. The resulting matrix therefore describes a network with identical per-column weight statistics that has not learned any joint structure, and its spectrum serves as the reference against which learned correlations are identified at every stage of training. At random initialization the columns of W^0 are uncorrelated by construction, so that the measured spectrum is that of a random Gram matrix and coincides with the decorrelated null spectrum. In this case the entire spectral structure is described analytically by the Marchenko–Pastur distribution [40].

During training, hidden representations that should be jointly evaluated are aligned along shared directions in hidden space. This alignment results in the emergence of collective eigenmodes of the superposition matrix that deviate from the random structure present at initialization. The emerging eigenmodes can be identified in the eigenspectrum as detached eigenvalues, which exceed the decorrelated null spectrum and can therefore no longer be explained by a matrix without cross-column correlations. The eigenspectra in Figure 5.10 reveals that the suppression of feature learning under SP causes the number of detached eigenvalues to decrease with increasing network width. Under μP the network instead learns meaningful cross-correlations between hidden representations, which is revealed by a large set of detached eigenvalues at every width, as shown in Figure 5.9.

This behaviour follows directly from the perspective of the empirical neuron distribution, since every entry of the superposition matrix is an empirical average of a neuron-wise observable and therefore converges to a deterministic limit. Under μP the limiting neuron distribution evolves during training, and the detached eigenmodes represent this evolution at every width, whereas under SP the limiting distribution remains frozen, so that the detached eigenvalues only reflect finite-width corrections that vanish with increasing width. In addition, the decorrelation preserves the trace of the superposition matrix, since the column norms remain unchanged under the permutation. The sum across all eigenvalues of the measured and the null spectrum is therefore identical, and the detached eigenvalues must be compensated by the remaining part of the spectrum. The measured bulk consequently falls below the decorrelated null spectrum, which is visible in the figures as the deflation of the bulk for the eigenvalues that are not detached.

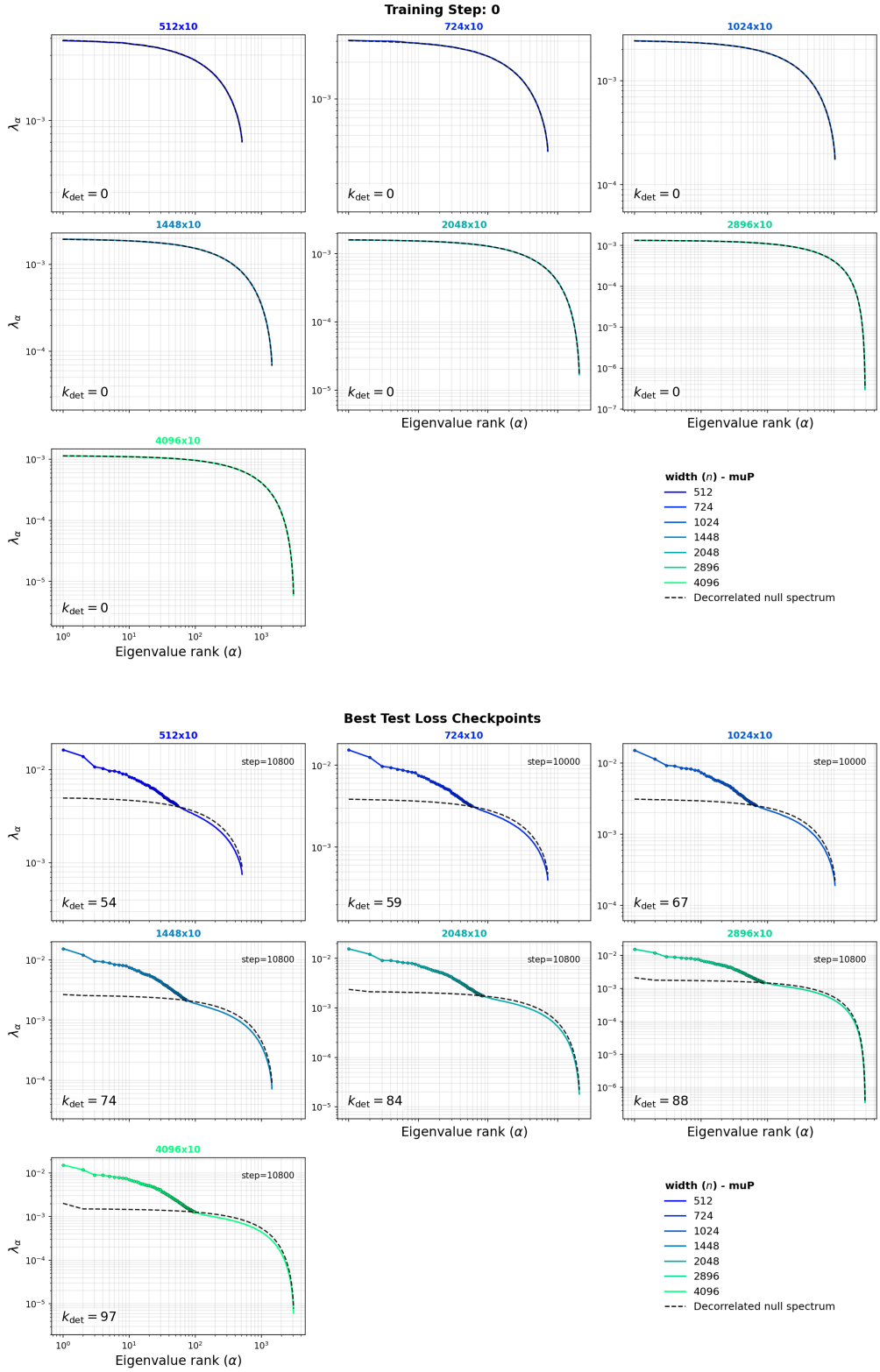


Figure 5.9: *Eigenvalue spectra of the superposition matrix for one-hidden-layer MLPs trained on CIFAR10. The legend indicates the width of the hidden layer (n) of each network and the utilized parametrization. The plots show the eigenvalue spectra before training (top) and at the training step at which each network achieved the best generalization performance (bottom). Under μP detached eigenvalues emerge during training that correspond to collective modes shared across hidden representations as a result of feature learning.*

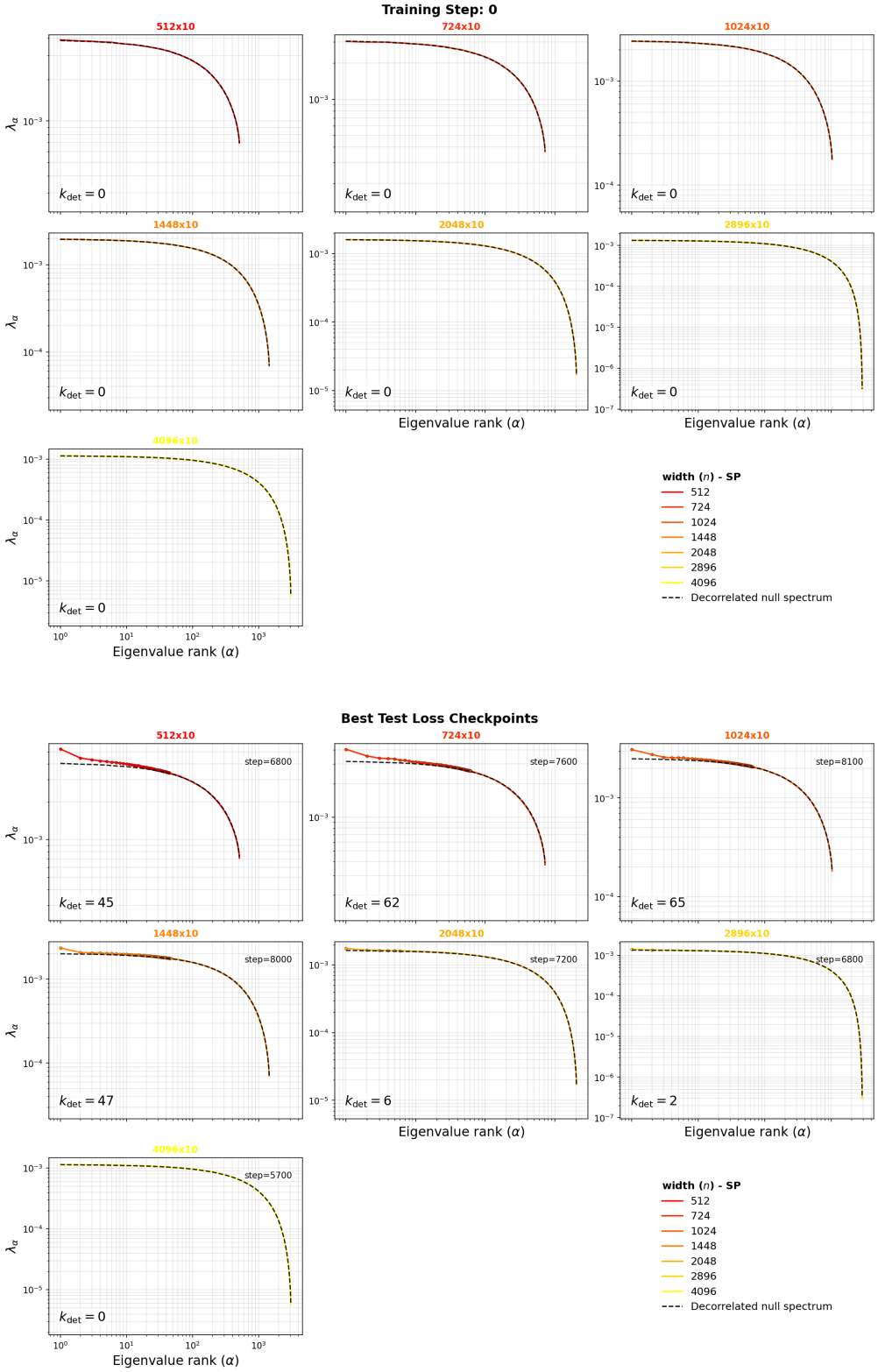


Figure 5.10: *Eigenvalue spectra of the superposition matrix for one-hidden-layer MLPs trained on CIFAR10. The legend indicates the width of the hidden layer (n) of each network and the utilized parametrization. The plots show the eigenvalue spectra before training (top) and at the training step at which each network achieved the best generalization performance (bottom). Under SP the decay of feature learning suppresses the emergence of collective modes and ensures that the spectra approach the decorrelated null spectrum, which describes a superposition matrix that contains no cross-column correlations.*

5.3.1 Dimensionality Reduction Through PCA

The low-rank structure that emerges during training under μP can be exploited to reduce the effective number of parameters required to make predictions with the trained network. Truncating the expansion of the hidden representations to the k leading modes yields the rank-k approximation of the actual hidden representation w_i after training,

$$\hat{w}_i^{(k)} \;=\; \sum_{\alpha=1}^{k} \sqrt{\lambda_\alpha}\, v_{i\alpha}\, u_\alpha \;\approx\; w_i, \qquad k < r, \tag{5.44}$$

where the residual $\varepsilon_i^{(k)} = w_i - \hat{w}_i^{(k)}$ collects the discarded modes and quantifies the truncation error. Applying this truncation to all hidden representations simultaneously replaces the original first-layer weight matrix by a low-rank approximation. The matrix $V^{(k)} \in \mathbb{R}^{d\times k}$ with $k \ll d$ contains the k dominant principal components of the superposition matrix. The input is projected onto these directions, and the hidden state is subsequently approximated from the projected input, so that the original mapping from the input to the hidden representation $h = Wx$ is replaced by

$$\hat{h}^{(k)} = A\, y, \qquad A = W V^{(k)}, \qquad y = V^{(k)\top} x, \tag{5.45}$$

with $W \in \mathbb{R}^{n\times d}$, $A \in \mathbb{R}^{n\times k}$, $V^{(k)} \in \mathbb{R}^{d\times k}$, and $y \in \mathbb{R}^k$. The input is projected onto the k leading principal directions collected in $V^{(k)}$, and the compressed layer A is obtained by evaluating the trained weight matrix on these directions. The resulting hidden pre-activation $\hat{h}^{(k)}$ is the rank-k approximation of the actual pre-activation $h = Wx$, with an approximation error that is controlled by the truncation residuals $\varepsilon_i^{(k)}$ of the individual hidden representations. The original weight matrix, which contains $P_{\text{full}} = nd$ parameters, is thus replaced by the projection of the input data through $V^{(k)}$ and the evaluation of the projected input through A, both of which must be stored and applied when an input is processed by the model. The approximated hidden representations $\hat{h}^{(k)}$ require only $P_k = nk + dk$ parameters, so that the compression ratio relative to the original hidden representations h is

$$R_k = \frac{P_k}{P_{\text{full}}} = \frac{k(n+d)}{nd}. \tag{5.46}$$

For $k \ll d$ the mapping from an input data sample to the corresponding hidden representation is therefore strongly compressed. The compression is a direct consequence of the feature hierarchy established during training, since the quality of the rank-k approximation is determined by the fraction of the spectral energy carried by the k leading modes. Under μP the detached eigenmodes contain this energy almost entirely, so that the predictive performance of the truncated model remains close to the performance of the full-rank model. Under SP no such privileged low-dimensional subspace is generated during training, so that the same truncation discards most of the learned structure. This can be validated from Figure 5.11, which reveals that the num-

ber of detached eigenmodes, which is denoted by $k = k_{det}$, for networks trained under μP contain almost the entire structure the model built during training, with a relative performance error below 10% across all widths. The set of detached eigenvalues was identified in this case by the set of eigenvalues that exceed the eigenvalue in the decorrelated null spectrum at the same rank by 10 %. The exact threshold is not important for the networks trained under μP, as the slope of the original eigenspectrum is steep at the point where the two spectra intersect. This allows the widest network to achieve a relative performance error of only $\frac{L-L_{\text{full}}}{L_{\text{full}}} \approx 2\%$, while the approximated network built from the $k = k_{det}$ leading eigenmodes contains only 5% of the number of parameters of the full model. The number of parameters contained in the output layer is negligible here as $p << d$. This reveals a remarkable dimensionality reduction that simultaneously preserves performance, which is enabled by feature learning during training and is reflected in the large number of detached eigenvalues in the eigenspectrum of the superposition matrix. In comparison, networks trained under SP show a strongly deteriorating performance when the network is approximated from the leading eigenmodes, since especially for wide networks feature learning is highly suppressed, so that no collective modes emerge that could form the foundation of this dimensionality reduction.

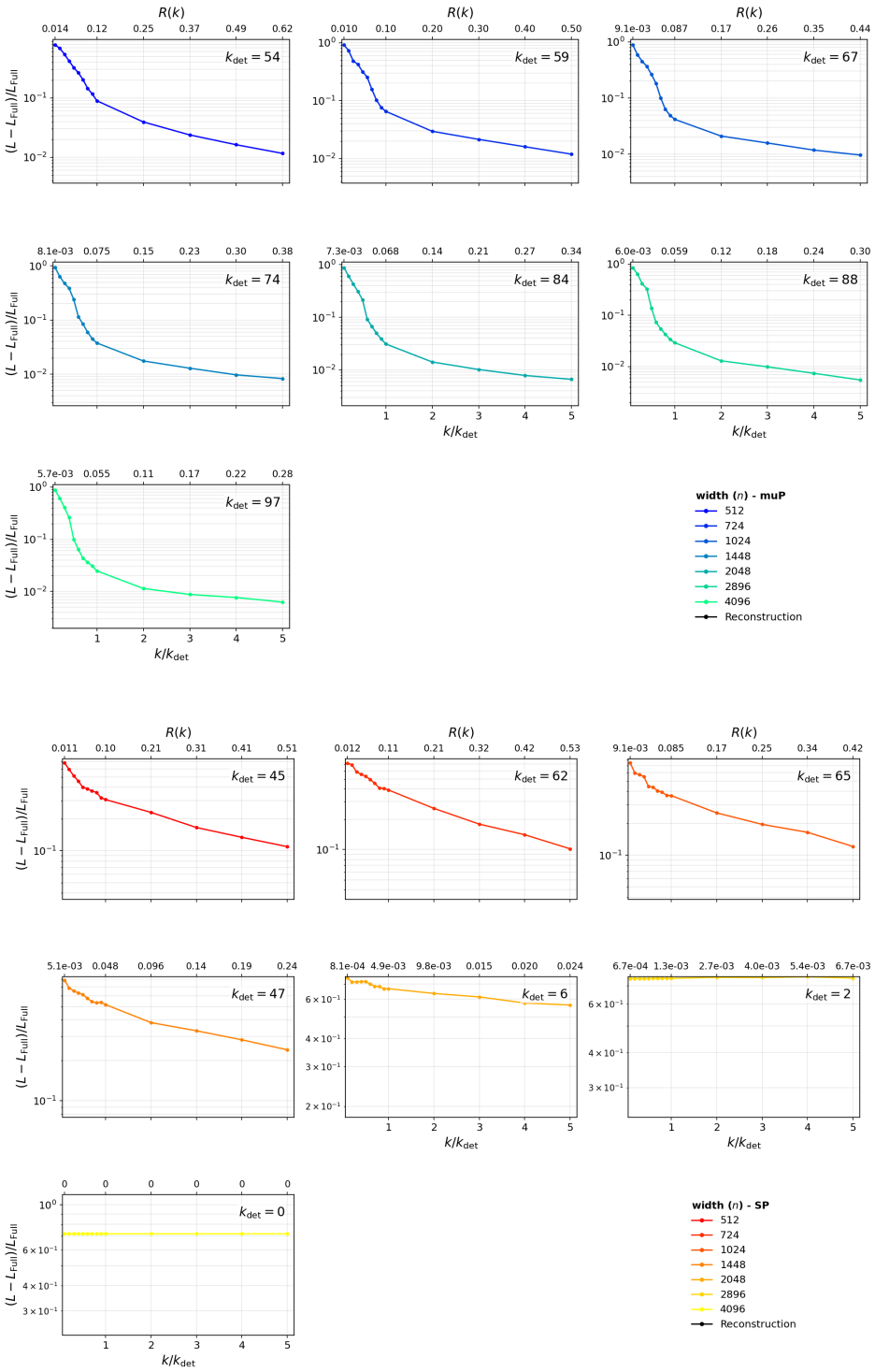


Figure 5.11: *Relative prediction performance of the reconstructed model compared to the original model containing all parameters, for one-hidden-layer MLPs trained on CIFAR10. The legend indicates the width of the hidden layer (n) of each network and the utilized parametrization. The bottom x-axis indicates the relative number of modes used for the approximation, whereas the top x-axis indicates the compression ratio as defined in Equation 5.46. The plots show that the top $k = k_{det}$ eigenmodes under μP (top) contain almost the entire structure the model built during training, with a relative performance error below 10% across all widths. In comparison, under SP the suppression of feature learning causes a large relative performance error when the full-size model is approximated from the leading $k = k_{det}$ collective modes.*

To understand the connection between the collective modes that correspond to detached eigenvalues, we need to consider again the empirical neuron distribution and its deterministic infinite-width limit. In context of the empirical neuron distribution the width dependence of the detached-mode count is best understood as a detection effect rather than a difference in the learned structure. Under μP the limiting neuron distribution $\mathring{\rho}_t$ evolves independently of width, so that the deterministic spectrum of $\mathring{C} := \mathbb{E}_{\mathring{\rho}_t}[W^\top W]$ consists of a fixed, width-independent sequence of collective modes with decaying strength. At finite width the measured superposition matrix represents a noisy approximation of this deterministic infinite-width spectrum. Therefore, the noise floor indicated by the decorrelated null spectrum decreases with increasing width. A collective mode is resolved as detached whenever its eigenvalue exceeds the null spectrum at the same rank. Increasing the width therefore does not create additional modes but lowers the level against which the measured spectrum is compared, so that progressively weaker modes of the same fixed signal spectrum are resolved. This is consistent with the conservation of the trace and the rank, which constrain the original and the null spectrum only relative to each other at fixed width. Both spectra share the same rank given by $\min(n,d)$ and the same trace, so that building collective modes redistributes eigenvalue mass from the bulk into the detached region. Increasing the network width therefore moves the crossing point between the original and the null spectrum further down the deterministic sequence of eigenvalues determined by the limiting eigenspectrum of $\mathring{C}$, while the total sum of eigenvalues remains constant.
In the infinite-width limit the noise contribution vanishes, the null spectrum collapses towards zero, and every collective mode of $\mathring{C}$ is exposed as a non-zero eigenvalue. This explains why the number of detached eigenmodes gradually increases and saturates as the network's width is increased, where a larger number of resolved eigenmodes is also reflected in a better generalization performance, as can be obtained from Figure 5.12.
Under SP the situation is qualitatively different, since the limiting neuron distribution remains asymptotically close to the distribution at initialization and the spectrum of $\mathring{C}$ carries no collective structure. The eigenvalues detected above the null spectrum at finite width therefore represent finite-width corrections that emerged from a remaining evolution of the empirical neuron distribution during training. As the width increases, the eigenspectrum approaches the infinite-width limit where no feature learning happens and consequently no information is stored in cross-column correlations, so that the decorrelated null spectrum is approached. In the infinite-width limit the SP spectrum therefore converges to the decorrelated null spectrum throughout the entire training process, in contrast to the nontrivial spectral evolution obtained under μP. Hence, the generalization performance gradually decreases when the network is approximated from the leading principal components, since the collective modes are an artifact of finite-width corrections.

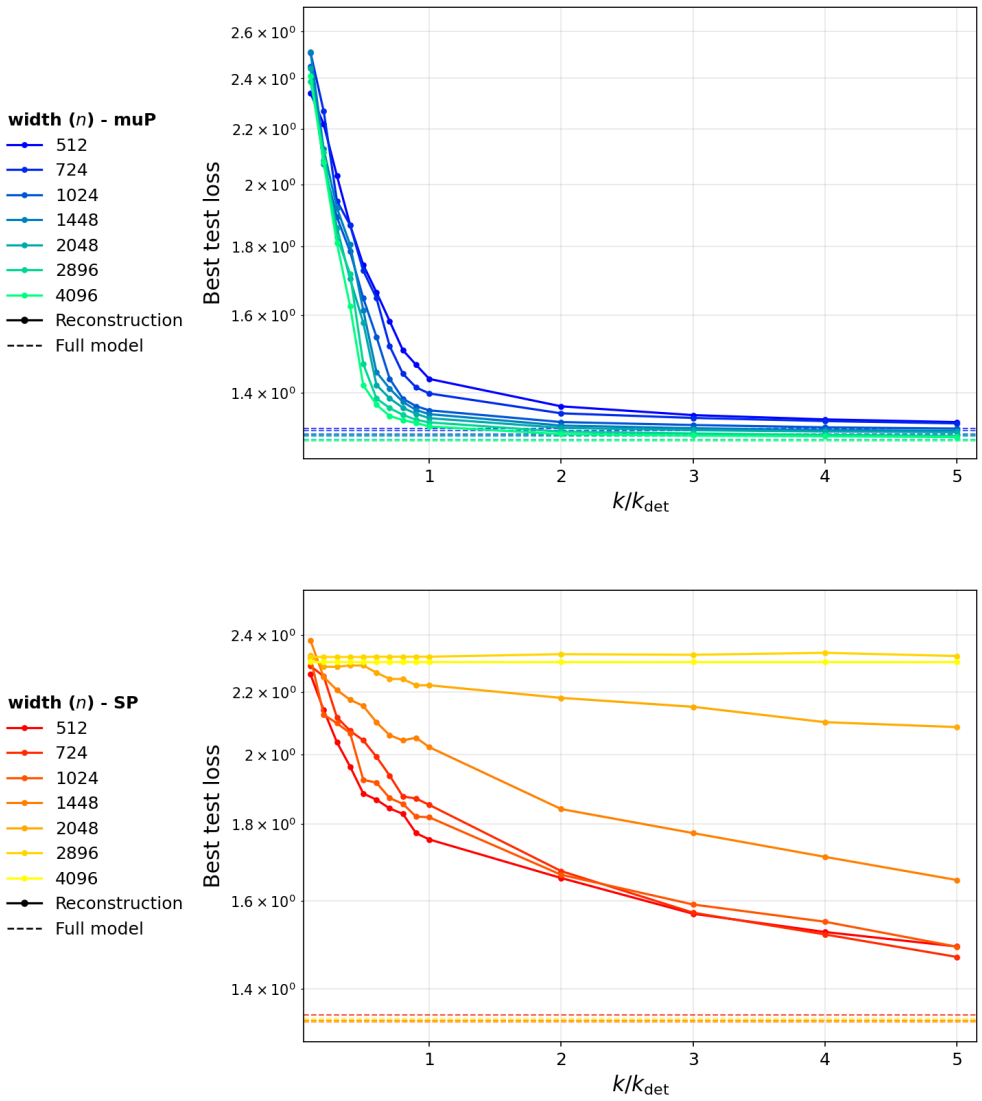


Figure 5.12: *Best test loss of the reconstructed model for one-hidden-layer MLPs trained on CIFAR10, shown as a function of the number of principal components used for the reconstruction. The legend indicates the width of the hidden layer (n) of each network and the utilized parametrization. The performance of the full-size models is indicated by a dashed line. Under μP (top) increasing the width of the network reduces the noise floor, which allows to detect more detached eigenvalues that correspond to collective modes identified during training. Models of larger width therefore reveal more of the principal components of the infinite-width limit, which results in a better generalization performance of the reconstructed model based on the $k = k_{det}$ leading eigenmodes. In comparison, under SP (bottom) the deterioration of feature learning with increasing network width results in a decaying number of detached eigenvalues, so that the reconstructed models show a significant decline in their generalization performance. The number of collective modes used for the reconstruction is indicated in units of detached eigenvalues.*

6 Summary and Conclusion

6.1 Summary and Conclusion

In this thesis the TPF [9, 10] and the perspective of neuron-wise observables [21, 22] have been introduced to understand the width dependence of neural network training dynamics, which is determined by the utilized abc-parametrization. Under μP the network's output becomes deterministic in the infinite-width limit, so that the empirical average of any neuron-wise observable with respect to the empirical neuron distribution converges to a deterministic limit with finite-width corrections of $O(n^{-1/2})$ [9]. Under SP the output converges to a GP instead, and the finite-width corrections to the network's output scale linearly with the network's width as $O(n)$. Furthermore, the empirical neuron distribution becomes static in the infinite-width limit, so that finite-width corrections become the central object that encode feature learning under SP [28]. This structural difference was illustrated for the layer-wise update norm and the test loss, which verified the collapse of empirical averages of neuron-wise observables under μP as well as the explicit width dependence present throughout all phases of training under SP. Under μP the finite-width corrections accumulate during training and induce correlations between neurons, which shift the joint distribution of neurons away from the deterministic infinite-width limit and thereby explain the deviations observed in late training stages.

Building on the concept of neuron-wise observables, a geometric picture of feature learning based on the superposition of hidden representations has been introduced. The superposition matrix captures the entire overlap structure between hidden representations and thereby reveals which input components the network learned to combine into meaningful features during training. Since each of its entries is an empirical average of a neuron-wise observable, the evolution of the superposition matrix is governed by the evolution of the empirical neuron distribution, which connects feature learning directly to the overlaps between hidden representations. At initialization these overlaps are random and the entire spectrum is described by random matrix theory, whereas under μP clear structure emerges during training. Transforming a single column of the superposition matrix back into the original image geometry shows that strong overlaps are established between pixels in the local vicinity of the reference pixel, so that the network learned to combine local sets of pixels into features that identify local structures in the image. Under SP the suppression of feature learning leaves the superposition matrix static and no such structure emerges. The eigenspectrum of the superposition matrix further identifies the collective modes built during training as detached

eigenvalues that cannot be explained by the decorrelated null spectrum. Under μP increasing the width can therefore be understood as a reduction of the noise floor that gradually uncovers more of the deterministic limiting spectrum. This can be exploited to approximate the hidden representations by the leading principal components of the superposition matrix, which substantially reduces the number of parameters required for a prediction while essentially preserving the performance of the full-size model. Since feature learning under SP is entirely contained in finite-width corrections, the number of detached eigenvalues decays with increasing width, the same low-rank approximation deteriorates strongly, and the dimensionality reduction ultimately cannot preserve the performance of the full-size model.
As μP preserves feature learning in the infinite-width limit, increasing the width enhances the feature-learning capabilities of the network. Each finite-width network follows the infinite-width limit up to the point in training at which the finite number of neurons causes deviations. Since feature learning remains of $O(1)$ per neuron in the limit, the limiting model possesses an unbounded capacity of features that are optimized during training, and since the corrections scale as $O(n^{-1/2})$, a wider network approximates this limit more closely and therefore generalizes better. In the spectral picture this improvement corresponds to the reduced noise floor that uncovers more of the limiting spectrum. Under SP no feature learning is possible in the infinite-width limit, so that the advantage of a larger number of available features is counteracted by the suppressed engineering of each individual feature. A formal proof of this hypothesis is currently out of reach, but it can be concluded that increasing the width under SP is not automatically accompanied by a better generalization performance. This is already visible for the networks trained on CIFAR10 under SP in this thesis, where the largest network generalizes worse than smaller ones, which indicates the turning point at which the increased number of trainable parameters is outweighed by the deteriorating engineering of features during training. In addition, the width dependence of the training process requires an unpredictable readjustment of the learning rate at every width under SP, which causes significant additional computational cost. Under μP the deterministic training dynamics in the infinite-width limit ensure a width-independent optimal learning rate, which explains why it can be tuned on a small network and transferred directly to much larger models.

These results can be categorized in the literature on the infinite-width behaviour of neural networks and feature learning dynamcis as follows. The abc-parametrizations and the property of μP to admit feature learning in the infinite-width limit as well as the transfer of the optimal learning rate across width have been established within the TPF [9, 10]. Additionally, the convergence of the network output to a GP with dynamics governed by the fixed NTK under SP is well established in the kernel-regime literature [27–29]. The present analysis does not modify these limiting statements but uses them to form a new perspective on universal behaviour. Describing the network by the empirical distribution of its neurons identifies self-averaging as the mechanism behind

the width-universal training dynamics under μP, so that the collapse of macroscopic observables and the transferability of the optimal learning rate appear as consequences of one property rather than as separate phenomena. This complements the mean-field description of one-hidden-layer networks [21, 22], which establishes the existence and the evolution of the limiting neuron distribution, by using it as a measurement tool that quantifies how closely a network of finite width approximates this limit and how the corresponding deviations accumulate during training.
The introduced spectral perspective on feature learning based on the superposition matrix further relates this work to the analysis of learned representations. While interpretability research typically examines the representations of a trained model directly, the superposition matrix reduces them to a self-averaging observable whose deterministic limit is controlled by the limiting neuron distribution providing a direct framework to monitor feature evolution [8]. Feature learning thereby becomes accessible as a spectral quantity, and the detached eigenvalues provide a criterion for the collective modes that a network has built during training. This directly connects the parametrization-dependent feature learning studied in the TPF to the structure of hidden representations and identifies the number of resolved collective modes as the quantity through which an increased width translates into an improved generalization performance. In this sense the results presented here suggest that the empirical scaling behaviour of neural networks and the microscopic organization of their learned features are two descriptions of the same underlying mechanism. This connection is exposed particularly clearly by the compact training setting used in this thesis, in which the width of the network is the only variable that affects the feature-learning dynamics. The observation is relevant beyond this setting, since neural scaling laws are found across far more complex architectures, for example in transformers trained on natural language [15, 31], where the same characteristic behaviour emerges from a considerably more complicated experimental setup. The minimal framework studied here may therefore provide evidence about the central mechanism behind scaling that carries over to substantially more complex models and training procedures.

This thesis has presented feature learning as the central mechanism governing the training dynamics of a neural network, and the compact experimental setup made the influence of the width particularly transparent. The comparison of SP and μP exposed feature learning as the mechanism through which a robust scaling of the generalization performance with model width arises. This constitutes an important step towards a deeper understanding of the origin of neural scaling laws, which underlie the predictable improvement of performance with increasing model size and thereby drive the broadly observed trend towards training larger and larger models across many domains of application. Beyond this, it was shown that feature learning can be exploited to approximate a trained network using only a subset of the learned features while preserving the performance of the full-size model. Such a dimensionality reduction

points towards a more efficient use of neural networks, which is of significant practical relevance given the resource requirements of modern machine learning models.

6.2 Limitations and Outlook

In this thesis feature learning was analyzed based on a one-hidden-layer MLP, which represents a minimal architecture that exhibits feature learning. Feature learning is, however, the integral property of neural networks, and the ability to form meaningful features from data autonomously is in principle shared by all different types of architectures. The most direct extension of the presented results is therefore their generalization to MLPs with more than one hidden layer, which requires extending the formalism of the empirical neuron distribution to deeper architectures. A generalization to other network architectures is an equally promising direction, since extensions of the μP framework already exist for ResNets and Transformers [10]. In this case the formalism has to be adapted such that it exploits the symmetry of the respective architecture in order to formulate the corresponding neuron-wise observables.
It was further shown that feature learning can be exploited to reconstruct the hidden representations of the network from the leading principal components while preserving the predictive performance of the full-size model. Under μP the number of detached eigenvalues determines how many collective modes of the infinite-width limit have been uncovered, and the generalization performance improves systematically with the network's width. This suggests that the gradual revelation of the infinite-width limit, expressed by the increasing feature-learning capabilities of wider networks, is a root cause of neural scaling laws in this compact experimental setup. Since the origin of neural scaling laws has not been identified so far, the setup used in this thesis provides a promising framework for studying how feature-learning dynamics give rise to predictable performance scaling with model width. In this context also the turning point observed under SP, beyond which the generalization performance deteriorates as the width is increased further, should be investigated in detail. Since the proposed explanation that the advantage of a larger number of features is outweighed by the suppressed engineering of each individual feature needs further verification, as this should in principle be confirmable for any learning rate.
Finally, the analysis conducted here was restricted to the general capability of neural networks to perform feature learning and to the question of whether an increase of the width can be expected to improve generalization performance. Whether the additional computational cost that necessarily accompanies a larger width is justified by the resulting gain in generalization performance is a separate question, where further studies in this direction constitutes a direction of future work that is highly relevant for practical applications.

A Appendices

A.1 Feature Learning in a One-Hidden-Layer MLP

To illustrate the difference between the standard parametrization (SP) and the maximal-update parametrization μP, an MLP with one hidden layer of width n is considered in the following. Using the notation introduced in the previous chapter, this corresponds to

$$L = 1, \qquad n_1 = n. \tag{A.1}$$

For simplicity, we consider a scalar regression output,

$$p = n_2 = 1, \tag{A.2}$$

and choose a linear output activation,

$$\sigma^1 = \mathrm{id}\,. \tag{A.3}$$

Biases are omitted in the following derivation because they do not affect the central width-scaling arguments. The input layer is given by

$$\boldsymbol{z}^0 := \boldsymbol{x} \in \mathbb{R}^d. \tag{A.4}$$

In contrast to the previous chapter, we now explicitly include a scalar time index t, which labels the SGD update step. Thus, W_t^0 and W_t^1 denote the weight matrices after t optimization steps. For a given input $\boldsymbol{x}$, the hidden preactivation vector at step t is

$$\boldsymbol{h}_t^0(\boldsymbol{x}) := W_t^0 \boldsymbol{x} \in \mathbb{R}^n. \tag{A.5}$$

The corresponding hidden activation vector is

$$\boldsymbol{z}_t^1(\boldsymbol{x}) = \sigma^0\left(\boldsymbol{h}_t^0(\boldsymbol{x})\right) \tag{A.6}$$

$$= \sigma^0\left(W_t^0 \boldsymbol{x}\right) \in \mathbb{R}^n, \tag{A.7}$$

where the activation function σ^0 is applied component-wise. The scalar network output at step t is

$$f_t(\boldsymbol{x}) := z_t^2(\boldsymbol{x}) \tag{A.8}$$
$$= W_t^1 \boldsymbol{z}_t^1(\boldsymbol{x}) \tag{A.9}$$
$$= W_t^1 \sigma^0\left(W_t^0 \boldsymbol{x}\right), \tag{A.10}$$

with

$$W_t^0 \in \mathbb{R}^{n\times d}, \qquad W_t^1 \in \mathbb{R}^{1\times n}. \tag{A.11}$$

At SGD step t, the network is updated using a training pair $(\boldsymbol{x}_t,y_t)$. For this sample, we define the derivative of the loss with respect to the network output as

$$\zeta_t := \frac{\partial \mathcal{L}\left(f_t(\boldsymbol{x}_t), y_t\right)}{\partial f_t(\boldsymbol{x}_t)}. \tag{A.12}$$

For typical loss functions and targets of order one, this derivative satisfies

$$\zeta_t = \Theta(1), \tag{A.13}$$

meaning that its magnitude does not explicitly scale with the hidden-layer width n. Similarly, we assume that the hidden activation function satisfies

$$\sigma^0(\boldsymbol{h}) = \Theta(1), \qquad (\sigma^0)'(\boldsymbol{h}) = \Theta(1), \tag{A.14}$$

for typical preactivations $h = \Theta(1)$. This includes smooth activation functions such as the hyperbolic tangent and applies to active ReLU neurons almost everywhere.
The derivatives of the network output with respect to the weight matrices are

$$\nabla_{W_t^1} f_t(\boldsymbol{x}_t) = \boldsymbol{z}_t^1(\boldsymbol{x})^\top, \tag{A.15}$$

and

$$\nabla_{W_t^0} f_t(\boldsymbol{x}_t) = \left[(W_t^1)^\top \odot (\sigma^0)'\left(\boldsymbol{h}_t^0(\boldsymbol{x}_t)\right)\right] \boldsymbol{x}_t^\top. \tag{A.16}$$

Here, $\odot$ denotes component-wise multiplication. The expression in square brackets is an n-dimensional column vector, and multiplication by $\boldsymbol{x}_t^\top$ forms the corresponding outer product. These gradients provide the basis for comparing how SP and μP scale the weight updates with the hidden-layer width n.

A.1.1 Standard Parametrization (SP)

The LeCun initialization is one of the most commonly used initializations where a network's weights and biases are initialized according to $W^l \sim \mathcal{N}(0, \frac{1}{n_l})$ and $\boldsymbol{b} = 0$ with n_l describing the number of neurons in layer l. This initialization ensures that for an input sample $\boldsymbol{x}$ with mean $\mathbb{E}[\boldsymbol{x}] = \mathbf{0}$ and variance $\mathrm{Var}[\boldsymbol{x}]$ that is processed by

the network, all preactivation vectors $\boldsymbol{h}^l$ preserve these statistical properties, meaning $\{\mathbb{E}[\boldsymbol{h}^l] = 0\}_{l=1\dots L+1}$ and $\{\sqrt{\text{Var}[\boldsymbol{h}^l]} = \text{Var}[\boldsymbol{x}]\}_{l=1\dots L+1}$. This idea is illustrated below for the first preactivation vector of a MLP using normalized inputs $x_i = \frac{x_i - \mu_i}{\sigma^i}$. Normalization of input features is commonly used in deep learning to ensure comparability of input feature variance and to center inputs around zero as this is the most expressive part of the non-linear activation function [23].

$$h_j = \sum_{i=1}^{d} W_{ji} x_i, \qquad W_{ji} \sim \mathcal{N}\left(0, \frac{1}{d}\right). \tag{A.17}$$

$$\begin{aligned}
\text{Var}(h_j) &= \text{Var}\left(\sum_{i=1}^{d} W_{ji} x_i\right) && \text{(A.18)}\\
&= \sum_{i=1}^{d} \text{Var}(W_{ji} x_i) && \text{(A.19)}\\
&= \sum_{i=1}^{d} \text{Var}(W_{ji}) \,\text{Var}(x_i) && \text{(A.20)}\\
&= d \cdot \frac{1}{d} \,\text{Var}(x_i) && \text{(A.21)}\\
&= \text{Var}(x_i). && \text{(A.22)}
\end{aligned}$$

However, the first short-comming of this idea is that non-linear activation functions generally do not preserve the variance of the preactivation vector.

$$z_j = \sigma(h_j) \tag{A.23}$$

$$\text{Var}(z_j) = \text{Var}(\sigma(h_j)) \neq \text{Var}(h_j) \tag{A.24}$$

Furthermore, it is important to notice that this statement about the statistical properties of the preactivations $\boldsymbol{h}^l$ is only true at initialization, but not for the entire training process, as generally the weights and biases are not stochastically independent and do not follow the distributions used at initialization anymore after multiple training steps have been conducted.

During the training process weights are updated using a gradient descent based optimization algorithm as introduced in 2.3 where the same learning rate is used for all weights across all layers of the neural network. A training setup that initializes weights according to the LeCun initialization and employs one global learning rate for all weights that is used during optimization is referred to as standard parametrization (SP) and represents the commonly used setup used in deep learning.

Empirically, the learning rate is chosen proportional to the network's width to preserve stable training dynamics when the width is increased. To illustrate the problem of feature learning in the infinite-width limit under SP, Eq. A.25 introduces a width-dependent learning-rate scaling of the form ($\eta \propto n^{-c}$). In formal definitions of SP, however, one sets (c=0), since no universal task-agnostic scaling exponent can be applied in practice.

$$\eta_n \sim n^{-c}. \tag{A.25}$$

In formal definitions of SP, however, the scaling exponent is set to $c = 0$, since no universal task-agnostic scaling exponent can be applied in practice.

As introduced under SP, the weights are initialized according to

$$W^0_{0,jk} \sim \mathcal{N}\left(0, \frac{1}{d}\right), \tag{A.26}$$

$$W^1_{0,j} \sim \mathcal{N}\left(0, \frac{1}{n}\right). \tag{A.27}$$

Because the input dimension d is fixed as $n \to \infty$, the coordinates of the hidden preactivation and activation vectors satisfy
The scalar index $t \in \mathbb{N}_0$ denotes the SGD update step, where $t = 0$ corresponds to initialization. Therefore, W^0_t, W^1_t, $\boldsymbol{h}^0_t$, $\boldsymbol{z}^1_t$, and f_{θ_t} denote the corresponding quantities after t update steps.

$$h^0_{t,j}(\boldsymbol{x}) = \Theta(1), \tag{A.28}$$

$$z^1_{t,j}(\boldsymbol{x}) = \Theta(1). \tag{A.29}$$

Consequently,

$$\boldsymbol{z}^1_t(\boldsymbol{x})^\top \boldsymbol{z}^1_t(\boldsymbol{x}) = \Theta(n), \tag{A.30}$$

$$W^1_{t,j} = \Theta\left(n^{-1/2}\right), \tag{A.31}$$

$$W^1_t (W^1_t)^\top = \Theta(1), \tag{A.32}$$

$$f_{\theta_t}(\boldsymbol{x}) = \Theta(1). \tag{A.33}$$

Please consider Appendix A.1.3 for a detailed derivation on why the network's prediction remains of order one in a probabilistic sense.
Using the specified learning rate both weight matrices are updated using the same effective learning rate

$$\eta_n = n^{-c}. \tag{A.34}$$

At an arbitrary SGD step t, the weights are updated according to

$$W_{t+1}^1 = W_t^1 - n^{-c}\zeta_t \, \boldsymbol{z}_t^1(\boldsymbol{x})^\top, \tag{A.35}$$
$$W_{t+1}^0 = W_t^0 - n^{-c}\zeta_t \left[(W_t^1)^\top \odot (\sigma^0)' \left(\boldsymbol{h}_t^0(\boldsymbol{x})\right)\right] \boldsymbol{x}^\top. \tag{A.36}$$

To determine whether the hidden representation changes during training, we evaluate the preactivation vector for an arbitrary input $\boldsymbol{x}'$. Its update is

$$\boldsymbol{h}_{t+1}^0(\boldsymbol{x}') - \boldsymbol{h}_t^0(\boldsymbol{x}') = \left(W_{t+1}^0 - W_t^0\right) \boldsymbol{x}' \tag{A.37}$$
$$= -n^{-c}\zeta_t \left[(W_t^1)^\top \odot (\sigma^0)' \left(\boldsymbol{h}_t^0(\boldsymbol{x})\right)\right] \boldsymbol{x}^\top \boldsymbol{x}'. \tag{A.38}$$

Since each coordinate of $(W_t^1)^\top$ is of order $\Theta(n^{-1/2})$, each hidden preactivation coordinate changes by

$$h_{t+1,j}^0(\boldsymbol{x}') - h_{t,j}^0(\boldsymbol{x}') = \Theta\left(n^{-c-1/2}\right). \tag{A.39}$$

Writing $\Delta W^l := W_{t+1}^l - W_t^l$ and $\Delta \boldsymbol{z}^1(\boldsymbol{x}') := \boldsymbol{z}_{t+1}^1(\boldsymbol{x}') - \boldsymbol{z}_t^1(\boldsymbol{x}')$, the full change of the network output for an arbitrary input $\boldsymbol{x}'$ decomposes exactly as

$$f_{t+1}(\boldsymbol{x}') - f_t(\boldsymbol{x}') = \underbrace{\Delta W^1 \, \boldsymbol{z}_t^1(\boldsymbol{x}')}_{\text{output-weight contribution}} + \underbrace{W_t^1 \, \Delta \boldsymbol{z}^1(\boldsymbol{x}')}_{\text{feature contribution}} + \underbrace{\Delta W^1 \, \Delta \boldsymbol{z}^1(\boldsymbol{x}')}_{\text{cross term}}. \tag{A.40}$$

The two mechanisms that distinguish SP from μP are already visible in the leading contributions of this decomposition. The output-weight contribution is driven by the output-layer update, which, evaluated at the training input $\boldsymbol{x}_t$ and on the current features, reads

$$\Delta W^1 \, \boldsymbol{z}_t^1(\boldsymbol{x}_t) = -\eta_{W^1}\zeta_t \, \boldsymbol{z}_t^1(\boldsymbol{x}_t)^\top \boldsymbol{z}_t^1(\boldsymbol{x}_t), \tag{A.41}$$

and controls whether the output update remains stable as $n \to \infty$.
The three contributions in Equation A.40 can now be compared directly under SP with $c = 1$. Each output weight and its update are of order $W_{t,\alpha}^1 = \Theta(n^{-1/2})$ and $\Delta W_\alpha^1 = -\eta_n \zeta_t z_{t,\alpha}^1(\boldsymbol{x}_t) = \Theta(n^{-1})$, the features are of order $z_{t,\alpha}^1 = \Theta(1)$, and their updates decay as $\Delta z_\alpha^1 = \Theta(n^{-3/2})$. Since each contribution is a sum over the n hidden coordinates, they scale as

$$\Delta W^1 \, \boldsymbol{z}_t^1(\boldsymbol{x}') = n \cdot \Theta(n^{-1}) \cdot \Theta(1) = \Theta(1), \tag{A.42}$$
$$W_t^1 \, \Delta \boldsymbol{z}^1(\boldsymbol{x}') = n \cdot \Theta(n^{-1/2}) \cdot \Theta(n^{-3/2}) = \Theta(n^{-1}), \tag{A.43}$$
$$\Delta W^1 \, \Delta \boldsymbol{z}^1(\boldsymbol{x}') = n \cdot \Theta(n^{-1}) \cdot \Theta(n^{-3/2}) = \Theta(n^{-3/2}). \tag{A.44}$$

The output-weight contribution is therefore the only term of order $\Theta(1)$, while the feature and cross terms vanish as $n \to \infty$. This confirms that the isolated output-

weight update in Equation A.41 captures the full output change at leading order under SP.
For the output update to remain of order $\mathcal{O}(1)$ as $n \to \infty$, the learning-rate exponent must satisfy

$$c = 1. \tag{A.45}$$

The largest stable and nontrivial effective learning rate is therefore

$$\eta_n = \Theta\left(\frac{1}{n}\right). \tag{A.46}$$

Setting $c = 1$ in Equation A.39 gives

$$h^0_{t+1,j}(\boldsymbol{x}') - h^0_{t,j}(\boldsymbol{x}') = \Theta\left(n^{-3/2}\right) \longrightarrow 0. \tag{A.47}$$

For a differentiable activation function, a first-order expansion gives

$$\begin{aligned} z^1_{t+1,j}(\boldsymbol{x}') - z^1_{t,j}(\boldsymbol{x}') &= \sigma^0\left(h^0_{t+1,j}(\boldsymbol{x}')\right) - \sigma^0\left(h^0_{t,j}(\boldsymbol{x}')\right) && \text{(A.48)} \\ &= (\sigma^0)'\left(h^0_{t,j}(\boldsymbol{x}')\right)\left[h^0_{t+1,j}(\boldsymbol{x}') - h^0_{t,j}(\boldsymbol{x}')\right] + o\left(n^{-3/2}\right) && \text{(A.49)} \\ &= \Theta\left(n^{-3/2}\right) \longrightarrow 0 && \text{(A.50)} \end{aligned}$$

where $o\left(n^{-3/2}\right)$ denotes terms that decay faster than $n^{-3/2}$.
The same conclusion follows directly for Lipschitz-continuous activation functions such as ReLU. The hidden representation therefore becomes frozen in the infinite-width limit [9].
At leading order, the output prediction for an arbitrary input $\boldsymbol{x}'$ evolves according to

$$\begin{aligned} f_{\theta_{t+1}}(\boldsymbol{x}') - f_{\theta_t}(\boldsymbol{x}') &= -\zeta_t \, \frac{\boldsymbol{z}^1_t(\boldsymbol{x})^\top \boldsymbol{z}^1_t(\boldsymbol{x}')}{n} + o(1) && \text{(A.51)} \\ &\longrightarrow -\zeta_t \, K_{\mathrm{SP}}\left(\boldsymbol{x}', \boldsymbol{x}\right), && \text{(A.52)} \end{aligned}$$

where

$$K_{\mathrm{SP}}\left(\boldsymbol{x}', \boldsymbol{x}\right) := \lim_{n\to\infty} \frac{\boldsymbol{z}^1_t(\boldsymbol{x})^\top \boldsymbol{z}^1_t(\boldsymbol{x}')}{n}. \tag{A.53}$$

Because the hidden features remain asymptotically unchanged, the kernel in Equation A.53 is fixed throughout training under SP. The network therefore approaches the NTK regime, which is also often referred to as the lazy learning regime, as the evolution of the networks output under gradient descent is determined by a constant feature Kernel in the infinite-width limit ($n \to \infty$). This allows to formulate gradient descent as kernel gradient descent in function space, where the entire training dynamics can be described based as an exponential decay on the frozen NTK [26–30].

A.1.2 Maximal-Update Parametrization (μP)

Under μP, the initialization and the layer-wise learning rates are chosen such that the hidden representation can change by an order-one amount without causing the network output to diverge. The hidden-layer weights are initialized according to

$$W^0_{0,jk} \sim \mathcal{N}\left(0, \frac{1}{d}\right), \tag{A.54}$$

while the output-layer weights are initialized according to

$$W^1_{0,j} \sim \mathcal{N}\left(0, \frac{1}{n^2}\right). \tag{A.55}$$

Consequently, the hidden preactivations and activations remain of order one:

$$h^0_{0,j}(\boldsymbol{x}) = \Theta(1), \tag{A.56}$$
$$z^1_{0,j}(\boldsymbol{x}) = \Theta(1). \tag{A.57}$$

The output-layer weights instead satisfy

$$W^1_{0,j} = \Theta\left(n^{-1}\right), \tag{A.58}$$
$$nW^1_{0,j} = \Theta(1). \tag{A.59}$$

At initialization, the network output satisfies

$$f_{\theta_0}(\boldsymbol{x}) = W^1_0 \boldsymbol{z}^1_0(\boldsymbol{x}) = \Theta\left(n^{-1/2}\right), \tag{A.60}$$

because it is a sum of n approximately independent terms of order $\Theta(n^{-1})$. Thus,

$$f_{\theta_0}(\boldsymbol{x}) \longrightarrow 0 \tag{A.61}$$

as $n \to \infty$. During training, correlations between the weights and the learned features can generate outputs of order $\Theta(1)$.
Instead of using a single learning rate for all layers, μP assigns different width-dependent learning rates to the hidden and output layers. Let

$$\eta = \Theta(1) \tag{A.62}$$

denote a width-independent base learning rate. The layer-wise learning rates are chosen as

$$\eta_{W^0} = n\eta, \tag{A.63}$$
$$\eta_{W^1} = \frac{\eta}{n}. \tag{A.64}$$

For a single training pair $(\boldsymbol{x},y)$ at SGD step t, the gradients are

$$\nabla_{W_t^1} f_{\theta_t}(\boldsymbol{x}) = \boldsymbol{z}_t^1(\boldsymbol{x})^\top, \tag{A.65}$$
$$\nabla_{W_t^0} f_{\theta_t}(\boldsymbol{x}) = \left[(W_t^1)^\top \odot (\sigma^0)' \left(\boldsymbol{h}_t^0(\boldsymbol{x})\right)\right] \boldsymbol{x}^\top. \tag{A.66}$$

An arbitrary SGD step therefore gives

$$W_{t+1}^1 = W_t^1 - \eta_{W^1}\zeta_t \boldsymbol{z}_t^1(\boldsymbol{x})^\top \tag{A.67}$$
$$= W_t^1 - \frac{\eta}{n}\zeta_t \boldsymbol{z}_t^1(\boldsymbol{x})^\top, \tag{A.68}$$
$$W_{t+1}^0 = W_t^0 - \eta_{W^0}\zeta_t \left[(W_t^1)^\top \odot (\sigma^0)' \left(\boldsymbol{h}_t^0(\boldsymbol{x})\right)\right] \boldsymbol{x}^\top \tag{A.69}$$
$$= W_t^0 - n\eta\zeta_t \left[(W_t^1)^\top \odot (\sigma^0)' \left(\boldsymbol{h}_t^0(\boldsymbol{x})\right)\right] \boldsymbol{x}^\top. \tag{A.70}$$

The output-layer update is suppressed by a factor of $1/n$, while the hidden-layer update is amplified by a factor of n.
To determine whether feature learning remains possible in the infinite-width limit, consider an arbitrary input $\boldsymbol{x}' \in \mathbb{R}^d$. The change in the hidden preactivation vector is

$$\boldsymbol{h}_{t+1}^0(\boldsymbol{x}') - \boldsymbol{h}_t^0(\boldsymbol{x}') = \left(W_{t+1}^0 - W_t^0\right) \boldsymbol{x}' \tag{A.71}$$
$$= -n\eta\zeta_t \left[(W_t^1)^\top \odot (\sigma^0)' \left(\boldsymbol{h}_t^0(\boldsymbol{x})\right)\right] \boldsymbol{x}^\top \boldsymbol{x}'. \tag{A.72}$$

Since

$$nW_{t,j}^1 = \Theta(1), \tag{A.73}$$

each hidden preactivation coordinate changes by

$$h_{t+1,j}^0(\boldsymbol{x}') - h_{t,j}^0(\boldsymbol{x}') = \Theta(1). \tag{A.74}$$

The corresponding hidden-feature update is

$$z_{t+1,j}^1(\boldsymbol{x}') - z_{t,j}^1(\boldsymbol{x}') = \sigma^0\left(h_{t+1,j}^0(\boldsymbol{x}')\right) - \sigma^0\left(h_{t,j}^0(\boldsymbol{x}')\right) \tag{A.75}$$
$$= \Theta(1) \tag{A.76}$$

Thus, the hidden representation can evolve by an order-one amount even when $n \to \infty$. In contrast to the SP case, the preactivation update does not vanish. A first-order Taylor expansion of the nonlinear activation function is therefore not generally justified for a finite SGD step. The exact updated output for an arbitrary input $\boldsymbol{x}'$ is

$$f_{\theta_{t+1}}(\boldsymbol{x}') = W_{t+1}^1 \boldsymbol{z}_{t+1}^1(\boldsymbol{x}') \tag{A.77}$$

$$= \left[W_t^1 - \frac{\eta}{n}\zeta_t \boldsymbol{z}_t^1(\boldsymbol{x})^\top\right] \sigma^0\left(\boldsymbol{h}_t^0(\boldsymbol{x}') - n\eta\zeta_t \left[(W_t^1)^\top \odot (\sigma^0)'\left(\boldsymbol{h}_t^0(\boldsymbol{x})\right)\right] \boldsymbol{x}^\top \boldsymbol{x}'\right). \tag{A.78}$$

Each coordinate of the updated output-layer weight vector remains of order $\Theta(n^{-1})$:

$$W_{t+1,j}^1 = \Theta\left(n^{-1}\right). \tag{A.79}$$

At the same time, each coordinate of the updated hidden activation vector remains of order $\Theta(1)$. Consequently, the updated output remains bounded:

$$f_{\theta_{t+1}}(\boldsymbol{x}') = \mathcal{O}(1). \tag{A.80}$$

The output therefore remains stable even though individual hidden features can change by an order-one amount [9, 10].

A.1.3 Why the Random Signed Sum Remains of Order One

Consider the network output

$$f_n(\boldsymbol{x}) = \sum_{\alpha=1}^{n} W_\alpha^1 z_\alpha^1(\boldsymbol{x}), \tag{A.81}$$

where the output-layer weights are initialized independently according to

$$W_\alpha^1 \sim \mathcal{N}\left(0, \frac{1}{n}\right). \tag{A.82}$$

We condition on the hidden-layer activation vector $\boldsymbol{z}^1(\boldsymbol{x})$. At initialization, the hidden-layer activations are independent of the output-layer weights. Define

$$X_\alpha := W_\alpha^1 z_\alpha^1(\boldsymbol{x}). \tag{A.83}$$

Conditioned on $\boldsymbol{z}^1(\boldsymbol{x})$, the variables X_α are independent centered random variables. Since each output-layer weight has zero mean,

$$\mathbb{E}\left[X_\alpha \mid \boldsymbol{z}^1(\boldsymbol{x})\right] = z_\alpha^1(\boldsymbol{x})\mathbb{E}\left[W_\alpha^1\right] \tag{A.84}$$

$$= 0. \tag{A.85}$$

Consequently,

$$\mathbb{E}\left[f_n(\boldsymbol{x}) \mid \boldsymbol{z}^1(\boldsymbol{x})\right] = 0. \tag{A.86}$$

The variance of an individual contribution is

$$\begin{aligned} \operatorname{Var}\left[X_\alpha \mid \boldsymbol{z}^1(\boldsymbol{x})\right] &= \left[z_\alpha^1(\boldsymbol{x})\right]^2 \operatorname{Var}\left[W_\alpha^1\right] && \text{(A.87)} \\ &= \frac{\left[z_\alpha^1(\boldsymbol{x})\right]^2}{n}. && \text{(A.88)} \end{aligned}$$

Because the X_α are conditionally independent, their variances add:

$$\begin{aligned} \operatorname{Var}\left[f_n(\boldsymbol{x}) \mid \boldsymbol{z}^1(\boldsymbol{x})\right] &= \sum_{\alpha=1}^{n} \operatorname{Var}\left[X_\alpha \mid \boldsymbol{z}^1(\boldsymbol{x})\right] && \text{(A.89)} \\ &= \frac{1}{n} \sum_{\alpha=1}^{n} \left[z_\alpha^1(\boldsymbol{x})\right]^2. && \text{(A.90)} \end{aligned}$$

If the hidden-layer activations are individually of order one and their empirical second moment remains bounded,

$$\frac{1}{n} \sum_{\alpha=1}^{n} \left[z_\alpha^1(\boldsymbol{x})\right]^2 = O_{\mathbb{P}}(1), \tag{A.91}$$

then Equation A.90 implies

$$\operatorname{Var}\left[f_n(\boldsymbol{x}) \mid \boldsymbol{z}^1(\boldsymbol{x})\right] = O_{\mathbb{P}}(1). \tag{A.92}$$

The conditional standard deviation therefore also remains of order one:

$$\operatorname{Std}\left[f_n(\boldsymbol{x}) \mid \boldsymbol{z}^1(\boldsymbol{x})\right] = O_{\mathbb{P}}(1). \tag{A.93}$$

Consequently,

$$f_n(\boldsymbol{x}) = O_{\mathbb{P}}(1). \tag{A.94}$$

The notation

$$f_n(\boldsymbol{x}) = O_{\mathbb{P}}(1) \tag{A.95}$$

means that the network output remains bounded in probability as the width increases. Because the output-layer weights are Gaussian, the conditional distribution can also be written exactly:

$$f_n(\boldsymbol{x}) \mid \boldsymbol{z}^1(\boldsymbol{x}) \sim \mathcal{N}\left(0, \frac{1}{n} \sum_{\alpha=1}^{n} \left[z_\alpha^1(\boldsymbol{x})\right]^2\right). \tag{A.96}$$

A.2 Generalization of Layer-Wise Update Norms

Arbitrary batch size B

For arbitrary batch size: $\mathcal{L}_B = \frac{1}{B}\sum_{i=1}^{B} L(x_i)$ and therefore:

$$\left\|\Delta W_t^l\right\|^2 = \sum_{j=1}^{n}\sum_{k=1}^{d}\left(\eta_{W^l}\frac{\partial \mathcal{L}_B(f_t)}{\partial (W_t^l)_{jk}}\right)^2 \tag{A.97}$$

$$= \sum_{j=1}^{n}\sum_{k=1}^{d}\left(\eta_{W^l}\frac{\partial}{\partial (W_t^l)_{jk}}\frac{1}{B}\sum_{i=tB}^{(t+1)\cdot B-1} L(f_t(x_i))\right)^2 \tag{A.98}$$

$$= \sum_{j=1}^{n}\sum_{k=1}^{d}\left(\frac{1}{B}\sum_{i=tB}^{(t+1)\cdot B-1}\underbrace{\eta_{W^l}\frac{\partial L(f_t(x_i))}{\partial (W_t^l)_{jk}}}_{=X^{(\Delta W_{i,t}^l)_{jk}}}\right)^2 \tag{A.99}$$

$$= \sum_{j=1}^{n}\sum_{k=1}^{d}\left(\frac{1}{B}\sum_{i=tB}^{(t+1)\cdot B-1} X^{(\Delta W_{i,t}^l)_{jk}}\right)^2 \tag{A.100}$$

$$\tag{A.101}$$

At this point index i was introduced that monitors the input sample x_i within the mini-batch of size B, while the index t still indicates the update step such that:

$$X^{(\Delta W_{i,t}^0)_{jk}} = -\eta\,\zeta_{i,t}\,X^{(d_{h_{i,t}})_j}(x_i)_k \tag{A.102}$$

$$\xrightarrow{n\to\infty} -\eta\,\mathring{\zeta}_{i,t}\,X^{(d_{h_{i,t}})_j}(x_i)_k \tag{A.103}$$

$$X^{n(\Delta W_{i,t}^1)_{jk}} = -\eta\,(\zeta_{i,t})_j\,X^{(z_{i,t})_k} \tag{A.104}$$

$$\xrightarrow{n\to\infty} -\eta\,(\mathring{\zeta}_{i,t})_j\,X^{(z_{i,t})_k} \tag{A.105}$$

Now the definitions from eq. A.103 and A.105 can be used to calculate the weight update norm analogously to 5.1, where the definitions of $\tilde{X}_{t,j}^l$ contain the sum over the mini-batch:

Case $l = 0$:

$$\left\|\Delta W_t^0\right\|^2(n) = n\langle \tilde{X}_t^0\rangle_n \tag{A.106}$$

with:

$$\tilde{X}^0_{t,j} = \sum_{k=1}^{d} \left(\frac{1}{B} \sum_{i=tB}^{(t+1)\cdot B-1} X^{(\Delta W^0_{i,t})_{jk}} \right)^2 \tag{A.107}$$

$$= \sum_{k=1}^{d} \left(\frac{1}{B} \sum_{i=tB}^{(t+1)\cdot B-1} \eta\, \zeta_{i,t}\, X^{(d_{h_{i,t}})_j}(x_i)_k \right)^2 \tag{A.108}$$

Case $l = 1$:

$$\left\| \Delta W^1_t \right\|^2 (n) = \frac{1}{n} \langle \tilde{X}^1_t \rangle_n \tag{A.109}$$

with:

$$\tilde{X}^1_{t,j} = \sum_{k=1}^{p} \left(\frac{1}{B} \sum_{i=tB}^{(t+1)\cdot B-1} X^{(\Delta W^1_{i,t})_{jk}} \right)^2 \tag{A.110}$$

$$= \sum_{k=1}^{p} \left(\frac{1}{B} \sum_{i=tB}^{(t+1)\cdot B-1} \eta\, (\zeta_{i,t})_j\, X^{(z_{i,t})_k} \right)^2 \tag{A.111}$$

Accumulated SGD updates

It is reasonable to monitor weight updates only every τ training steps to reduce the amount of data monitored during the training process and to average the weight updates among multiple consecutive SGD update steps. This allows to reduce fluctuations in these quantities, as the cumulative weight updates after τ consecutive SGD steps yields an estimation of the effective step size.

Generalizing equation 5.1, the average weight updates accumulated over τ consecutive SGD steps under mini-batch gradient descent is given as:

$$u_{t,\tau} = \theta_{(t+1)\cdot\tau-1} - \theta_{t\cdot\tau} \tag{A.112}$$

$$= \sum_{s=t\cdot\tau}^{(t+1)\cdot\tau-1} \Delta\theta_s \tag{A.113}$$

$$= - \sum_{s=t\cdot\tau}^{(t+1)\cdot\tau-1} \eta \nabla_{\theta_s} \mathcal{L}_{B_s}(f_s) \tag{A.114}$$

Such that the layer-wise average weight updates are given by:

$$u^l_{t,\tau} := -\sum_{s=t\cdot\tau}^{(t+1)\cdot\tau-1} \eta_{W^l} \nabla_{W^l_s} \mathcal{L}_{B_s}(f_s) \tag{A.115}$$

$$\tag{A.116}$$

This allows to calculate the update norms of $u^l_{t,\tau}$

$$\left\|u^l_{t,\tau}\right\|_2^2 = \sum_{j=1}^{n}\sum_{k=1}^{d\wedge p}\left(\sum_{s=t\cdot\tau}^{(t+1)\cdot\tau-1} \eta_{W^l} \frac{\partial \mathcal{L}_{B_s}(f_s)}{\partial (W^l_s)_{jk}}\right)^2, \tag{A.117}$$

where the sum limit $d \wedge p$ refers to d for $l = 0$ and to p for $l = 1$. Using eq. A.101, this yields the final form:

$$\left\|u^l_{t,\tau}\right\|^2 = \sum_{j=1}^{n}\sum_{k=1}^{d\wedge p}\left(\sum_{s=t\cdot\tau}^{(t+1)\cdot\tau-1} \eta_{W^l} \frac{\partial \mathcal{L}_{B_s}(f_s)}{\partial (W^l_s)_{jk}}\right)^2 \tag{A.118}$$

$$= \sum_{j=1}^{n}\sum_{k=1}^{d\wedge p}\left(\sum_{s=t\cdot\tau}^{(t+1)\cdot\tau-1} \frac{1}{B} \sum_{i=sB}^{(s+1)\cdot B-1} X^{(\Delta W^l_{i,s})_{jk}}\right)^2 \tag{A.119}$$

Thus, the final form of the average weight update-norm accumulated over τ update steps for SGD with batch size B is given by:

Case $l = 0$:

$$\left\|u^l_{t,\tau}\right\|^2 (n) = n\langle \tilde{X}^0_t \rangle_n \tag{A.120}$$

with:

$$\tilde{X}^0_{t,j} = \sum_{k=1}^{d}\left(\sum_{s=t\cdot\tau}^{(t+1)\cdot\tau-1} \frac{1}{B} \sum_{i=sB}^{(s+1)\cdot B-1} X^{(\Delta W^0_{i,s})_{jk}}\right)^2 \tag{A.121}$$

$$\tag{A.122}$$

Case $l = 1$:

$$\left\|u^l_{t,\tau}\right\|^2 (n) = \frac{1}{n}\langle \tilde{X}^1_t \rangle_n \tag{A.123}$$

with:

$$\tilde{X}^1_{t,j} = \sum_{k=1}^{p} \left(\sum_{s=t\cdot\tau}^{(t+1)\cdot\tau-1} \frac{1}{B} \sum_{i=sB}^{(s+1)\cdot B-1} X^{(\Delta W^1_{i,s})_{jk}} \right)^2, \tag{A.124}$$

$$\tag{A.125}$$

where at this point the update norm can again be written in form of expectation values as in equations 5.8 and 5.13 that allows to compute the ratio of the layer-wise weight update norms as in equations 5.17 and 5.22.

A.3 Coordinate-Distribution Analysis of SP

This appendix repeats the derivation of Chapter 4 for the standard parametrization (SP) and derives the results used in Section 5.2.

A.3.1 Setup and Coordinate Distributions at Initialization

In SP both layers are initialized with fan-in scaling and share a single, layer-independent learning rate:

$$W^0_t = (W^0_t)_{ij} \in \mathbb{R}^{n\times d}, \qquad W^1_t = (W^1_t)_i \in \mathbb{R}^{1\times n}, \tag{A.126}$$

$$(W^0_0)_{ij} \sim \mathcal{N}\Big(0, \tfrac{1}{d}\Big), \qquad (W^1_0)_i \sim \mathcal{N}\Big(0, \tfrac{1}{n}\Big), \tag{A.127}$$

$$\eta_{W^0} = \eta_{W^1} = \eta. \tag{A.128}$$

To keep all coordinate random variables independent of n, the second-layer weights are rescaled by $\sqrt{n}$:

$$X^{W^0_0} \sim \mathcal{N}\Big(0, \tfrac{1}{d}\Big), \qquad X^{\sqrt{n}\,W^1_0} \sim \mathcal{N}(0,1). \tag{A.129}$$

Note the contrast to muP, where the natural rescaling is nW^1_0 and the readout carries an explicit $1/n$ normalization.

A.3.2 First Forward Pass ($t = 0$)

The pre- and post-activations are identical to the muP case:

$$\boldsymbol{h}_0 = W^0_0 \boldsymbol{x}_0, \qquad X^{\boldsymbol{h}_0} = X^{W^0_0}\|\boldsymbol{x}_0\| \sim \mathcal{N}\Big(0, \tfrac{\|\boldsymbol{x}_0\|^2}{d}\Big), \qquad \boldsymbol{z}_0 = \sigma(\boldsymbol{h}_0). \tag{A.130}$$

The output, however, is now a sum of n terms of size $O(n^{-1/2})$:

$$(\boldsymbol{f}_0)_j = \sum_{k=1}^{n} (W_0^1)_{jk} (\boldsymbol{z}_0)_k = \frac{1}{\sqrt{n}} \sum_{k=1}^{n} \left(\sqrt{n}\, W_0^1\right)_{jk} (\boldsymbol{z}_0)_k \tag{A.131}$$

$$\xrightarrow{n\to\infty} \mathcal{N}\Big(0,\, \mathbb{E}\Big[(X^{\boldsymbol{z}_0})^2\Big]\Big) \qquad \text{(CLT)}. \tag{A.132}$$

In contrast to muP, the first forward pass does not become deterministic: in the infinite-width limit $(\boldsymbol{f}_0)_j$ converges only in distribution to a Gaussian (the NNGP limit [41]). Consequently $\boldsymbol{\zeta}_0 = L'(\boldsymbol{f}_0)$ also remains a random variable and does not converge almost surely to a deterministic value.

A.3.3 First Backward Pass ($t = 0$)

Without the factor n used in muP, the gradients read

$$(\boldsymbol{d}_{\boldsymbol{z}_0})_j := \nabla_{\boldsymbol{z}_0} (\boldsymbol{f}_0)_j = (W_0^1)_j^{\mathsf{T}}, \qquad X^{\sqrt{n}\, \boldsymbol{d}_{\boldsymbol{z}_0}} = X^{\sqrt{n}\, W_0^1}, \tag{A.133}$$

$$\boldsymbol{d}_{\boldsymbol{h}_0} = \boldsymbol{d}_{\boldsymbol{z}_0} \odot \sigma'(\boldsymbol{h}_0), \tag{A.134}$$

i.e. the coordinates of $\boldsymbol{d}_{\boldsymbol{h}_0}$ are of size $O(n^{-1/2})$.

A.3.4 One SGD Step and the Scale Mismatch

With $B = 1$ and the shared learning rate η,

$$(W_1^1)_{jk} = (W_0^1)_{jk} - \eta\, \zeta_0\, (\boldsymbol{z}_0)_k, \tag{A.135}$$

$$(W_1^0)_{jk} = (W_0^0)_{jk} - \eta\, \zeta_0\, (\boldsymbol{d}_{\boldsymbol{h}_0})_j (\boldsymbol{x}_0)_k = (W_0^0)_{jk} - \frac{\eta}{\sqrt{n}}\, \zeta_0 \left(\sqrt{n}\, \boldsymbol{d}_{\boldsymbol{h}_0}\right)_j (\boldsymbol{x}_0)_k, \tag{A.136}$$

or in coordinate-distribution notation

$$X^{(W_1^0)_{jk}} = X^{(W_0^0)_{jk}} - \frac{\eta}{\sqrt{n}}\, \zeta_0\, X^{\sqrt{n}\,(\boldsymbol{d}_{\boldsymbol{h}_0})_j (\boldsymbol{x}_0)_k} \xrightarrow{n\to\infty} X^{(W_0^0)_{jk}}, \tag{A.137}$$

$$X^{\sqrt{n}\,(W_1^1)_{jk}} = X^{\sqrt{n}\,(W_0^1)_{jk}} - \eta\sqrt{n}\, \zeta_0\, X^{(\boldsymbol{z}_0)_k}. \tag{A.138}$$

Two imbalances appear that are absent in muP: the first-layer update vanishes relative to its initialization ($\Delta W^0 = O(\eta\, n^{-1/2})$ per coordinate), while the second-layer update grows as $\sqrt{n}$ relative to its initialization ($\Delta W^1 = O(\eta)$ vs. $W_0^1 = O(n^{-1/2})$). Since $\Delta W^0 \to 0$ in the limit, the hidden representation is unchanged to leading order, $\boldsymbol{z}_1 =$

$\sigma(W_1^0 \boldsymbol{x}_1) \to \sigma(W_0^0 \boldsymbol{x}_1)$, and the output after one step becomes

$$(\boldsymbol{f}_1)_j = \sum_{k=1}^{n} (W_1^1)_{jk} (\boldsymbol{z}_1)_k = \underbrace{\sum_{k=1}^{n} (W_0^1)_{jk} (\boldsymbol{z}_1)_k}_{= O(1) \text{ (CLT)}} - \eta\, \zeta_0 \sum_{k=1}^{n} (\boldsymbol{z}_0)_k (\boldsymbol{z}_1)_k \tag{A.139}$$

$$= O(1) \;-\; \eta\, \zeta_0\, n \cdot \frac{1}{n} \sum_{k=1}^{n} (\boldsymbol{z}_0)_k (\boldsymbol{z}_1)_k \;\xrightarrow{n\to\infty}\; O(1) - \eta\, \zeta_0\, n\, \mathbb{E}\Big[X^{\boldsymbol{z}_0} X^{\boldsymbol{z}_1}\Big]. \tag{A.140}$$

The functional update after a single SGD step is therefore of order $\eta\, n$ and, since $\mathbb{E}[X^{\boldsymbol{z}_0} X^{\boldsymbol{z}_1}]$ is in general nonzero, grows linearly in n at fixed η: the output diverges in the infinite-width limit, and the effective function-space learning rate of SP grows linearly with width.

A.3.5 Scaling Bookkeeping: Why f_t Is Not a Single Expectation

Expanding every weight as initialization plus accumulated updates, $W_t^l = W_0^l + \sum_{s<t} \Delta W_s^l$, the output $(\boldsymbol{f}_t)_j = \sum_k (W_t^1)_{jk} (\boldsymbol{z}_t)_k$ decomposes into monomials, each of which is a coordinate sum of the following type.

A.3.5.0.1 Coordinate scaling overview

Let $\boldsymbol{a}$,$\boldsymbol{b} \in \mathbb{R}^n$ be coordinate vectors whose coordinates are described by n-independent, $O(1)$ random variables $X^{\boldsymbol{a}}$, $X^{\boldsymbol{b}}$ (i.i.d. across the index k, jointly distributed within each k). Then

$$\sum_{k=1}^{n} a_k b_k \;=\; n\, \mathbb{E}\Big[X^{\boldsymbol{a}} X^{\boldsymbol{b}}\Big] \;+\; O(\sqrt{n}), \tag{A.141}$$

and if $\mathbb{E}[X^{\boldsymbol{a}} X^{\boldsymbol{b}}] = 0$,

$$\frac{1}{\sqrt{n}} \sum_{k=1}^{n} a_k b_k \;\xrightarrow{n\to\infty}\; \mathcal{N}\Big(0, \operatorname{Var}\Big(X^{\boldsymbol{a}} X^{\boldsymbol{b}}\Big)\Big) \qquad \text{(CLT)}. \tag{A.142}$$

Correlated summands therefore contribute coherently at scale n (LLN type, deterministic limit), while zero-mean summands contribute diffusively at scale $\sqrt{n}$ (CLT type, Gaussian limit). n terms of typical size $n^{-1/2}$ yield $\sqrt{n} \cdot n^{-1/2} = O(1)$, not $n \cdot n^{-1/2} = \sqrt{n}$, because random signs cancel as in a random walk.

A.3.5.0.2 muP: a single expectation.

In muP the readout is normalized as $(\boldsymbol{f}_t)_j = \frac{1}{n} \sum_k (n W_t^1)_{jk} (\boldsymbol{z}_t)_k$, i.e. with the LLN normalization $1/n$. The initialization monomial has $\mathbb{E}[X^{nW_0^1} X^{\boldsymbol{z}_t}] = 0$ and dies at rate $n^{-1/2}$, while every update monomial is LLN type with a deterministic prefactor ($\zeta_s \to \mathring{\zeta}_s$ a.s.). Hence at every t the entire output collapses to the single deterministic expectation $\mathbb{E}[X^{nW_t^1} X^{\boldsymbol{z}_t}]$, independent of n to leading order at any fixed η.

A.3.5.0.3 SP: an irremovable $\sqrt{n}$ mismatch.

In SP the initialization monomial in (A.131) carries exactly the CLT normalization and therefore survives as an $O(1)$ Gaussian for all t, whereas the update monomials are LLN type ((A.140)). For fixed η the latter enter at order n and diverge (compounding through $\zeta_t = L'(\boldsymbol{f}_t)$ at later steps). Especially, no choice of η can align both terms, since they differ by the fixed factor $\sqrt{n}$. The output at time t is thus intrinsically a hierarchy of terms of different probabilistic type and cannot be written as a single expectation.

Acknowledgements

I would especially like to thank Professor Rulands for giving me the opportunity to carry out my Master's thesis in his group. I am particularly grateful for his encouraging discussions, for the freedom to propose a topic of my own, and for the confidence he placed in me to pursue this project. I would further like to thank the members of his group for creating such a welcoming and supportive atmosphere, which greatly helped me throughout my Master's project and offered many valuable opportunities for exchange.

My particular thanks go to Daniel and Pascal for their guidance and supervision throughout my Master's project. Their openness and our numerous discussions greatly supported me from the first explorations to the final phase of my research project, especially as I found my way into a new research area.

Finally, I would like to thank Paul, Oskar, and Lenz for making the writing of this thesis one of the most enjoyable periods of my studies. I greatly value our scientific discussions, coffee breaks, and personal conversations during our time in the Rulands group, and I look forward to the possibility of working with you all again in the future.

Bibliography

[1] Yann LeCun, Léon Bottou, Yoshua Bengio and Patrick Haffner. Gradient-Based Learning Applied to Document Recognition. *Proceedings of the IEEE* **86**(11), 2278–2324, 1998.

[2] Kaiming He, Xiangyu Zhang, Shaoqing Ren and Jian Sun. Deep Residual Learning for Image Recognition. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2016, pp. 770–778.

[3] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser and Illia Polosukhin. Attention is All You Need. In: *Advances in Neural Information Processing Systems*. Vol. 30. 2017,

[4] Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei and Ilya Sutskever. Language Models are Unsupervised Multitask Learners. 2019.

[5] Hugo Touvron et al. *LLaMA: Open and Efficient Foundation Language Models*. 2023. arXiv: 2302.13971 [cs.CL]. URL: https://arxiv.org/abs/2302.13971.

[6] Yann LeCun, Yoshua Bengio and Geoffrey Hinton. Deep learning. *Nature* **521**(7553), 436–444, 2015. DOI: 10.1038/nature14539. URL: https://www.nature.com/articles/nature14539.

[7] Lee Sharkey et al. *Open Problems in Mechanistic Interpretability*. arXiv:2501.16496 [cs.LG]. Jan. 2025. DOI: 10.48550/arXiv.2501.16496. URL: http://arxiv.org/abs/2501.16496.

[8] Jason Yosinski, Jeff Clune, Yoshua Bengio and Hod Lipson. *How transferable are features in deep neural networks?* arXiv:1411.1792 [cs.LG]. Nov. 2014. DOI: 10.48550/arXiv.1411.1792. URL: http://arxiv.org/abs/1411.1792.

[9] Greg Yang and Edward J. Hu. *Feature Learning in Infinite-Width Neural Networks*. arXiv:2011.14522 [cs]. July 2022. DOI: 10.48550/arXiv.2011.14522. URL: http://arxiv.org/abs/2011.14522.

[10] Greg Yang, Edward J. Hu, Igor Babuschkin, Szymon Sidor, Xiaodong Liu, David Farhi, Nick Ryder, Jakub Pachocki, Weizhu Chen and Jianfeng Gao. *Tensor Programs V: Tuning Large Neural Networks via Zero-Shot Hyperparameter Transfer*. arXiv:2203.03466 [cs]. Mar. 2022. DOI: 10.48550/arXiv.2203.03466. URL: http://arxiv.org/abs/2203.03466.

[11] Greg Yang, James B. Simon and Jeremy Bernstein. *A Spectral Condition for Feature Learning*. arXiv:2310.17813 [cs]. May 2024. DOI: 10.48550/arXiv.2310.17813. URL: http://arxiv.org/abs/2310.17813.

[12] Yoshua Bengio, Aaron Courville and Pascal Vincent. *Representation Learning: A Review and New Perspectives*. arXiv:1206.5538 [cs.LG]. Apr. 2014. DOI: 10.48550/arXiv.1206.5538. URL: http://arxiv.org/abs/1206.5538.

[13] Eric J. Michaud, Ziming Liu, Uzay Girit and Max Tegmark. *The Quantization Model of Neural Scaling*. arXiv:2303.13506 [cs.LG]. Jan. 2024. DOI: 10.48550/arXiv.2303.13506. URL: http://arxiv.org/abs/2303.13506.

[14] Shikai Qiu, Lechao Xiao, Andrew Gordon Wilson, Jeffrey Pennington and Atish Agarwala. *Scaling Collapse Reveals Universal Dynamics in Compute-Optimally Trained Neural Networks*. arXiv:2507.02119 [cs]. July 2025. DOI: 10.48550/arXiv.2507.02119. URL: http://arxiv.org/abs/2507.02119.

[15] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu and Dario Amodei. *Scaling Laws for Neural Language Models*. arXiv:2001.08361 [cs]. Jan. 2020. DOI: 10.48550/arXiv.2001.08361. URL: http://arxiv.org/abs/2001.08361.

[16] Utkarsh Sharma and Jared Kaplan. *A Neural Scaling Law from the Dimension of the Data Manifold*. arXiv:2004.10802 [cs]. Apr. 2020. DOI: 10.48550/arXiv.2004.10802. URL: http://arxiv.org/abs/2004.10802.

[17] Blake Bordelon, Alexander Atanasov and Cengiz Pehlevan. How feature learning can improve neural scaling laws*. *Journal of Statistical Mechanics: Theory and Experiment* **2025**(8), 084002, 2025. DOI: 10.1088/1742-5468/adefb1. URL: https://iopscience.iop.org/article/10.1088/1742-5468/adefb1.

[18] Katie Everett et al. *Scaling Exponents Across Parameterizations and Optimizers*. arXiv:2407.05872 [cs.LG]. July 2024. DOI: 10.48550/arXiv.2407.05872. URL: http://arxiv.org/abs/2407.05872.

[19] Matthias Feurer and Frank Hutter. Hyperparameter Optimization. en. In: *Automated Machine Learning*. Ed. by Frank Hutter, Lars Kotthoff and Joaquin Vanschoren. Series Title: The Springer Series on Challenges in Machine Learning. Springer International Publishing, Cham, 2019, pp. 3–33. DOI: 10.1007/978-3-030-05318-5_1. URL: http://link.springer.com/10.1007/978-3-030-05318-5_1.

[20] Nelson Elhage et al. *Toy Models of Superposition*. arXiv:2209.10652 [cs.LG]. Sept. 2022. DOI: 10.48550/arXiv.2209.10652. URL: http://arxiv.org/abs/2209.10652.

[21] Song Mei, Andrea Montanari and Phan-Minh Nguyen. A mean field view of the landscape of two-layer neural networks. *Proceedings of the National Academy of Sciences* **115**(33), 2018. DOI: 10.1073/pnas.1806579115. URL: https://pnas.org/doi/full/10.1073/pnas.1806579115.

[22] Justin Sirignano and Konstantinos Spiliopoulos. *Mean Field Analysis of Neural Networks: A Law of Large Numbers*. arXiv:1805.01053 [math.PR]. Nov. 2019. DOI: 10.48550/arXiv.1805.01053. URL: http://arxiv.org/abs/1805.01053.

[23] Martin Erdmann, Jonas Glombitza, Gregor Kasieczka and Uwe Klemradt. *Deep Learning for Physics Research*. World Scientific, 2021. DOI: 10.1142/12294. eprint: https://worldscientific.com/doi/pdf/10.1142/12294. URL: http://deeplearningphysics.org.

[24] Yoshua Bengio. *Deep Learning of Representations: Looking Forward*. arXiv:1305.0445 [cs.LG]. June 2013. DOI: 10.48550/arXiv.1305.0445. URL: http://arxiv.org/abs/1305.0445.

[25] W. Jeffrey Johnston and Stefano Fusi. Abstract representations emerge naturally in neural networks trained to perform multiple tasks. *Nature Communications* **14**(1), 1040, 2023. DOI: 10.1038/s41467-023-36583-0. URL: https://www.nature.com/articles/s41467-023-36583-0.

[26] Arthur Jacot, Franck Gabriel and Clément Hongler. *Neural Tangent Kernel: Convergence and Generalization in Neural Networks*. arXiv:1806.07572 [cs]. Feb. 2020. DOI: 10.48550/arXiv.1806.07572. URL: http://arxiv.org/abs/1806.07572.

[27] Eugene Golikov, Eduard Pokonechnyy and Vladimir Korviakov. *Neural Tangent Kernel: A Survey*. arXiv:2208.13614 [cs]. Aug. 2022. DOI: 10.48550/arXiv.2208.13614. URL: http://arxiv.org/abs/2208.13614.

[28] Greg Yang. *Tensor Programs I: Wide Feedforward or Recurrent Neural Networks of Any Architecture are Gaussian Processes*. arXiv:1910.12478 [cs]. May 2021. DOI: 10.48550/arXiv.1910.12478. URL: http://arxiv.org/abs/1910.12478.

[29] Greg Yang. *Tensor Programs II: Neural Tangent Kernel for Any Architecture*. arXiv:2006.14548 [stat]. Nov. 2020. DOI: 10.48550/arXiv.2006.14548. URL: http://arxiv.org/abs/2006.14548.

[30] Greg Yang and Etai Littwin. *Tensor Programs IIb: Architectural Universality of Neural Tangent Kernel Training Dynamics*. arXiv:2105.03703 [cs]. May 2021. DOI: 10.48550/arXiv.2105.03703. URL: http://arxiv.org/abs/2105.03703.

[31] Yasaman Bahri, Ethan Dyer, Jared Kaplan, Jaehoon Lee and Utkarsh Sharma. Explaining neural scaling laws. *Proceedings of the National Academy of Sciences* **121**(27), e2311878121, 2024. DOI: 10.1073/pnas.2311878121. URL: https://pnas.org/doi/10.1073/pnas.2311878121.

[32] Jordan Hoffmann et al. *Training Compute-Optimal Large Language Models*. arXiv:2203.15556 [cs]. Mar. 2022. DOI: 10.48550/arXiv.2203.15556. URL: http://arxiv.org/abs/2203.15556.

[33] Lorenzo Noci, Alexandru Meterez, Thomas Hofmann and Antonio Orvieto. *Super Consistency of Neural Network Landscapes and Learning Rate Transfer*. arXiv:2402.17457 [cs]. Nov. 2024. DOI: 10.48550/arXiv.2402.17457. URL: http://arxiv.org/abs/2402.17457.

[34] Soufiane Hayou. *A Proof of Learning Rate Transfer under μP*. arXiv:2511.01734 [stat.ML]. Feb. 2026. DOI: 10.48550/arXiv.2511.01734. URL: http://arxiv.org/abs/2511.01734.

[35] Zixiang Chen, Greg Yang, Qingyue Zhao and Quanquan Gu. *Global Convergence and Rich Feature Learning in L-Layer Infinite-Width Neural Networks under μP Parametrization*. Version Number: 2. 2025. DOI: 10.48550/ARXIV.2503.09565. URL: https://arxiv.org/abs/2503.09565.

[36] Nikhil Vyas, Alexander Atanasov, Blake Bordelon, Depen Morwani, Sabarish Sainathan and Cengiz Pehlevan. *Feature-Learning Networks Are Consistent Across Widths At Realistic Scales*. en. arXiv:2305.18411 [cs]. Dec. 2023. DOI: 10.48550/arXiv.2305.18411. URL: http://arxiv.org/abs/2305.18411.

[37] Jeremy M. Cohen, Simran Kaur, Yuanzhi Li, J. Zico Kolter and Ameet Talwalkar. *Gradient Descent on Neural Networks Typically Occurs at the Edge of Stability*. arXiv:2103.00065 [cs.LG]. Nov. 2022. DOI: 10.48550/arXiv.2103.00065. URL: http://arxiv.org/abs/2103.00065.

[38] Richard Durrett. *Probability: Theory and Examples*. 5th ed. edition. Cambridge University Press, 2019. URL: https://sites.math.duke.edu/~rtd/PTE/PTE5_011119.pdf.

[39] Lenaic Chizat, Edouard Oyallon and Francis Bach. *On Lazy Training in Differentiable Programming*. arXiv:1812.07956 [math.OC]. Jan. 2020. DOI: 10.48550/arXiv.1812.07956. URL: http://arxiv.org/abs/1812.07956.

[40] V A Marčenko and L A Pastur. DISTRIBUTION OF EIGENVALUES FOR SOME SETS OF RANDOM MATRICES. *Mathematics of the USSR-Sbornik* **1**(4), 457–483, 1967. DOI: 10.1070/SM1967v001n04ABEH001994. URL: https://www.mathnet.ru/eng/sm4101.

[41] Jaehoon Lee, Yasaman Bahri, Roman Novak, Samuel S. Schoenholz, Jeffrey Pennington and Jascha Sohl-Dickstein. *Deep Neural Networks as Gaussian Processes*. arXiv:1711.00165 [stat.ML]. Mar. 2018. DOI: 10.48550/arXiv.1711.00165. URL: http://arxiv.org/abs/1711.00165.

Declaration of authorship

I, Moritz Rautenberg, declare that this thesis, titled "Universal Feature Learning Dynamics Revealed by Self-Averaging Observables", and the work presented in it are my own and have been generated by me as the result of my own original research. Where I have consulted the published or unpublished work of others, this is always clearly attributed, with an accurate and complete reference.

München,

Date	Moritz Rautenberg